

Introduction to Deep Learning

Magda Gregorová

magda.gregorova@thws.de

March 2026

Licensed according to [CC-BY-SA 4.0](#)

Contents

I. Foundations	5
1. What is Deep Learning?	5
1.1. Demonstration	5
1.2. What is Deep Learning?	5
1.3. Brief History	6
1.4. Why Now?	8
1.4.1. Data	8
1.4.2. Compute	9
1.4.3. Algorithms	9
1.4.4. Community	10
1.5. What Can Deep Learning Do?	11
1.5.1. Language and Communication	11
1.5.2. Vision and Perception	11
1.5.3. Healthcare and Medicine	11
1.5.4. Generation	11
1.5.5. Scientific Discovery	12
1.5.6. Decision Making and Control	12
2. Shallow Neural Networks	12
2.1. Supervised Learning	12
2.2. Regression and the Linear Model	13
2.3. Nonlinear Regression and Activation Functions	14
2.4. Combining ReLUs: Piecewise Linear Functions	15
2.5. 1-Hidden-Layer Neural Networks	16
2.6. Universal Approximation	17
2.7. Binary Classification	18
2.8. Biological vs Artificial Neuron	19

3. Deep Neural Networks & Training	20
3.1. From Shallow to Deep Networks	20
3.2. Layer Notation and Function Composition	21
3.3. Batch Formulation	22
3.4. The Learning Problem	23
3.5. Loss Functions	23
3.5.1. Mean Squared Error	24
3.6. Gradient Descent	24
4. Computation Graphs & Backpropagation	25
4.1. Forward and Backward Pass	25
4.2. Computation Graphs	26
4.2.1. Example: Linear Regression	26
4.2.2. Flexible Level of Detail	26
4.2.3. Flexible Operations	27
4.2.4. Scalar vs Vector Notation	27
4.3. Backpropagation	27
4.3.1. Local and Global Gradients	28
4.4. Worked Examples	28
4.4.1. Linear Regression - Symbolic	28
4.4.2. Linear Regression - Numerical	29
4.4.3. Logistic Regression	30
4.5. Multiple Neurons and Gradient Accumulation	30
4.6. Technical Implementation and Vectorisation	30
5. Optimization	31
5.1. Optimisation Problem	31
5.2. Gradient Descent	32
5.2.1. Loss Landscape	32
5.2.2. Gradient Descent on Loss Landscape	33
5.3. Stochastic and Mini-Batch Gradient Descent	34
5.3.1. Cost of Full-Batch Gradient Descent	34
5.3.2. Batch Size and SGD Spectrum	34
5.3.3. Training Loop	35
5.4. Setting Up for Optimisation	36
5.4.1. Input normalisation	36
5.4.2. Weight Initialisation	37
5.5. Momentum and Adaptive Methods	39
5.5.1. Momentum	40
5.5.2. RMSProp	40
5.5.3. Adam	41
5.6. Learning Rate Schedules	41
6. Generalization & Regularization	42
6.1. Generalisation	43
6.1.1. True Goal and Generalisation Gap	43
6.1.2. Train, Validation, and Test Split	43
6.1.3. Overfitting and Underfitting	44

6.2.	Generalisation Paradox	44
6.2.1.	Bias-Variance Tradeoff	44
6.2.2.	Limits of the Classical View	45
6.2.3.	Many Good Minima	45
6.2.4.	Flat vs Sharp Minima	45
6.2.5.	Double Descent	46
6.3.	Regularisation Techniques	47
6.3.1.	Weight Decay	47
6.3.2.	Dropout	47
6.3.3.	<code>model.train()</code> and <code>model.eval()</code>	48
6.3.4.	Data Augmentation	49
6.3.5.	Early Stopping	50
6.4.	Training Stability	51
6.4.1.	Internal Covariate Shift and Batch Normalisation	51
6.4.2.	Vanishing and Exploding Gradients	53
6.4.3.	Dying ReLU and Activation Functions	53
6.5.	Embracing Randomness	54
7.	Classification & Maximum Likelihood	55
7.1.	Classification Problem	55
7.1.1.	From Regression to Classification	55
7.1.2.	Accuracy and Why We Cannot Optimise It Directly	55
7.2.	Binary Classification	56
7.2.1.	Binary Cross-Entropy	57
7.3.	Multiclass Classification	57
7.3.1.	Categorical Cross-Entropy	58
7.4.	Maximum Likelihood	59
7.4.1.	Maximum Likelihood Principle	59
7.4.2.	Gaussian Assumption $\rightarrow$ MSE	61
7.4.3.	Bernoulli Assumption $\rightarrow$ Binary Cross-Entropy	61
7.4.4.	Categorical Assumption $\rightarrow$ Categorical Cross-Entropy	62
7.5.	Information Theory	63
7.5.1.	Information Content	63
7.5.2.	Entropy	64
7.5.3.	Binary Entropy and Coding	65
7.5.4.	Cross-Entropy	65
7.5.5.	KL Divergence	66
7.5.6.	Three Views, One Objective	67
7.6.	Practical Implementation	67
7.6.1.	PyTorch Loss Functions	68
7.6.2.	Predictions and Accuracy	68
7.7.	Advanced Considerations	69
7.7.1.	Overconfidence and Calibration	69
7.7.2.	Label Smoothing	70
7.7.3.	Class Imbalance	70
7.7.4.	Weak Labels	70

8. Convolutional Neural Networks	71
8.1. Why Not MLPs for Images?	71
8.1.1. Images as Structured Tensors	71
8.1.2. The Parameter Explosion	72
8.1.3. The Missing Inductive Bias	72
8.2. The Convolution Operation	72
8.2.1. 1D Convolution	72
8.2.2. 2D Convolution	73
8.2.3. Padding	74
8.2.4. Stride	74
8.2.5. Multiple Input Channels	75
8.2.6. Multiple Output Channels	75
8.2.7. Convolution as Sparse, Tied Matrix Multiplication	76
8.3. CNN Architecture	76
8.3.1. Receptive Field	76
8.3.2. Pooling	77
8.3.3. The Standard CNN Building Block	78
8.3.4. Full CNN Architecture: Backbone and Head	78
8.4. Classic Architectures	78
8.4.1. LeNet-5	78
8.4.2. AlexNet	79
8.4.3. VGGNet	79
8.4.4. ResNet	80
8.5. Modern Details	80
8.5.1. 1×1 Convolutions	81
8.5.2. Batch Normalisation in CNNs	81
8.5.3. Global Average Pooling	81
8.6. Summary	82
9. CNNs in Practice	82
9.1. Transfer Learning	82
9.1.1. Three Regimes	83
9.1.2. Why Pretrained Models Work	83
9.1.3. Feature Extraction vs Fine-tuning	84
9.1.4. When to Use Which	84
9.1.5. Practical Tuning	85
9.2. Practical CNN Training	85
9.2.1. Sources of Pretrained Models	85
9.2.2. Adapting a Pretrained Model	87
9.2.3. Image Transforms Pipeline	88
9.3. Beyond Classification	90
9.3.1. Object Detection	90
9.3.2. Semantic Segmentation	91
9.3.3. Instance Segmentation	92
9.3.4. Annotation Cost	93
Acknowledgements	94
References	95

Part I.

Foundations

1. What is Deep Learning?

1.1. Demonstration

► Session 1, Slides
1-12

Before any theory, consider the following experiment. A large language model - specifically Claude - is prompted with a single sentence:

“Write me a PyTorch script that trains a neural network to classify CIFAR-10 images into 10 classes. Use a simple MLP architecture. Print the training loss each epoch and plot a few example predictions with their true and predicted labels at the end.”

The model produces a complete, runnable Python script. When executed, the script downloads the CIFAR-10 dataset - 60,000 colour images of 32×32 pixels across 10 classes - trains a neural network, and achieves reasonable classification accuracy. A second prompt requesting a CNN instead of an MLP produces a different script that achieves noticeably higher accuracy on the same task.

This demonstration works. It is genuinely easy. And therein lies the problem: *what just happened?*

The script contains weights, layers, a loss function, a training loop, gradient updates. None of these were explained. The goal of this course is to open that black box - not to prompt better, but to think deeper.

1.2. What is Deep Learning?

Artificial intelligence, machine learning, and deep learning are often used interchangeably in popular discourse, but they refer to distinct and nested concepts.

► Session 1: What
is Deep Learning?

Artificial intelligence is the broadest term - it encompasses any technique that enables machines to exhibit behaviour we would consider intelligent, including rule-based expert systems, search algorithms, and learned models.

Machine learning is a subfield of AI in which systems learn from data rather than being programmed with explicit rules. Instead of writing down every decision, we provide examples and let the system discover the underlying patterns.

Deep learning is a subfield of machine learning that uses neural networks with many layers - so-called deep neural networks. The depth allows the network to learn increasingly abstract representations of the input data.

Generative AI - encompassing large language models as well as image, video, and music generation systems - is a specific and currently prominent subset of deep learning. What the general public refers to as “AI” today is almost exclusively generative AI, which sits at the innermost level of this hierarchy.

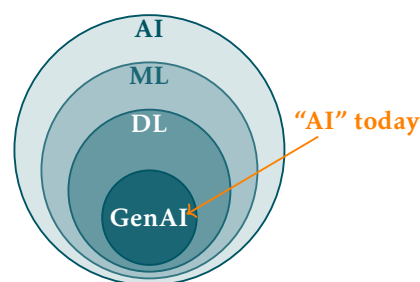


Figure 1.1: The relationship between AI, machine learning, deep learning, and generative AI.

Definition 1 (Deep Learning). Deep learning is a class of machine learning methods that use multi-layered neural networks to learn hierarchical representations directly from raw data.

This definition will sharpen as the course progresses. For now it is sufficient to hold it loosely - by the end of the course every word in it will have a concrete meaning.

1.3. Brief History

Figure 1.2 shows the key milestones in the development of deep learning alongside the periods of reduced funding and interest known as AI winters. For a comprehensive overview of the field's history see LeCun, Bengio, and G. Hinton 2015 and Goodfellow, Bengio, and Courville 2016.

► Session 1: A Brief History of Deep Learning

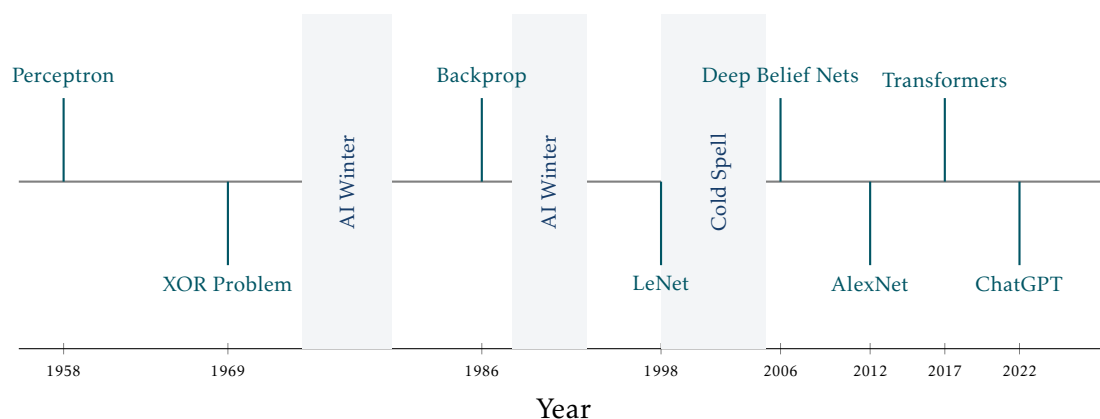


Figure 1.2: Key milestones in the history of deep learning. Shaded regions indicate AI winters and the cold spell of the early 2000s.

1958 - Perceptron. Frank Rosenblatt 1958 introduced the perceptron - the first trainable neural network. It could learn to classify linearly separable patterns by adjusting weighted connections between inputs and a single output unit. The result generated enormous excitement and equally enormous overpromising about the future of machine intelligence. Rosenblatt himself claimed the perceptron would eventually be able to walk, talk, see, write, reproduce itself, and be conscious of its existence - a level of optimism that would later contribute to the severity of the backlash.

The years following the perceptron saw genuine progress. The ADALINE model Widrow and Hoff 1960 introduced the least mean squares learning rule, which remains in use today. The concept of multilayer networks was understood theoretically, and researchers were actively exploring how to train them. There was a widespread belief that human-level artificial intelligence was only a decade away - a belief shared by many prominent scientists and echoed in generous research funding from DARPA and other agencies.

However, the limitations of single-layer networks were becoming apparent in practice. Many of the problems researchers wanted to solve - pattern recognition, language understanding, game playing - turned out to be far harder than anticipated. The gap between the promise of the perceptron and the reality of what it could achieve was widening.

1969 - XOR Problem. Minsky and Papert 1969 published *Perceptrons*, demonstrating formally that single-layer networks could not solve problems as simple as the XOR

function. The 1970s were a difficult period for neural network research. The Lighthill Report Lighthill 1973 delivered a damning assessment of AI research in 1973, concluding that the field had failed to deliver on its promises. DARPA cut funding significantly and many researchers moved away from neural networks entirely. This was the first AI winter.

Yet work continued quietly. Paul Werbos derived the backpropagation algorithm in his 1974 PhD thesis Werbos 1974 - years before it became widely known. Fukushima 1980 introduced the Neocognitron, a hierarchical neural network inspired by the visual cortex that anticipated many ideas later formalised in convolutional networks. Hopfield 1982 networks demonstrated that recurrent networks could function as associative memories. These contributions kept the field alive through the cold years, largely unnoticed by the mainstream AI community which had shifted its focus toward symbolic reasoning and expert systems.

Japan's role in this period deserves particular mention. Fukushima's Neocognitron emerged from NHK Science and Technology Research Laboratories in Tokyo. In 1982 the Japanese government launched the ambitious Fifth Generation Computer Project Ministry of International Trade and Industry 1982, investing heavily in AI hardware and parallel computing. The project caused significant alarm in the US and UK and triggered substantial counter-investment in AI research on both sides of the Atlantic. Its ultimate failure to deliver human-level reasoning contributed to the disillusionment of the second AI winter.

1986 - Backpropagation. Rumelhart, G. E. Hinton, and Williams 1986 popularised the backpropagation algorithm, providing a practical method for training multi-layer networks by propagating error gradients backwards through the layers. The revival of backpropagation generated enormous enthusiasm. Multilayer networks were successfully applied to speech recognition, image classification, and financial forecasting. LeCun applied backpropagation to convolutional networks at Bell Labs, demonstrating practical handwriting recognition systems used by the US postal service. Hochreiter and Schmidhuber introduced the Long Short-Term Memory (LSTM) network in 1997 Hochreiter and Schmidhuber 1997b, solving the vanishing gradient problem for recurrent networks and laying the foundation for sequence modelling over the following two decades.

However, the second AI winter arrived in the late 1980s and early 1990s. Expert systems - rule-based AI programs that had attracted enormous commercial investment - collapsed as their maintenance costs proved prohibitive and their brittleness became apparent. The Lisp machine market crashed in 1987. DARPA again cut funding. Neural networks hit their own practical ceiling: training deep networks remained unstable, datasets were small, and support vector machines (SVMs) - introduced by Vapnik 1995 - offered better performance on many tasks with stronger theoretical guarantees. By the mid-1990s SVMs had largely displaced neural networks as the method of choice for classification problems.

1998 - LeNet. Yann LeCun, Bottou, et al. 1998a demonstrated convolutional neural networks for handwritten digit recognition, achieving state-of-the-art results on the MNIST dataset. Despite its success, neural networks remained out of favour through the early 2000s - a period sometimes called the cold spell - as support vector machines and other kernel methods dominated the field.

This cold spell was not a complete winter - small groups continued working on deep architectures. Bengio, Delalleau, and Le Roux 2004 explored the difficulty of training

deep networks, identifying the vanishing gradient problem as the central obstacle. Hinton's group at Toronto worked on unsupervised pretraining as a way to initialise deep networks before fine-tuning with backpropagation. The theoretical groundwork for the 2006 breakthrough was being laid quietly throughout this period.

2006 - Deep Belief Nets. G. E. Hinton, Osindero, and Teh 2006 showed that deep networks could be trained effectively using a layer-wise pretraining procedure. This reignited interest in deep architectures and marked the beginning of the modern deep learning era, even before large datasets and GPU training were fully established.

The key developments of this period were infrastructural rather than algorithmic. Raina, Madhavan, and Ng 2009 demonstrated that GPUs could accelerate neural network training by an order of magnitude, making experiments that previously took weeks feasible in hours. The ImageNet dataset was released by Deng et al. 2009, providing for the first time a benchmark large and challenging enough to reveal the true potential of deep architectures. Nair and G. E. Hinton 2010 introduced the ReLU activation function, which proved dramatically more effective than sigmoid and tanh activations for training deep networks. These three developments - GPU training, large-scale data, and better activations - set the stage for AlexNet's breakthrough in 2012.

2012 - AlexNet. Krizhevsky, Sutskever, and G. E. Hinton 2012a entered a deep convolutional network trained on GPUs into the ImageNet challenge and won by a margin that shocked the field - reducing the top-5 error rate from 26% to 15% in a single year. This moment is widely regarded as the turning point that established deep learning as the dominant paradigm in machine learning.

2017 - Transformers. Vaswani et al. 2017 introduced the transformer architecture in the paper *Attention Is All You Need*, replacing recurrent computation with self-attention mechanisms. The transformer became the foundation of virtually all modern large-scale models in both language and vision.

2022 - ChatGPT. OpenAI released ChatGPT, bringing large language models to a mass audience for the first time. Within two months it had reached 100 million users, making it the fastest-growing consumer application in history and triggering a wave of public and institutional interest in artificial intelligence that continues today.

The pattern visible in Figure 1.2 is worth noting: the history of deep learning is one of recurring cycles of excitement and disappointment, punctuated by genuine breakthroughs. The current era is distinguished not just by the scale of the results but by the breadth of their impact - for the first time, deep learning systems are changing how ordinary people work and communicate on a daily basis.

1.4. Why Now?

Deep learning is not a new idea - neural networks have existed since the 1950s. What changed is the confluence of four factors that together made the approach viable at scale. None of these alone was sufficient; together they triggered a revolution.

1.4.1. Data

The internet created the largest dataset in history by accident. Every uploaded image, every written caption, every search query became potential training material. This was crystallised deliberately in benchmark datasets - ImageNet (2009) provided 1.2 million labelled images across 1,000 categories, and GLUE provided a standardised benchmark

► Session 1: Why Now? - Four Driving Forces

► Session 1: Why Now? - Four Driving Forces, Data - Scale Revolution

for natural language understanding. These benchmarks did not just measure progress, they created it by giving the community a shared target to optimise against.

Crucially, the field also learned that labels are not always necessary. Self-supervised and weakly supervised learning techniques extract signal directly from unlabelled data, massively expanding what counts as usable training material. The web, in its entirety, became a dataset.

Figure 1.3 shows the growth in dataset scale over time. Two aspects deserve attention. First, the left axis is logarithmic - each gridline represents a tenfold increase, so the jump from ImageNet to LAION-5B represents roughly four thousand times more data. Second, the orange line shows that resolution grew alongside quantity - from 28×28 pixels for MNIST to 1024×1024 for LAION-5B, a 1,000-fold increase in pixels per image. The two dimensions of scale - how many images and how much information per image - grew together.

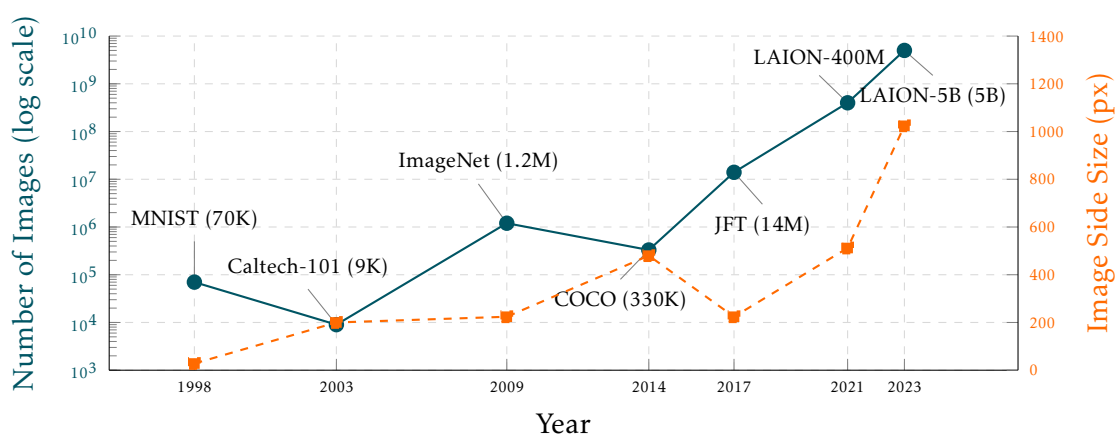


Figure 1.3: Image dataset sizes and resolutions over time. Blue line: number of images (log scale, left axis). Orange line: image side size in pixels (linear scale, right axis).

1.4.2. Compute

Graphics processing units, designed for the parallel computation of gaming graphics, turned out to be ideally suited for the matrix multiplications at the heart of neural network training. NVIDIA's CUDA programming model made GPUs accessible to researchers without hardware expertise. Custom chips - Google's TPUs, Apple Silicon, and a wave of AI-specific inference hardware - followed, with architectures designed around deep learning from the ground up.

Cloud platforms such as AWS, Microsoft Azure, and Google Cloud democratised access to large-scale compute - a researcher can now rent state-of-the-art hardware by the hour. Learning platforms such as Google Colab, Kaggle Notebooks, and Lightning.ai went further, providing free GPU access directly in the browser with zero setup. As Figure 1.4 shows, GPU performance has grown from single-digit TFLOPS in 2012 to nearly 2,000 TFLOPS in 2024.

1.4.3. Algorithms

Several targeted innovations removed practical obstacles that had blocked progress for decades. At the architectural level, the multilayer perceptron (MLP), convolutional neu-

► Session 1:
Compute &
Progress

► Session 1: Why
Now? - Four
Driving Forces,
Compute &
Progress

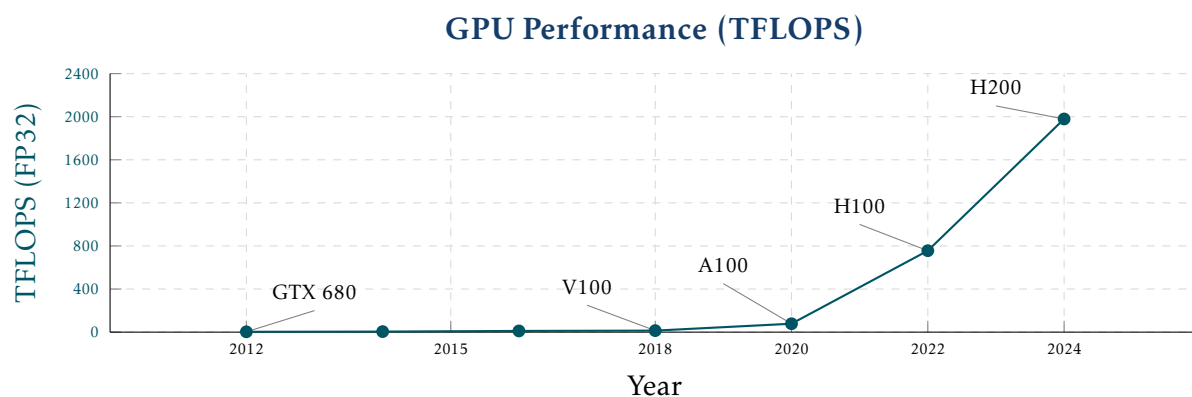


Figure 1.4: GPU performance in TFLOPS (FP32) from 2012 to 2024.

ral network (CNN), and recurrent neural network (RNN) established the foundational building blocks, all trained via the backpropagation algorithm. The ReLU activation function mitigated the vanishing gradient problem that had plagued deep networks. Batch normalisation stabilised training. Dropout provided effective regularisation.

At a higher level, residual connections (ResNets) made it possible to train networks of hundreds of layers. The attention mechanism and the transformer architecture, introduced in 2017, replaced recurrent computation entirely and became the foundation of modern language and vision models.

Figure 1.5 illustrates the impact of these algorithmic advances on the ImageNet benchmark. In 2011, the best classical methods using SVMs achieved around 74% top-5 accuracy. AlexNet's jump to 84.7% in 2012 marked the beginning of the deep learning era. Each subsequent architectural innovation - VGG, ResNet, EfficientNet, ViT - pushed accuracy further, surpassing human performance of approximately 95% around 2015 and reaching over 99% by 2022.

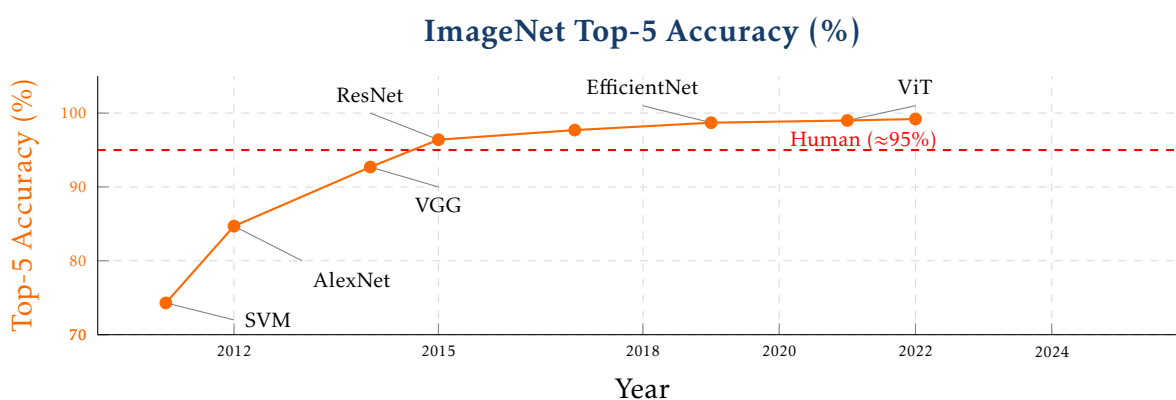


Figure 1.5: ImageNet top-5 accuracy from 2011 to 2022. Each labelled point corresponds to a key architectural innovation. The dashed red line indicates approximate human performance.

1.4.4. Community

Perhaps the most underappreciated factor is the open culture of the deep learning community. Frameworks such as PyTorch, TensorFlow, and JAX are open source,

actively maintained, and freely available. The norm of releasing code alongside papers - through platforms such as Papers with Code, Kaggle, and HuggingFace - means that results are reproducible and immediately buildable upon. Research is shared on arXiv before peer review, and major conferences such as NeurIPS, ICML, and ICLR publish proceedings openly.

This compounding effect - where each breakthrough immediately becomes everyone's foundation - has no parallel in most other scientific fields.

1.5. What Can Deep Learning Do?

Deep learning today touches almost every domain of human activity. Rather than organising applications by the models used, it is more instructive to group them by the type of task they perform.

► Session 1: What Can Deep Learning Do?

1.5.1. Language and Communication

Natural language was one of the earliest and remains one of the most impactful application areas. Machine translation systems such as DeepL and Google Translate now produce fluent output across dozens of language pairs, a task that resisted decades of rule-based approaches. Conversational agents - ChatGPT, Claude, Gemini - have brought language models to a mass audience, demonstrating capabilities in reasoning, summarisation, and question answering that were considered out of reach just a few years ago. Code generation tools such as GitHub Copilot and Cursor assist developers by completing, explaining, and generating code in real time.

1.5.2. Vision and Perception

Computer vision was the domain in which deep learning first demonstrated decisive superiority over classical methods, with AlexNet's 2012 ImageNet result marking the turning point. Today deep learning powers image classification, object detection, and semantic segmentation across a vast range of applications. Autonomous driving systems parse complex road scenes in real time, combining camera, lidar, and radar inputs to make driving decisions. Satellite and aerial imagery analysis enables large-scale monitoring of land use, deforestation, and infrastructure.

1.5.3. Healthcare and Medicine

Medical imaging has been transformed by deep learning. Systems trained on large collections of labelled scans can detect tumours, classify X-rays, and segment anatomical structures with accuracy approaching or exceeding that of specialist clinicians. Beyond imaging, deep learning is applied to genomics - identifying disease-associated variants in DNA sequences - and to clinical decision support systems that assist physicians with diagnosis and treatment planning.

1.5.4. Generation

Generative models have emerged as one of the most publicly visible applications of deep learning. Image generation systems such as Stable Diffusion and DALL-E produce photorealistic or stylised images from text descriptions. Video and music

generation - tools such as Suno and Udio - extend this capability to temporal media. The same underlying technology that enables generation also enables detection: deepfake detection systems attempt to identify synthetically generated media, an increasingly important capability as generation quality improves.

1.5.5. Scientific Discovery

Perhaps the most consequential application of deep learning to date is AlphaFold Jumper, Evans, Pritzel, et al. 2021, DeepMind's system for predicting protein three-dimensional structure from amino acid sequence. AlphaFold solved a fifty-year-old open problem in structural biology and earned Demis Hassabis and John Jumper the 2024 Nobel Prize in Chemistry - the first Nobel Prize awarded for work driven by deep learning. Deep learning is also applied to drug discovery - predicting molecular properties and identifying candidate compounds - and to climate and weather modelling, where neural networks are beginning to challenge traditional physics-based simulation approaches.

1.5.6. Decision Making and Control

Reinforcement learning - a branch of deep learning in which agents learn by interacting with an environment - has produced some of the field's most dramatic results. AlphaGo Silver, Huang, Maddison, et al. 2016 defeated the world champion Go player in 2016, a milestone considered a decade away by most experts at the time. Robotic systems learn manipulation and locomotion skills directly from experience, without hand-coded controllers. Recommendation systems - the algorithms that determine what appears in TikTok feeds, Netflix homepages, and Spotify playlists - are among the most economically impactful applications of deep learning, shaping the daily media consumption of billions of people.

2. Shallow Neural Networks

These notes accompany the second lecture of the course. The goal of this session is to build up, from first principles, the simplest possible neural network - a so-called **shallow** or **one-hidden-layer** neural network - and to understand precisely what it can represent. No optimisation is discussed here; that comes in later sessions. The question we ask is purely one of expressiveness: what class of functions can such a network compute?

► Session 2

2.1. Supervised Learning

We begin with the general framework that motivates everything that follows. In **supervised learning** we are given a dataset of input-output pairs

► Session 2:
Supervised
Learning

$$\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N \quad (2.1)$$

where each input $\mathbf{x}^{(i)} \in \mathcal{X}$ belongs to some input space $\mathcal{X}$ and each output $y^{(i)} \in \mathcal{Y}$ belongs to some output space $\mathcal{Y}$. We assume that there exists a true underlying relationship - a mapping $f^* : \mathcal{X} \rightarrow \mathcal{Y}$ such that $y = f^*(\mathbf{x})$, but we never have direct access to f^* . We only have access to the finitely many observations in the dataset $\mathcal{D}$.

The goal is to learn a parameterised function $f_\theta : \mathcal{X} \rightarrow \mathcal{Y}$ from $\mathcal{D}$ such that it approximates the true unknown mapping $f_\theta \approx f^*$. The notation $f_\theta(\mathbf{x}) = f(\mathbf{x}; \theta) = \hat{y}$ will be used throughout the course: $\hat{y}$ denotes the model's prediction for input $\mathbf{x}$ given parameters θ . The goal is not merely to fit the training data, but to predict accurately on **unseen** inputs - this is the **generalisation** requirement, which we will revisit in depth in later sessions.

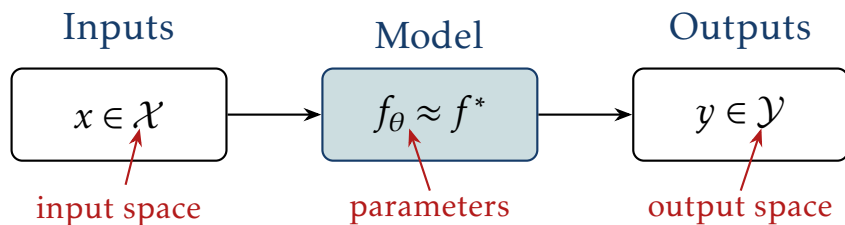


Figure 2.1: Supervised learning: given data pairs $(\mathbf{x}, y)$, learn a model f_θ that approximates the true relationship f^* .

Supervised learning encompasses two fundamental task types. In **regression**, the output space is continuous: $\mathcal{Y} = \mathbb{R}$ (or more generally $\mathbb{R}^k$), and the goal is to predict a numerical value. In **classification**, the output space is discrete, and the goal is to assign each input to one of two categories. These two tasks are illustrated in Figure 2.2 and Figure 2.3.

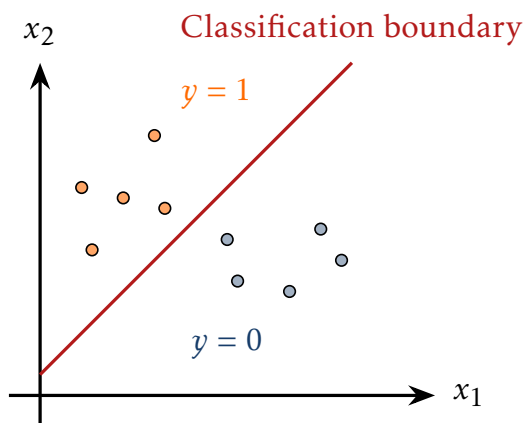


Figure 2.2: Binary classification: two classes separated by a decision boundary.

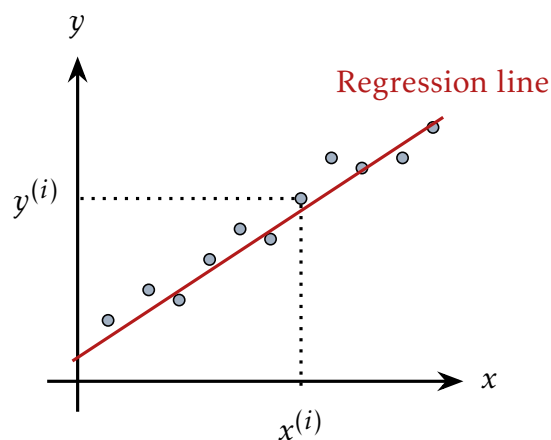


Figure 2.3: Regression: fitting a continuous function to data points.

2.2. Regression and the Linear Model

We focus first on the regression setting with a single input $x \in \mathbb{R}$ and a single output $y \in \mathbb{R}$.

The simplest parameterised function we will work with is an affine function - often called simply **linear model** in the machine learning literature:

$$\hat{y} = f_\theta(x) = \theta_1 x + \theta_0, \quad \theta = (\theta_0, \theta_1) \in \mathbb{R}^2. \quad (2.2)$$

This model has two parameters: a slope θ_1 and an intercept θ_0 and can be represented graphically as a single **neuron**, shown in Figure 2.4. The input x and the constant 1 (carrying the bias θ_0) feed into the neuron, which computes the weighted sum and produces the output $\hat{y}$. This graphical representation may seem unnecessary for such a simple model, but it will prove invaluable as we build more complex networks - the same diagram extends naturally to multiple inputs, multiple neurons, and multiple layers.

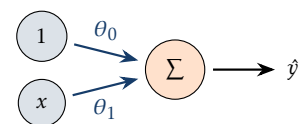


Figure 2.4: Graphical representation of the linear model as a single neuron.

The representational capacity of the linear model is severely limited - it can only fit data that lies approximately along a straight line. When the true relationship f^* is nonlinear, a linear model will fail systematically, as illustrated in Figure 2.6: it will underfit the data regardless of how the parameters are chosen. This is not a failure of optimisation; it is a failure of expressiveness. The model class is simply too small to contain the true function.

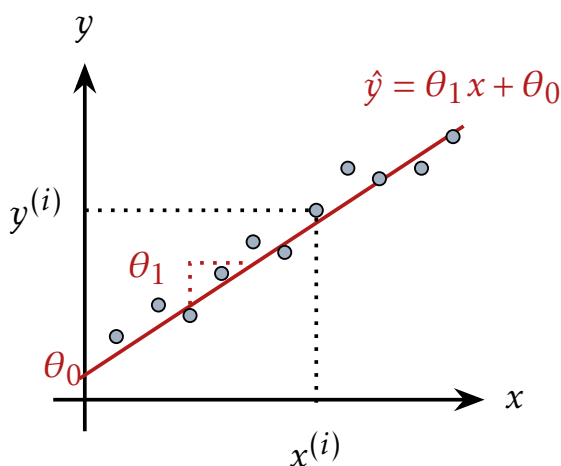


Figure 2.5: A linear model fits the data well when the true relationship is linear.

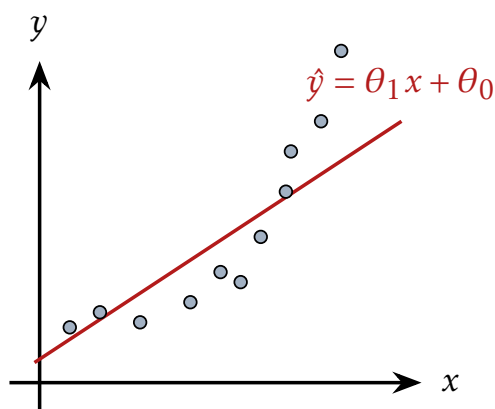


Figure 2.6: A linear model fails to capture nonlinear relationships in the data.

This observation motivates the central question of the lecture: how can we construct a richer class of functions f_θ that can capture nonlinear relationships?

2.3. Nonlinear Regression and Activation Functions

The linear model introduced in the previous section has a fundamental limitation: it can only represent affine relationships between inputs and outputs. To move beyond this, we introduce a **nonlinear activation function** $a: \mathbb{R} \rightarrow \mathbb{R}$ applied to the output of the linear computation. The neuron now computes:

$$\hat{y} = f_\theta(x) = a(\theta_1 x + \theta_0). \quad (2.3)$$

The activation function is what gives the neural network its expressive power - changing the mapping between inputs $\mathcal{X}$ and outputs $\mathcal{Y}$ from linear to nonlinear. The graphical representation of this nonlinear neuron is shown in

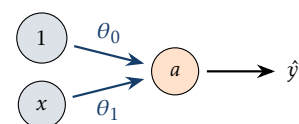
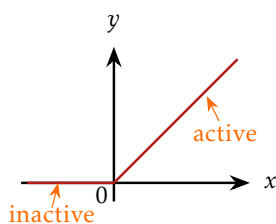


Figure 2.7: A nonlinear neuron applies activation function a to the linear combination of its inputs.

Figure 2.7: it is identical to the linear neuron, but the node now carries the symbol a to indicate that a nonlinearity is applied before the output is produced.



The choice of activation function matters. The most widely used activation function in modern deep learning is the **Rectified Linear Unit**, or **ReLU** (Nair and G. E. Hinton 2010; Glorot, Bordes, and Bengio 2011):

$$a(x) = \text{ReLU}(x) = \max(0, x) = \begin{cases} x & x > 0 \quad (\text{active}) \\ 0 & x \leq 0 \quad (\text{inactive}). \end{cases} \quad (2.4)$$

Figure 2.8: The ReLU activation function: inactive (zero) for $x \leq 0$, active (linear) for $x > 0$.

The ReLU is remarkably simple: it passes positive values unchanged and clamps negative values to zero. A **ReLU neuron** is said to be **active** when its pre-activation value is positive, and **inactive** otherwise. Despite its simplicity - or perhaps because of it - the ReLU is extremely effective in practice and will be our default activation function throughout this course. We return to a broader comparison of activation functions in a later session.

Composing the ReLU with a linear function introduces exactly one breakpoint, at $x = -\theta_0/\theta_1$, where the output transitions from inactive (zero) to active (rising linearly). This single bend is still not expressive enough to capture complex nonlinear relationships. The key insight, developed in the next section, is that a *weighted sum* of such ReLU activated functions produces a piecewise linear function with one bend per ReLU - and piecewise linear functions can approximate any continuous function.

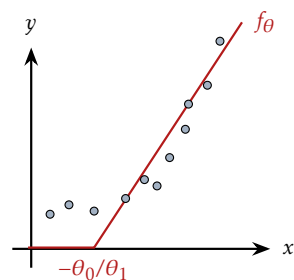


Figure 2.9: A single ReLU applied to a linear function produces one breakpoint at $x = -\theta_0/\theta_1$.

► Session 2:
Combining
Multiple ReLUs

2.4. Combining ReLUs: Piecewise Linear Functions

A single ReLU neuron introduces one breakpoint and is therefore only marginally more expressive than a linear model. The power comes from combining multiple such neurons. Consider a weighted sum of k ReLU-activated linear functions:

$$\hat{y} = f_{\theta}(x) = \theta_0 + \sum_{j=1}^k \theta_j a(\theta_{j1}x + \theta_{j0}). \quad (2.5)$$

Each term contributes one breakpoint at $x = -\theta_{j0}/\theta_{j1}$. The resulting function is **piecewise linear** - linear within each interval between breakpoints, with up to k bends in total. As k increases, the function can capture increasingly complex shapes, as illustrated in Figure 2.10.

The graphical representation of this computation for $k = 2$ ReLU activations is shown in Figure 2.11. Each **hidden neuron** j - so called because its activations are internal to the network and not directly observed as inputs or outputs - receives the input x and independently computes $h_j = a(\theta_{j1}x + \theta_{j0})$. Note that in the full representation (left), the input x and bias constant 1 are shown separately for each neuron — this makes explicit that each neuron has its own independent set of parameters $(\theta_{j1}, \theta_{j0})$, even

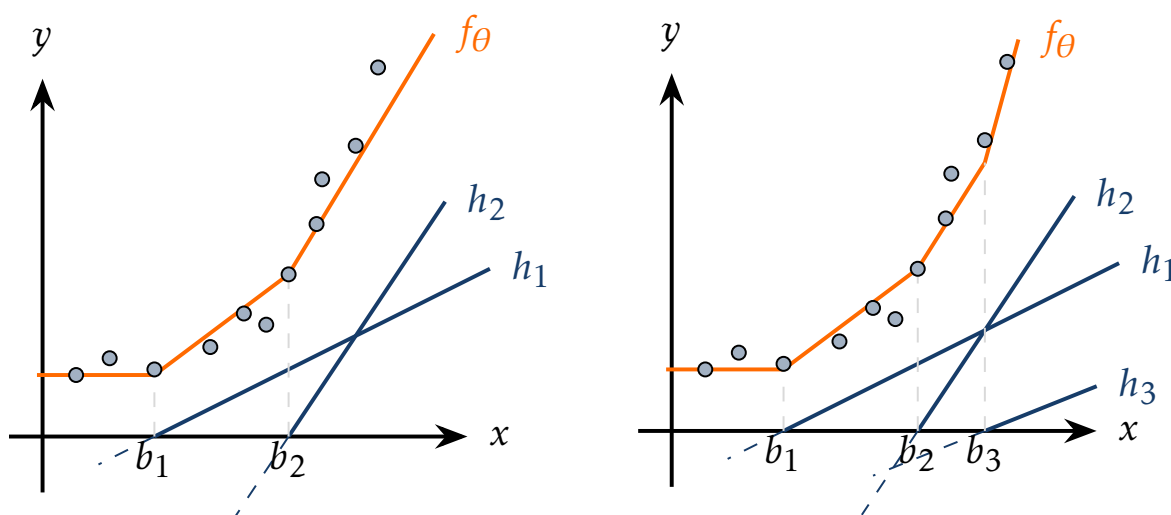


Figure 2.10: Left: combination of two ReLU functions producing two breakpoint. Right: combination of three ReLU functions producing three breakpoints. More neurons yield more expressive functions.

though they share the same input. The output neuron then forms the weighted sum $\hat{y} = \theta_0 + \theta_1 h_1 + \theta_2 h_2$. The centre diagram shows the same network with parameters omitted and the bias nodes drawn as dashed - a common convention indicating that bias inputs are always present but typically not drawn explicitly. The right diagram shows only the bare network topology. An important detail not visible in any of these diagrams: the output neuron o applies no activation function - it computes a plain weighted sum. Nonlinearity is introduced only in the hidden layer; the output is linear in the hidden activations.

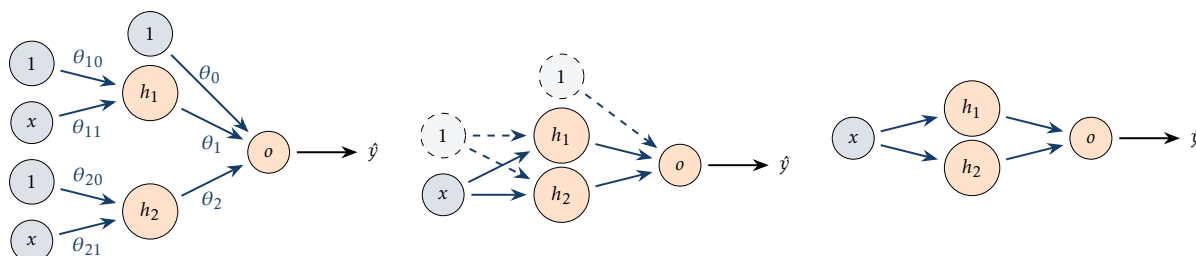


Figure 2.11: Three levels of abstraction for a two-neuron network. Left: full representation with all weights θ_{j1}, θ_{j0} and output weights labelled. Centre: simplified representation with bias nodes shown as dashed. Right: compact schematic showing topology only.

2.5. 1-Hidden-Layer Neural Networks

We now generalise from two hidden neurons to k hidden neurons, and from a single input to multiple inputs. The resulting architecture is a **one-hidden-layer neural network**, also called a **shallow neural network**.

With a single input $x \in \mathbb{R}$ and k hidden neurons, the network computes:

$$h_j = a(\theta_{j1}x + \theta_{j0}), \quad j = 1, \dots, k \quad (2.6)$$

$$\hat{y} = f_{\theta}(x) = \theta_0 + \sum_{j=1}^k \theta_j h_j. \quad (2.7)$$

Each hidden neuron j has its own parameters θ_{j1} (weight) and θ_{j0} (bias), and contributes one breakpoint to the piecewise linear output. The output neuron forms a weighted sum of all hidden activations with weights θ_j and a global bias θ_0 . The graphical representation is shown in Figure 2.12.

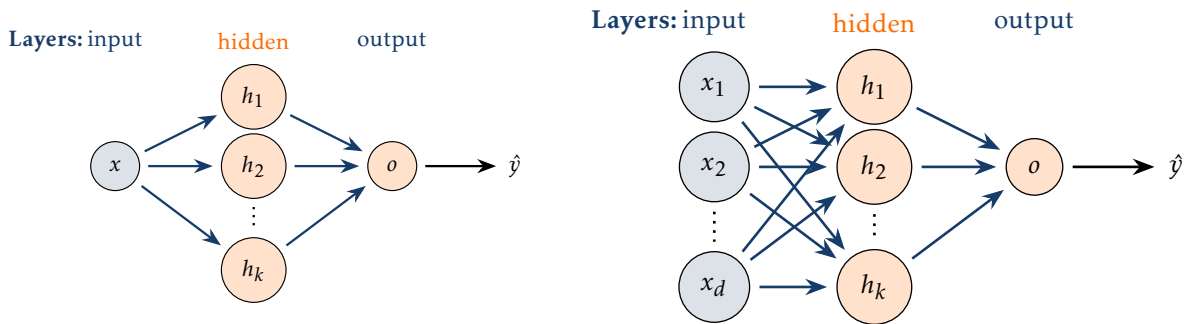


Figure 2.12: Left: one-hidden-layer network with a single input x and k hidden neurons. Right: extension to d inputs $\mathbf{x} = (x_1, \dots, x_d)$. Each hidden neuron applies a ReLU activation; the output neuron computes a plain weighted sum.

Extending to d inputs $\mathbf{x} = (x_1, \dots, x_d) \in \mathbb{R}^d$ is straightforward. Each hidden neuron now computes a linear combination of all inputs:

$$h_j = a\left(\sum_{i=1}^d \theta_{ji} x_i + \theta_{j0}\right), \quad j = 1, \dots, k \quad (2.8)$$

$$\hat{y} = f_{\theta}(\mathbf{x}) = \theta_0 + \sum_{j=1}^k \theta_j h_j. \quad (2.9)$$

The weight θ_{ji} connects input x_i to hidden neuron j , and θ_{j0} is the bias of neuron j . The output formula is unchanged — the hidden activations h_j are combined into a single prediction $\hat{y}$ regardless of the input dimensionality.

The collection of all parameters — the weights θ_{ji} , biases θ_{j0} , output weights θ_j , and output bias θ_0 — is denoted collectively as θ . Learning this network means finding the values of θ that make $f_{\theta}(\mathbf{x}) \approx f^*(\mathbf{x})$ on the training data.

2.6. Universal Approximation

The piecewise linear intuition suggests that a shallow network with enough neurons can approximate any reasonable function. This is made precise by the **Universal Approximation Theorem**.

Theorem 1 (Universal Approximation (Cybenko 1989; Hornik 1991)). *For any continuous function $g : \mathcal{X} \rightarrow \mathbb{R}$ defined on a compact set $\mathcal{X} \subseteq \mathbb{R}^d$ and any $\varepsilon > 0$, there exists a one-hidden-layer neural network f_θ such that*

$$\sup_{\mathbf{x} \in \mathcal{X}} |f_\theta(\mathbf{x}) - g(\mathbf{x})| < \varepsilon. \quad (2.10)$$

This result was first proved by Cybenko 1989 for sigmoid activations and later extended by Hornik 1991 to a broad class of activation functions. We do not prove this result here as it relies on advanced math concepts from measure theory and the Stone-Weierstrass theorem. The piecewise linear intuition developed in the previous section - more neurons, more breakpoints, better approximation - provides the geometric essence of why the theorem holds.

The theorem is powerful: it says that the class of shallow neural networks is dense in the space of continuous functions. No matter what continuous function you want to approximate, a shallow network can get arbitrarily close. However, the theorem has important limitations that are easily overlooked. It guarantees existence - there *exists* a network that approximates g - but says nothing about how large that network must be, or how to find its parameters. In the worst case, exponentially many neurons may be required. The theorem also says nothing about generalisation: a network that perfectly approximates f^* on $\mathcal{X}$ may still fail on out-of-distribution inputs. Depth, which the theorem ignores, turns out to matter enormously in practice - deeper networks can represent the same functions far more efficiently than shallow ones, a point we return to later.

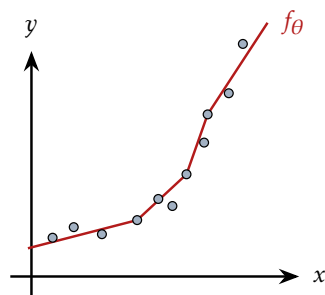


Figure 2.13: A piecewise linear function approximating a nonlinear curve.

2.7. Binary Classification

We now turn to the classification setting. In **binary classification**, the output space is $\mathcal{Y} = \{0, 1\}$, and the goal is to assign each input $\mathbf{x} \in \mathcal{X}$ to one of two classes.

The simplest approach is a **linear classifier**: predict class 1 if a linear function of the input exceeds zero, and class 0 otherwise:

$$\hat{y} = \mathbf{1} \left[\theta_0 + \sum_{i=1}^d \theta_i x_i \geq 0 \right]. \quad (2.11)$$

The set of points satisfying $\theta_0 + \sum_i \theta_i x_i = 0$ is a hyperplane in $\mathbb{R}^d$ - the **decision boundary**. The weight vector $(\theta_1, \dots, \theta_d)$ is normal to this hyperplane, and θ_0 controls its offset from the origin. In two dimensions, the decision boundary is a line; in three dimensions, a plane; in general, a $(d - 1)$ -dimensional hyperplane.

A linear classifier can only represent **linearly separable** problems - those in which the two classes can be separated by a single hyperplane. Many real problems are not linearly separable. A simple example: consider two clusters of one class located at the top-left and bottom-right of the input space, with a cluster of the other class in the middle. No straight line can separate them.

The solution is exactly analogous to the regression case: replace the linear function with a neural network. The decision boundary $f_{\theta}(\mathbf{x}) = 0$ becomes a nonlinear surface, capable of carving out arbitrarily complex regions of the input space:

$$\hat{y} = \mathbf{1}[f_{\theta}(\mathbf{x}) \geq 0]. \quad (2.12)$$

The same shallow network architecture used for regression - a linear combination of ReLU-activated linear functions - can serve as f_{θ} here. The universal approximation theorem applies equally to classification.

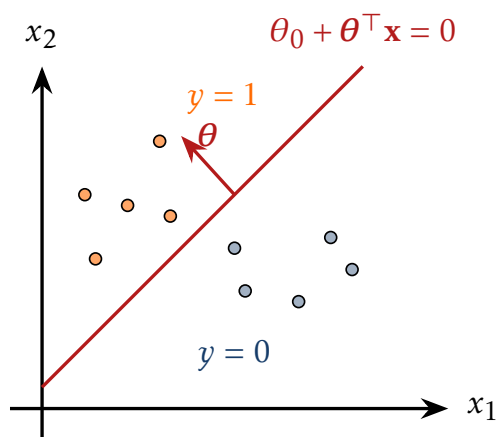


Figure 2.14: A linear classifier separates two classes with a hyperplane. The weight vector θ is normal to the decision boundary.

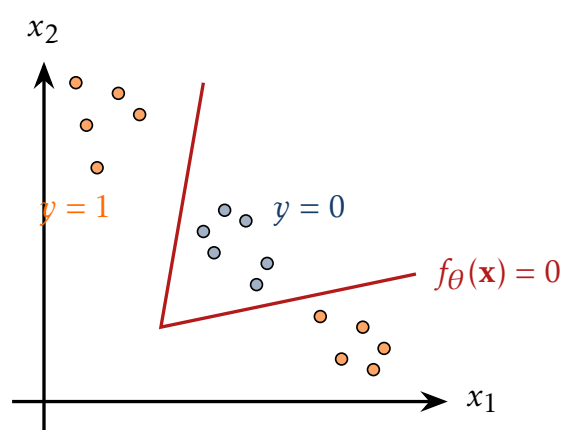


Figure 2.15: A nonlinearly separable dataset. A neural network with a non-linear decision boundary $f_{\theta}(\mathbf{x}) = 0$ is required.

2.8. Biological vs Artificial Neuron

Having built up the neural network entirely from mathematical and computational considerations, it is worth pausing to examine the biological analogy that motivates the terminology.

A biological neuron illustrated in Figure 2.16 receives signals from other neurons via **dendrites**, integrates these signals in the **cell body** (soma), and fires an electrical pulse along its **axon** if the integrated signal exceeds a threshold. The artificial neuron in Figure 2.17 mimics this schematically: inputs $\mathbf{x}$ play the role of dendrites, the weighted sum $\sum_i \theta_i x_i + \theta_0$ plays the role of integration in the soma, and the activation function a plays the role of the firing threshold.

The analogy is useful as a narrative device but should not be taken literally. Real neurons are vastly more complex than the linear-threshold model: they operate in continuous time, communicate via spike trains rather than real-valued signals, exhibit rich temporal dynamics, and are embedded in three-dimensional physical structures that constrain their connectivity. The artificial neuron strips all of this away, retaining only the most abstract essence of the computation.

Indeed, the mathematical objects we have been working with - piecewise linear functions, weighted sums, nonlinear activations - were derived entirely from the regression and classification problems, without any reference to biology. The biological inspiration played a historical role in motivating the research programme, but the justification for using neural networks today is empirical and mathematical, not biological.

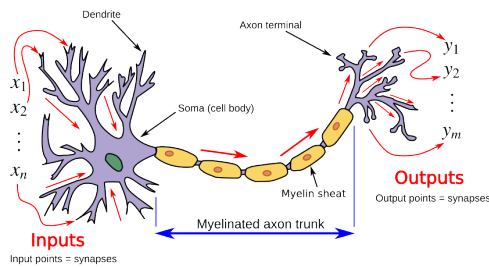


Image:

Egm4313.s12 at English Wikipedia, CC BY-SA 3.0, via Wikimedia Commons

Figure 2.16: A biological neuron with dendrites, soma, and axon.

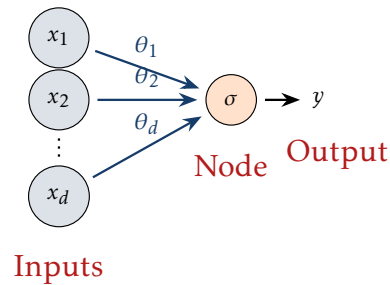


Figure 2.17: The artificial neuron: a weighted sum of inputs followed by a non-linear activation function.

3. Deep Neural Networks & Training

These notes accompany the third lecture. In Session 2 we established that a one-hidden-layer network with ReLU activations can approximate any continuous function - but we stopped short of asking how to actually *find* good parameters. This session addresses that question. We first generalise the architecture to arbitrary depth, then introduce the machinery needed to train it: a loss function to measure error, and gradient descent to minimise it.

» Session 3

3.1. From Shallow to Deep Networks

The one-hidden-layer network from Session 2 computes, for input $\mathbf{x} \in \mathbb{R}^d$:

$$h_j = a \left(\theta_{j0} + \sum_{i=1}^d \theta_{ji} x_i \right), \quad j = 1, \dots, k \quad (3.1)$$

$$\hat{y} = \theta_0 + \sum_{j=1}^k \theta_j h_j. \quad (3.2)$$

» Session 3: From Shallow to Deep, Matrix Notation

The natural extension is to stack additional layers: the output of one layer becomes the input of the next. Each layer applies a linear transformation followed by a nonlinear activation. Stacking layers allows the network to learn increasingly abstract representations - the first layer might detect simple patterns in the input, the second layer combinations of those patterns, and so on. Figure 3.1 contrasts the shallow network from Session 2 with a two-hidden-layer deep network.

To handle multiple layers cleanly, scalar notation becomes cumbersome. We start by collecting the individual quantities into vectors. The inputs $x_1, \dots, x_d$ are gathered into an input vector $\mathbf{x} \in \mathbb{R}^d$, and the hidden neuron activations $h_1, \dots, h_k$ into a hidden activation vector $\mathbf{h} \in \mathbb{R}^k$. Correspondingly, the weights and biases are collected into a matrix and a vector. The bias of each output neuron j gives a bias vector $\boldsymbol{\theta}_0 \in \mathbb{R}^k$, and the weights connecting all d input neurons to all k output neurons form a weight matrix $\boldsymbol{\Theta} \in \mathbb{R}^{k \times d}$:

$$\boldsymbol{\Theta} = \begin{pmatrix} \theta_{11} & \cdots & \theta_{1d} \\ \vdots & \ddots & \vdots \\ \theta_{k1} & \cdots & \theta_{kd} \end{pmatrix} \in \mathbb{R}^{k \times d}, \quad \boldsymbol{\theta}_0 = \begin{pmatrix} \theta_{10} \\ \vdots \\ \theta_{k0} \end{pmatrix} \in \mathbb{R}^k. \quad (3.3)$$

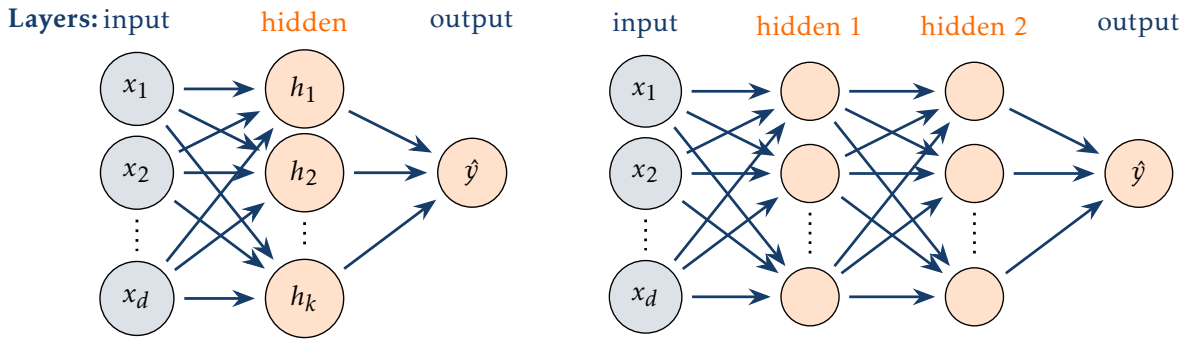


Figure 3.1: Left: shallow network with one hidden layer. Right: deep network with two hidden layers. Each additional layer introduces another linear transform followed by a nonlinearity.

The matrix Θ has k rows and d columns. Each row j contains the weights $(\theta_{j1}, \dots, \theta_{jd})$ connecting all d input neurons to output neuron j - so there is one row per output neuron. Each column i contains the weights $(\theta_{1i}, \dots, \theta_{ki})$ connecting input neuron x_i to all k output neurons - so there is one column per input neuron. The bias vector $\theta_0 \in \mathbb{R}^k$ has one entry per output neuron.

With this notation, the entire hidden layer computation collapses to a single vector equation:

$$\mathbf{h} = a(\theta_0 + \Theta \mathbf{x}) \in \mathbb{R}^k, \quad (3.4)$$

where a is applied elementwise. The matrix-vector product $\Theta \mathbf{x} \in \mathbb{R}^{k \times d} \cdot \mathbb{R}^d = \mathbb{R}^k$ produces one pre-activation value per output neuron. Adding the bias $\theta_0 \in \mathbb{R}^k$ and applying a elementwise gives the hidden activation vector $\mathbf{h} \in \mathbb{R}^k$.

Figure 3.2: Hidden layer computation as matrix-vector operation.

3.2. Layer Notation and Function Composition

We index layers $l = 0, 1, \dots, L$, where $l = 0$ denotes the input and $l = L$ the output. The forward pass is defined recursively:

$$\mathbf{h}^{(0)} = \mathbf{x}, \quad \mathbf{h}^{(l)} = a\left(\theta_0^{(l)} + \Theta^{(l)} \mathbf{h}^{(l-1)}\right), \quad l = 1, \dots, L-1, \quad (3.5)$$

with output

$$\hat{y} = \theta_0^{(L)} + \Theta^{(L)} \mathbf{h}^{(L-1)}. \quad (3.6)$$

The output layer applies no activation function - the raw linear output $\hat{y}$ is passed directly as the network's output. This is the general convention for all network types: for regression, $\hat{y}$ is used directly as the prediction; for classification, $\hat{y}$ contains the raw logits to which a task-specific transformation (sigmoid, softmax) is applied afterwards, outside

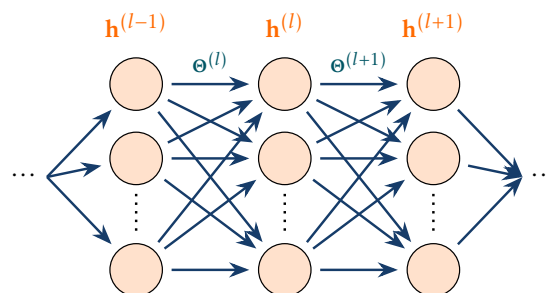


Figure 3.3: Layer notation.

the network. The layer structure is illustrated in Figure 3.3.

At layer l , the weight matrix has shape $\Theta^{(l)} \in \mathbb{R}^{k_l \times k_{l-1}}$ and the bias vector $\theta_0^{(l)} \in \mathbb{R}^{k_l}$, with $k_0 = d$. The matrix-vector product $\Theta^{(l)} \mathbf{h}^{(l-1)} \in \mathbb{R}^{k_l \times k_{l-1}} \cdot \mathbb{R}^{k_{l-1}} = \mathbb{R}^{k_l}$ produces one pre-activation value per output neuron in layer l .

Viewed as a whole, the deep network is a composition of functions:

$$\hat{y} = f_{\theta}(\mathbf{x}) = \left(h_{\theta}^{(L)} \circ \dots \circ h_{\theta}^{(1)} \right)(\mathbf{x}) = \theta_0^{(L)} + \Theta^{(L)} a \left(\dots a \left(\theta_0^{(1)} + \Theta^{(1)} \mathbf{x} \right) \right). \quad (3.7)$$

The total number of parameters is the sum over all layers of the number of weights plus biases:

$$|\theta| = \sum_{l=1}^L k_l \cdot (k_{l-1} + 1). \quad (3.8)$$

For example, the small network in Figure 3.4 with $d = 3$ inputs, two hidden layers of width $k_1 = k_2 = 4$, and $m = 2$ outputs has:

$$\begin{aligned} & 4 \cdot (3 + 1) + 4 \cdot (4 + 1) + 2 \cdot (4 + 1) \\ & = 16 + 20 + 10 = 46 \text{ parameters.} \end{aligned} \quad (3.9)$$

This is a toy example. In practice, networks are much wider and deeper - parameter counts grow quadratically with layer width and linearly with depth. A fully-connected network with $d = 784$ inputs (a 28×28 image), two hidden layers of width 1024, and 10 outputs already has approximately 1.8 million parameters. Modern architectures are far larger: ResNet-50, a standard image classification model, has 25 million parameters; GPT-2 has 1.5 billion; GPT-3 has 175 billion. Finding good values for all parameters jointly from data is the central challenge of deep learning.

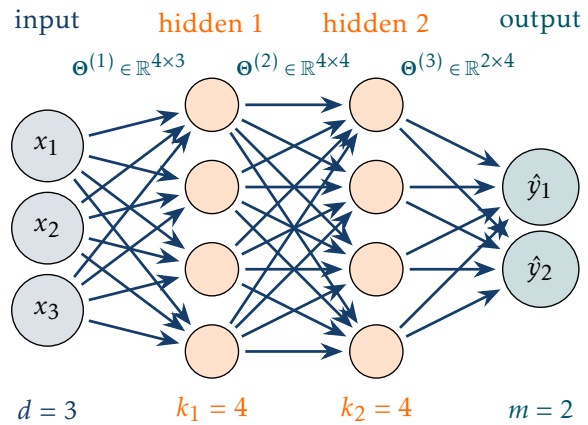


Figure 3.4: Network with $d = 3$, $k_1 = k_2 = 4$, $m = 2$. Total: $16 + 20 + 10 = 46$ parameters.

3.3. Batch Formulation

In practice, we process multiple training samples simultaneously in a **batch**. Given N samples, we stack the inputs as rows of a matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$, and the hidden activations of layer l form a matrix $\mathbf{H}^{(l)} \in \mathbb{R}^{N \times k_l}$.

The batch forward pass for layer l is:

$$\mathbf{H}^{(l)} = \mathbf{1} \theta_0^{(l)\top} + \mathbf{H}^{(l-1)} \cdot \left(\Theta^{(l)} \right)^\top, \quad (3.10)$$

$N \times k_l$ $N \times k_l$ $N \times k_{l-1}$ $k_{l-1} \times k_l$

Figure 3.5: Batch forward pass.

what PyTorch computes with $H = X @ W.T + b$: the batch dimension comes first, and the matrix multiply operates across the feature dimension.

Processing data in batches is computationally efficient: modern hardware (GPUs, TPUs) is designed for large parallel matrix operations, and a single batched matrix multiply is far faster than N sequential vector operations.

3.4. The Learning Problem

We now have a network f_θ and a dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$. The goal is to find parameters θ such that $f_\theta(\mathbf{x}) \approx y$ on unseen data. This raises two immediate questions:

► Session 3: The Learning Problem

1. How do we **measure** how wrong f_θ is? $\Rightarrow$ *loss function*
2. How do we **find** good θ systematically? $\Rightarrow$ *optimisation*

As illustrated in Figure 3.6, there are infinitely many functions that could plausibly fit the training data - they differ in how they interpolate between observed points and how they extrapolate beyond them. With a low-dimensional input we could perhaps inspect candidate functions visually. But f_θ maps from $\mathbb{R}^d$ where d may be hundreds or thousands - visual inspection is impossible. Trial and error is equally hopeless: the parameter space has millions of dimensions, and randomly trying parameter configurations would never converge to anything useful.

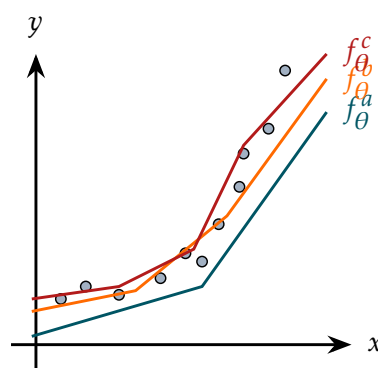


Figure 3.6: Many functions fit the data. Which is best?

What we need is a single scalar number that summarises how wrong the current f_θ is on the training data - small when the predictions are good, large when they are bad. This scalar is the **loss**, and it turns the vague goal of “find a good function” into a concrete mathematical optimisation problem: minimise the loss over all possible parameter settings.

3.5. Loss Functions

A **loss function** $\ell(\hat{y}, y)$ measures the discrepancy between a prediction $\hat{y}$ and a true label y for a single sample. It satisfies $\ell(\hat{y}, y) = 0$ when $\hat{y} = y$ and $\ell(\hat{y}, y) > 0$ otherwise, growing larger as the prediction worsens. The loss is the compass that guides training: it converts the vague notion of “a good model” into a concrete scalar that can be minimised.

For a full dataset, we average the per-sample loss over all training examples to get the **dataset loss**:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_\theta(\mathbf{x}^{(i)}), y^{(i)}). \quad (3.11)$$

Averaging rather than summing makes $\mathcal{L}$ independent of dataset size, which is convenient when comparing runs with different amounts of data or different batch sizes.

The choice of ℓ depends on the task. For regression we use Mean Squared Error; for binary classification, Binary Cross-Entropy; for multiclass classification, Cross-Entropy with a softmax output. Their probabilistic motivation via maximum likelihood

► Session 3: What is a Loss Function?, Regression: Mean Squared Error

estimation is developed in a later session. Here we focus on MSE for regression; classification losses are covered when we discuss classification in detail.

3.5.1. Mean Squared Error

For regression with scalar output $y \in \mathbb{R}$ and prediction $\hat{y} \in \mathbb{R}$, the per-sample loss is the squared residual:

$$\ell(\hat{y}, y) = (\hat{y} - y)^2. \quad (3.12)$$

The dataset loss is accordingly:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)})^2. \quad (3.13)$$

As shown in Figure 3.7, the residuals $\hat{y}^{(i)} - y^{(i)}$ are the vertical distances between the predicted function and the data points. MSE averages the squares of these distances. Three properties make squaring the natural choice. First, prediction errors can be positive or negative and would cancel if averaged directly - squaring removes the sign. Second, squaring penalises large errors more heavily than small ones: a prediction off by 10 contributes 100 times more to the loss than one off by 1. Third, the squared function is differentiable everywhere, which is essential for gradient-based optimisation.

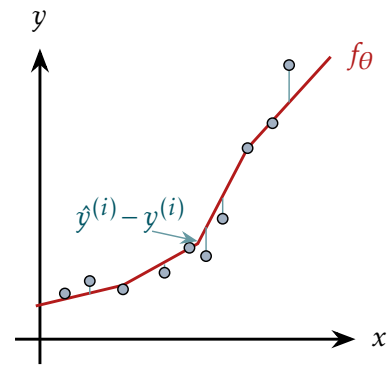


Figure 3.7: MSE loss. Vertical lines show residuals $\hat{y}^{(i)} - y^{(i)}$.

3.6. Gradient Descent

Training the network means finding parameters that minimise the dataset loss:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta). \quad (3.14)$$

Unlike linear regression, no closed-form solution exists: $\mathcal{L}(\theta)$ is highly nonlinear in θ through the nested compositions of weight matrices and activations, the parameter space has millions of dimensions, and the loss surface is non-convex with many local minima and saddle points. We need an iterative strategy that navigates this landscape using only local information.

The key local information at any point θ is the **gradient** $\nabla_{\theta} \mathcal{L}(\theta)$ - the vector of partial derivatives of $\mathcal{L}$ with respect to each parameter:

$$\nabla_{\theta} \mathcal{L}(\theta) = \frac{\partial \mathcal{L}}{\partial \theta} = \left(\frac{\partial \mathcal{L}}{\partial \theta_1}, \frac{\partial \mathcal{L}}{\partial \theta_2}, \dots \right). \quad (3.15)$$

The gradient points in the direction of steepest *ascent* of the loss surface. To *minimise* the loss we step in the opposite direction, giving the **gradient descent** update rule:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta), \quad (3.16)$$

where $\eta > 0$ is the **learning rate**, controlling the step size. Starting from a random initialisation, we repeatedly apply this update until the loss stops improving.

Figure 3.8 illustrates gradient descent on a one-dimensional quadratic loss $\mathcal{L}(\theta) = \theta^2$. Starting from $\theta_0 > 0$, the gradient $\frac{\partial \mathcal{L}}{\partial \theta} = 2\theta$ is positive, so the update $\theta \leftarrow \theta - \eta \cdot 2\theta$ moves θ to the left, towards the minimum at $\theta^* = 0$. As θ decreases, the gradient decreases too, so subsequent steps are naturally smaller even with a fixed learning rate — the algorithm automatically slows down as it approaches the optimum.

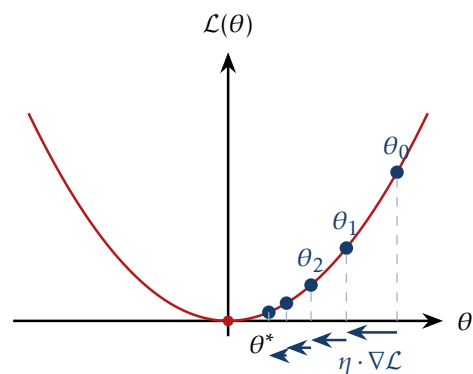


Figure 3.8: Gradient descent on a quadratic loss. Steps from θ_0 towards minimum θ^* .

The learning rate η is a critical hyperparameter. Too large and the updates overshoot the minimum, bouncing back and forth or even diverging. Too small and convergence is impractically slow. In practice, η is carefully tuned and often adapted during training - a topic we return to in the dedicated optimisation session.

The central computational challenge is evaluating $\nabla_{\theta} \mathcal{L}(\theta)$ efficiently. The loss has millions of components in θ , and computing each partial derivative $\frac{\partial \mathcal{L}}{\partial \theta_j}$ separately would require one forward pass per parameter - completely intractable. The answer is **backpropagation**: an algorithm that computes all partial derivatives simultaneously in a single backward pass, at a cost comparable to a single forward pass. We develop it in the next session.

4. Computation Graphs & Backpropagation

These notes accompany the fourth lecture. The central question left open at the end of Session 3 was: how do we compute $\nabla_{\theta} \mathcal{L}(\theta)$ efficiently for a deep network? The answer - backpropagation - is the algorithmic backbone of all modern deep learning. We develop it here through the framework of computation graphs.

► Session 4

4.1. Forward and Backward Pass

Training a deep network requires two passes through the network for each update step. The **forward pass** evaluates the network layer by layer from input to output, computing the prediction $\hat{y}$ and then the loss $\mathcal{L}$:

$$\mathbf{x} = \mathbf{h}^{(0)} \xrightarrow{\Theta^{(1)}} \mathbf{h}^{(1)} \xrightarrow{\Theta^{(2)}} \dots \xrightarrow{\Theta^{(L)}} \mathbf{h}^{(L)} = \hat{y} \xrightarrow{\ell} \mathcal{L}. \quad (4.1)$$

The **backward pass** propagates gradients in the reverse direction, from the loss back through each layer to the parameters:

$$\frac{\partial \mathcal{L}}{\partial \theta} \xleftarrow{\Theta^{(1)}} \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(1)}} \xleftarrow{\Theta^{(2)}} \dots \xleftarrow{\Theta^{(L)}} \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(L)}} \xleftarrow{\ell} \mathcal{L}. \quad (4.2)$$

The **gradient** $\nabla_{\theta} \mathcal{L}(\theta) = \frac{\partial \mathcal{L}}{\partial \theta}$ collects one partial derivative per parameter. Backpropagation is the algorithm that computes this backward pass efficiently. It is important to note that backpropagation *computes gradients* - it does not perform any optimisation. The gradient descent update step is separate.

► Session 4:
Forward and
Backward Pass

4.2. Computation Graphs

A **computation graph** is a directed acyclic graph in which each node represents an elementary operation and each edge represents the flow of data between operations. Decomposing a complex function into a computation graph makes gradient computation tractable for two reasons. First, each atomic operation has a simple, easy-to-compute local derivative. Second, intermediate values computed during the forward pass can be stored and reused during the backward pass, avoiding redundant computation.

A computation graph has three types of nodes:

- **Input nodes** - hold fixed data values (x , y). They store a value but have no gradient.
- **Parameter nodes** - hold learnable parameters (θ). They store a value and accumulate a gradient.
- **Compute nodes** - implement elementary operations. They store a value, accumulate a gradient, and know how to compute both their forward function and their local gradient.

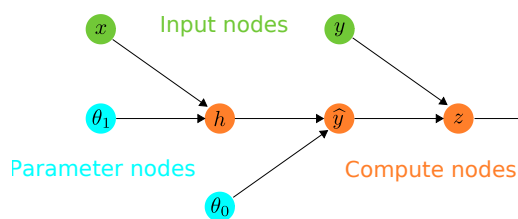


Figure 4.1: Computation graph.

4.2.1. Example: Linear Regression

Consider linear regression with MSE loss: $\ell(x, y, \theta) = (\theta_0 + \theta_1 x - y)^2$. Figure 4.1 shows the computation graph. We decompose this into four atomic steps: (1) $h = \theta_1 x$, (2) $\hat{y} = \theta_0 + h$, (3) $z = \hat{y} - y$, (4) $\ell = z^2$. Each step is a simple operation - multiplication, addition, subtraction, squaring - with a derivative that can be written down immediately.

4.2.2. Flexible Level of Detail

The same function can be decomposed at different levels of granularity, as illustrated in Figure 4.2. The four-step decomposition above is fine-grained; we could also use a coarser two-step decomposition: (1) $\hat{y} = \theta_0 + \theta_1 x$, (2) $\ell = (\hat{y} - y)^2$, or a vectorised version with $\hat{y} = \theta^\top \mathbf{x}$. The choice of granularity is a design decision: finer decompositions make each local gradient simpler, while coarser ones reduce the number of nodes and may be more efficient if the compound derivative is known analytically.

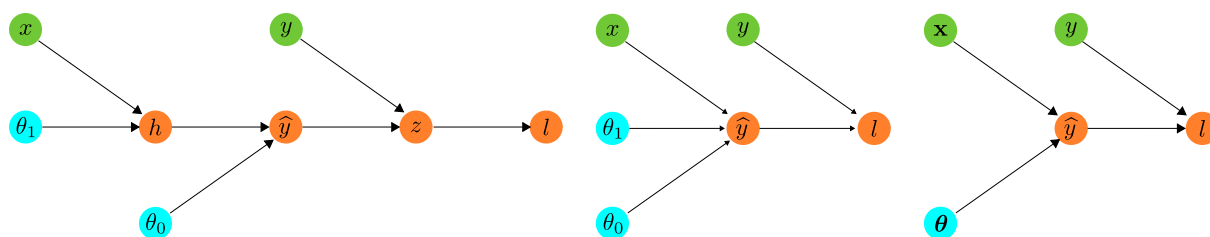


Figure 4.2: Three levels of decomposition for the same function $\ell = (\theta_0 + \theta_1 x - y)^2$. Left: fine-grained (4 nodes). Centre: coarser (2 nodes). Right: vectorised.

4.2.3. Flexible Operations

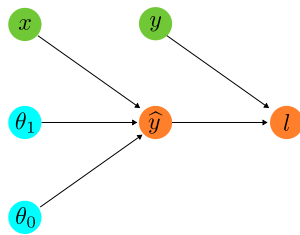


Figure 4.3: Same graph, different node functions.

The same graph structure can represent different models by changing the node functions. As shown in Figure 4.3, a two-node graph covers both linear regression with MSE and logistic regression with BCE - only the functions at each node differ.

Linear regression + MSE: (1) $\hat{y} = \theta_0 + \theta_1 x$, (2) $\ell = (\hat{y} - y)^2$

Logistic regression + BCE: (1) $\hat{y} = \sigma(\theta_0 + \theta_1 x)$, (2) $\ell = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$

This modularity is the foundation of modern deep learning frameworks, which implement a library of differentiable operations that can be composed freely.

4.2.4. Scalar vs Vector Notation

Computation graphs can be drawn at the level of scalar operations - one node per neuron, one edge per weight - or at the level of vector operations - one node per layer, as shown in Figure 4.4. Scalar graphs make the gradient flow explicit; vector graphs are more compact and correspond directly to the matrix operations implemented in practice. The two representations compute the same quantities; the vector form simply groups many scalar operations into a single matrix node.

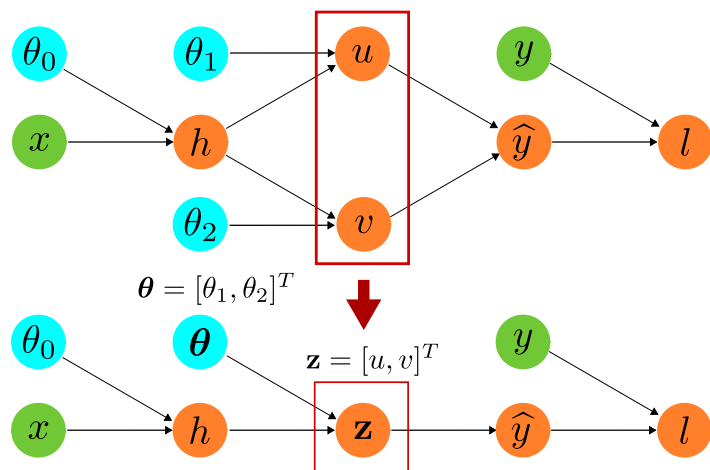


Figure 4.4: Scalar (top) and vector (bottom) computation graphs.

4.3. Backpropagation

Backpropagation relies on two rules from calculus.

Remark 1 (Leibniz notation). We use **Leibniz notation**: $\frac{df}{dx}$ denotes the derivative of f with respect to x , and $\frac{\partial f}{\partial x}$ the partial derivative when f depends on multiple variables. This is equivalent to the prime notation $f'(x)$ but makes the variable of differentiation explicit, which is essential when tracking gradients through multiple composed functions.

Sum rule. The gradient of an average is the average of the gradients:

$$\nabla_{\theta} \mathcal{L}(\theta) = \nabla_{\theta} \frac{1}{N} \sum_{i=1}^N \ell(\mathbf{x}^{(i)}, y^{(i)}, \theta) = \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \ell(\mathbf{x}^{(i)}, y^{(i)}, \theta). \quad (4.3)$$

This has an important practical consequence: we only need to work out how to compute the gradient for a *single* training example $(\mathbf{x}^{(i)}, y^{(i)})$. Once we know $\nabla_{\theta} \ell(\mathbf{x}^{(i)}, y^{(i)}, \theta)$ for one example, the gradient for the full dataset is simply the average of these individual gradients. This means the entire complexity of backpropagation reduces to differentiating through the network for a single sample.

Chain rule. For a composed function $f(g(x))$:

$$\frac{d}{dx} f(g(x)) = \frac{df}{dg} \frac{dg}{dx}. \quad (4.4)$$

When a variable x feeds into multiple downstream nodes $g_1(x), \dots, g_m(x)$, the chain rule generalises to:

$$\frac{d}{dx} f(g_1(x), \dots, g_m(x)) = \sum_{i=1}^m \frac{\partial f}{\partial g_i} \frac{dg_i}{dx}. \quad (4.5)$$

The sum arises because x influences f through multiple paths; the total effect is the sum of contributions from each path.

The chain rule is the engine of backpropagation: it tells us that to differentiate through a composition of functions, we multiply local derivatives layer by layer. This multiplication of many terms has a crucial consequence for deep networks - gradients can grow or shrink exponentially as they are propagated backwards through many layers. When they shrink towards zero, parameters in early layers receive negligible gradient signal and fail to learn (**vanishing gradients**); when they grow without bound, training becomes unstable (**exploding gradients**). These phenomena and the techniques used to address them are developed in a later session.

4.3.1. Local and Global Gradients

At each node in the computation graph we distinguish two quantities:

- The **local gradient** $\frac{\partial \text{out}}{\partial \text{in}}$ is the derivative of the node's output with respect to its input. It depends only on the local computation at that node, and can be computed during the forward pass from the stored input values.
- The **global gradient** $\frac{\partial \ell}{\partial \text{in}}$ is the derivative of the loss with respect to that node's input. It accumulates contributions from all paths leading from that node to the loss.

The backpropagation update at each node is:

$$\frac{\partial \ell}{\partial \text{in}} = \frac{\partial \ell}{\partial \text{out}} \cdot \frac{\partial \text{out}}{\partial \text{in}}. \quad (4.6)$$

In words: multiply the incoming **global gradient** (flowing in from the right) by the **local gradient** to obtain the outgoing **global gradient** (flowing to the left). This is applied at every node, proceeding right to left through the graph.

4.4. Worked Examples

4.4.1. Linear Regression - Symbolic

We work through the backpropagation pass for linear regression using the four-step decomposition, illustrated in Figure 4.5. The three columns below correspond to the

► Session 4:
Backprop
Example: Linear
Regression
(Symbolic),
(Numerical),
Logistic
Regression

three passes through the graph: forward pass and **local gradients** are evaluated left to right through the graph, reading top to bottom; **global gradients** are accumulated right to left, reading bottom to top.

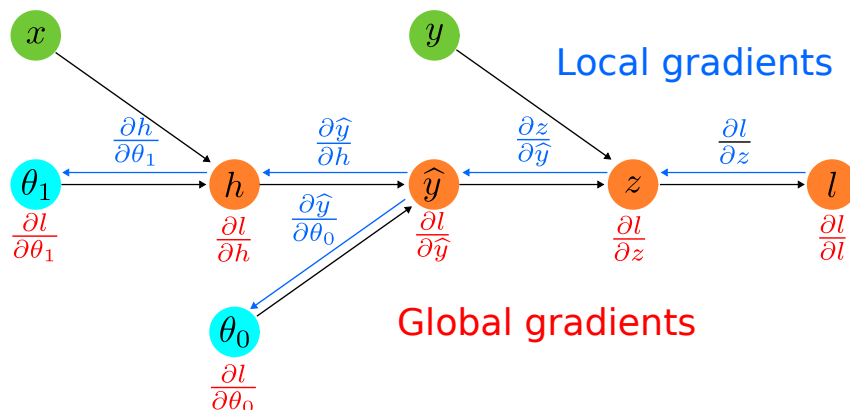


Figure 4.5: Symbolic local (blue) and global (red) gradients on the linear regression graph.

Forward pass ↓

- (1) $h = \theta_1 x$
- (2) $\hat{y} = \theta_0 + h$
- (3) $z = \hat{y} - y$
- (4) $\ell = z^2$

Local gradients ↓

- (1) $\partial h / \partial \theta_1 = x$
- (2) $\partial \hat{y} / \partial \theta_0 = 1$
- $\partial \hat{y} / \partial h = 1$
- (3) $\partial z / \partial \hat{y} = 1$
- (4) $\partial \ell / \partial z = 2z$

Global gradients ↑

- (4) $\partial \ell / \partial \theta_1 = \partial \ell / \partial h \cdot x$
- $\partial \ell / \partial \theta_0 = \partial \ell / \partial h \cdot 1$
- (3) $\partial \ell / \partial h = \partial \ell / \partial \hat{y} \cdot 1$
- (2) $\partial \ell / \partial \hat{y} = \partial \ell / \partial z \cdot 1$
- (1) $\partial \ell / \partial z = 1 \cdot 2z$

The gradients with respect to the parameters θ_0 and θ_1 are what gradient descent requires.

4.4.2. Linear Regression - Numerical

With concrete values $x = 2$, $y = 5$, $\theta_0 = 1$, $\theta_1 = 3$, Figure 4.6 shows the forward values and gradients on the graph.

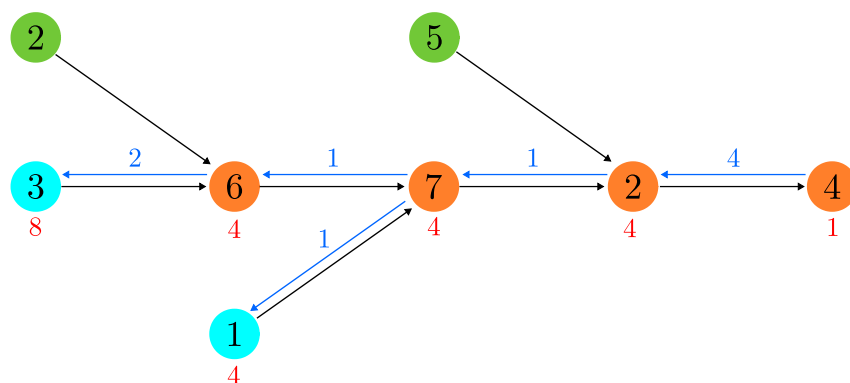


Figure 4.6: Numerical values on the computation graph for $x = 2$, $y = 5$, $\theta_0 = 1$, $\theta_1 = 3$.

Forward pass ↓

- (1) $h = 3 \cdot 2 = 6$
- (2) $\hat{y} = 1 + 6 = 7$
- (3) $z = 7 - 5 = 2$
- (4) $\ell = 2^2 = 4$

Local gradients ↓

- (1) $\partial h / \partial \theta_1 = 2$
- (2) $\partial \hat{y} / \partial \theta_0 = 1$
- $\partial \hat{y} / \partial h = 1$
- (3) $\partial z / \partial \hat{y} = 1$
- (4) $\partial \ell / \partial z = 4$

Global gradients ↑

- (4) $\partial \ell / \partial \theta_1 = 4 \cdot 2 = 8$
- $\partial \ell / \partial \theta_0 = 4 \cdot 1 = 4$
- (3) $\partial \ell / \partial h = 4 \cdot 1 = 4$
- (2) $\partial \ell / \partial \hat{y} = 4 \cdot 1 = 4$
- (1) $\partial \ell / \partial z = 1 \cdot 4 = 4$

The backward pass gives $\partial \ell / \partial \theta_0 = 4$ and $\partial \ell / \partial \theta_1 = 8$, which are the gradients used to update the parameters.

4.4.3. Logistic Regression

The same graph structure as in Figure 4.5 applies to logistic regression with sigmoid activation and binary cross-entropy loss — only the node functions differ. This is the key insight: backpropagation is a general algorithm that operates on the graph, independent of what functions the nodes implement.

Forward pass ↓

- (1) $h = \theta_0 + \theta_1 x$
- (2) $\hat{y} = \sigma(h)$
- (3) $\ell = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$

Local gradients ↓

- (1) $\partial h / \partial \theta_0 = 1$
 $\partial h / \partial \theta_1 = x$
- (2) $\partial \hat{y} / \partial h = \sigma(h)(1 - \sigma(h))$
- (3) $\partial \ell / \partial \hat{y} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}$

Global gradients ↑

- (3) $\partial \ell / \partial \theta_0 = \partial \ell / \partial h \cdot 1$
 $\partial \ell / \partial \theta_1 = \partial \ell / \partial h \cdot x$
- (2) $\partial \ell / \partial h = \partial \ell / \partial \hat{y} \cdot \partial \hat{y} / \partial h$
- (1) $\partial \ell / \partial \hat{y} = 1 \cdot \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}$

4.5. Multiple Neurons and Gradient Accumulation

When a node's output feeds into multiple downstream nodes, as in Figure 4.7, the chain rule generalisation applies: the global gradient at that node is the *sum* of contributions from each downstream path. Concretely, if node h feeds into nodes u and v :

$$\frac{\partial \ell}{\partial h} = \frac{\partial \ell}{\partial u} \cdot \frac{\partial u}{\partial h} + \frac{\partial \ell}{\partial v} \cdot \frac{\partial v}{\partial h}. \quad (4.7)$$

In implementation, gradients from all downstream paths are simply summed: each downstream node contributes its share, and the total gradient at the shared upstream node is their sum.

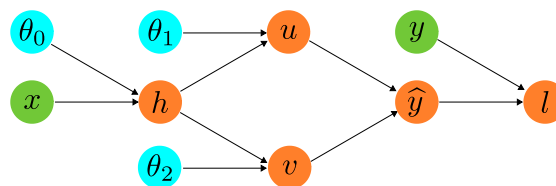


Figure 4.7: A node h feeding into two downstream nodes u and v . Gradients accumulate from both paths.

► Session 4:
Backprop:
Multiple Neurons
in a Layer

4.6. Technical Implementation and Vectorisation

Each node stores two quantities: `*.val` (the forward value) and `*.grad` (the accumulated gradient), with input nodes storing only `*.val`. The forward pass initialises all `*.val` by evaluating nodes from left to right. The backward pass initialises all `*.grad`

► Session 4:
Technical
Implementation,
From For Loops to
Vectorization

to zero, sets `l.grad = 1` (the seed), and then accumulates gradients from right to left using `grad += incoming_global * local_grad`.

In practice, the scalar for-loop implementation is far too slow for large networks. The key optimisation is **vectorisation**: replace loops over individual neurons with matrix operations. For a single layer:

```
Forward: z.val = W.val @ x.val + b.val
         a.val = act(z.val)
Backward: z.grad = a.grad * dact(z.val)
         W.grad = z.grad[:, None] * x.val[None, :].      (4.8)
```

The vectorised version computes the same quantities as the for-loop, but exploits the parallelism of modern hardware. On a GPU, this can be $\sim 100\times$ faster.

5. Optimization

These notes accompany the fifth lecture. Sessions 3 and 4 established the two ingredients needed for training: a loss function $\mathcal{L}(\theta)$ that measures how wrong the network is, and back-propagation to compute its gradient $\nabla_{\theta}\mathcal{L}(\theta)$ efficiently. What we have not yet addressed is how to *use* those gradients to find good parameters. That is the subject of this session. We develop the theory of gradient-based optimisation for deep networks: starting from vanilla gradient descent, motivating the need for stochastic approximations, then building up to the adaptive methods that are standard in practice. We also discuss two practical prerequisites that are often underemphasised: input normalisation and weight initialisation, both of which have a direct and significant effect on whether optimisation succeeds at all.

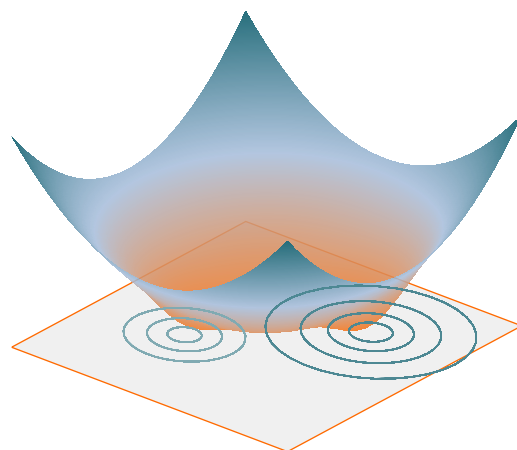


Figure 5.1: A non-convex loss surface over two parameters. Two distinct basins are visible, each with its own minimum. The contours projected on the base plane reveal their shape.

5.1. Optimisation Problem

Training a network means solving:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta; \mathcal{D}) = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(\mathbf{x}^{(i)}), y^{(i)}). \quad (5.1)$$

Remark 2. It is worth distinguishing $\min$ from $\arg \min$. The expression $\min_{\theta} \mathcal{L}(\theta)$ denotes the smallest value the loss attains — a scalar. The expression $\arg \min_{\theta} \mathcal{L}(\theta)$ denotes the parameter vector at which that value is attained — an element of the parameter space. As a simple example, $\min_{\theta} (\theta - 3)^2 = 0$, whereas $\arg \min_{\theta} (\theta - 3)^2 = 3$. In training we want θ^* itself, since it is the parameters we will deploy at test time,

► Session 5

► Session 5: Where We Are, Optimization Problem

not the loss value they achieve.

This is a high-dimensional, non-convex optimisation problem. The parameter vector θ may have millions of components; the loss surface has no closed-form minimum; and the landscape is riddled with local minima, saddle points, and flat regions. Figure 5.1 illustrates a simple two-parameter loss surface with two distinct basins. Unlike linear regression, no analytical solution exists — we must resort to iterative methods.

5.2. Gradient Descent

The fundamental algorithm is **gradient descent**. Starting from an initial parameter vector θ_0 , we repeatedly apply the update:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta; \mathcal{D}), \quad (5.2)$$

where $\eta > 0$ is the **learning rate**. Since the gradient points in the direction of steepest ascent of the loss, stepping opposite to it moves us downhill. As illustrated in Figure 3.8 from Session 3, the iterates move toward the minimum with steps that naturally decrease as the gradient shrinks near the optimum. Algorithm 1 states this formally.

Algorithm 1: Gradient Descent

Input: Dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$, learning rate η , initial parameters θ_0
for $t = 1, 2, \dots$ *until convergence* **do**
 forward: $f_{\theta}(\mathbf{x}^{(i)})$ for $i = 1, \dots, N$
 loss: $\mathcal{L}(\theta; \mathcal{D}) = \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(\mathbf{x}^{(i)}), y^{(i)})$
 backprop: $\nabla_{\theta} \mathcal{L}(\theta; \mathcal{D})$
 update: $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta; \mathcal{D})$
end

5.2.1. Loss Landscape

The loss surface of a deep network is a function from the parameter space — potentially millions of dimensions — to a scalar. Understanding its geometry is essential for understanding why gradient descent behaves the way it does. Figure 5.2 shows a one-dimensional cross-section illustrating the four main features encountered in practice.

Local minima are points where the gradient is zero and the loss curves upward in every direction — gradient descent cannot escape them by following the gradient. A common concern is that deep networks have many local minima and training will get stuck in a bad one. In practice this turns out to be less of a problem than feared: empirical evidence suggests that local minima in deep networks tend to have loss values close to the global minimum, and the network performs well regardless of which one is found.

Saddle points are zero-gradient points where the loss curves upward in some di-

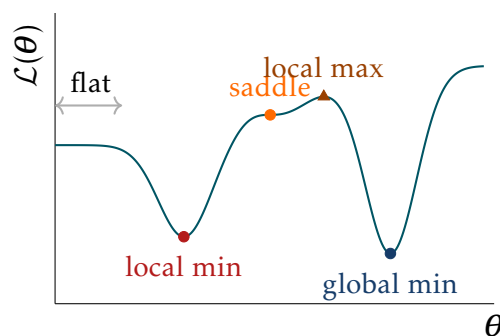


Figure 5.2: A non-convex loss landscape: local minimum, local maximum, saddle point (inflection), and flat region.

rections and downward in others. In a one-dimensional cross-section they appear as inflection points — where the curvature changes sign — with a near-zero gradient. In high dimensions, saddle points are far more common than local minima: a random critical point in a high-dimensional landscape is overwhelmingly likely to be a saddle rather than a true minimum. They are the more serious obstacle for gradient descent, because the gradient can be very small without the iterate being near a good solution.

Flat regions are extended areas where the gradient is very small but not exactly zero. Gradient descent makes negligible progress here — steps are tiny and many iterations are wasted. Flat regions often appear early in training when the network has not yet learned anything useful, or late in training when the network is near but not at a minimum.

Cliffs are sharp changes in the loss surface where the gradient is very large. A single gradient step in a cliff region can move the parameters by a huge amount, potentially undoing many previous steps. This is particularly common in recurrent networks and very deep architectures, and is the motivation for gradient clipping, which we will revisit in a later session.

The overarching message is that the loss landscape of a deep network is highly non-convex and has complex geometry. This means there is no guarantee that gradient descent will find the global minimum — or even a good local minimum — from an arbitrary starting point. The choice of initialisation, learning rate, and optimiser all affect which part of the landscape is explored and what solution is ultimately found.

5.2.2. Gradient Descent on Loss Landscape

An important practical point is that the loss surface is not known in advance — gradient descent probes it one point at a time. The algorithm starts from a random initialisation and uses local gradient information to decide where to step next. It has no global view of the landscape.

The step size — determined by the learning rate η — creates a fundamental tension. A small η leads to slow, expensive progress; a large η causes the iterates to jump erratically, potentially missing minima or diverging altogether. The choice of η is therefore critical and will be revisited when we discuss learning rate schedules.

Crucially, which minimum is found depends on the initialisation. Figure 5.3 illustrates this: two trajectories starting from nearby points on either side of a local maximum end up in completely different minima. This sensitivity to initialisation is not merely a theoretical curiosity — it has direct practical consequences, which is why weight initialisation deserves careful attention.

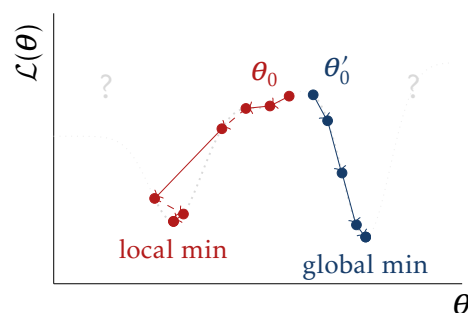


Figure 5.3: Two gradient descent trajectories starting from nearby initialisations on either side of a local maximum.

► Session 5:
Gradient Descent
on Loss Landscape

5.3. Stochastic and Mini-Batch Gradient Descent

5.3.1. Cost of Full-Batch Gradient Descent

Full-batch gradient descent as stated in Algorithm 1 computes the gradient over the entire dataset at each step. For datasets with millions of examples this is prohibitively expensive: one gradient evaluation requires one forward and one backward pass over all N samples.

The key observation is that the full loss is an average over individual sample losses:

$$\mathcal{L}(\theta; \mathcal{D}) = \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(\mathbf{x}^{(i)}), y^{(i)}). \quad (5.3)$$

A random subset $\mathcal{B} \subset \{1, \dots, N\}$, called a **mini-batch**, gives an unbiased estimate of the full loss:

$$\hat{\mathcal{L}}(\theta; \mathcal{B}) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \ell(f_{\theta}(\mathbf{x}^{(i)}), y^{(i)}), \quad \mathbb{E}_{\mathcal{B}}[\hat{\mathcal{L}}(\theta; \mathcal{B})] = \mathcal{L}(\theta; \mathcal{D}). \quad (5.4)$$

Since the gradient is a linear operator, an unbiased estimate of the loss immediately gives an unbiased estimate of the gradient:

$$\mathbb{E}_{\mathcal{B}}[\nabla_{\theta} \hat{\mathcal{L}}(\theta; \mathcal{B})] = \nabla_{\theta} \mathcal{L}(\theta; \mathcal{D}). \quad (5.5)$$

This justifies replacing the exact gradient with its mini-batch estimate in the update rule (5.2). In expectation, the update moves in the same direction as full-batch gradient descent; in practice, each step introduces some noise from the random batch selection.

Remark 3. The notation $\mathbb{E}_{\mathcal{B}}[\cdot]$ denotes the **expected value** over all possible mini-batches $\mathcal{B}$ drawn uniformly at random from $\mathcal{D}$. Concretely, if every subset of size B is equally likely, then $\mathbb{E}_{\mathcal{B}}[\hat{\mathcal{L}}]$ is the average of $\hat{\mathcal{L}}(\theta; \mathcal{B})$ over all such subsets.

An estimator $\hat{\mathcal{L}}$ is called **unbiased** for $\mathcal{L}$ if $\mathbb{E}_{\mathcal{B}}[\hat{\mathcal{L}}] = \mathcal{L}$ — that is, averaged over all possible randomly drawn batches, the mini-batch loss equals the full dataset loss. Any individual batch estimate will typically differ from the true loss: depending on which samples happen to be selected, $\hat{\mathcal{L}}$ may be higher or lower than $\mathcal{L}$. What unbiasedness guarantees is that these fluctuations are symmetric and cancel out on average — there is no systematic tendency to over- or under-estimate the true loss.

This connects directly to the iterative nature of training. Over many gradient steps, each using a freshly drawn mini-batch, we are effectively computing many independent estimates of the gradient and acting on each in turn. Even though each individual step is noisy, the cumulative effect of many unbiased steps approximates what many steps with the exact gradient would achieve. The algorithm never computes the average explicitly — it simply takes many steps, and the averaging happens implicitly across iterations.

5.3.2. Batch Size and SGD Spectrum

The batch size $|\mathcal{B}| = B$ interpolates between two extremes. When $B = N$ we recover full-batch gradient descent: each update uses the exact gradient, convergence is smooth, but each step requires a full pass through the data — prohibitively expensive for large datasets. When $B = 1$ we obtain **stochastic gradient descent** (SGD): each step uses

the gradient of a single sample, making updates maximally cheap but also maximally noisy. Despite the noise, SGD has solid theoretical convergence guarantees and was the method of choice before GPU computing made larger batches practical.

Between these extremes lies **mini-batch SGD**, which introduces a controlled amount of noise into the gradient estimates. This noise turns out to have a beneficial effect on how the optimiser navigates the loss landscape. Because each mini-batch is a different random subset of the data, consecutive gradient steps are computed on slightly different loss surfaces. A sharp, narrow minimum that looks like a minimum on one batch may not look like one on the next — the iterate gets perturbed away from it. A wide, flat minimum, by contrast, looks like a minimum for essentially every batch, so the iterate settles there and stays. Over the course of training, this stochastic perturbation acts as a kind of implicit regularisation: the optimiser tends to drift away from sharp minima and toward flat ones. This matters because flat minima generalise better — the loss does not change much when the parameters shift slightly, which is exactly what happens when moving from training data to unseen test data.

The same mechanism helps with saddle points. At a saddle point, the full-batch gradient is near zero — the optimiser would stall. But the mini-batch gradient is not zero: the random selection of samples breaks the cancellation that produces the near-zero full-batch gradient, providing a noisy push that can carry the iterate out of the saddle region.

The practical default is mini-batch SGD with B typically between 32 and 512. The choice of batch size involves several competing considerations. Larger batches give more accurate gradient estimates, leading to more stable updates and smoother convergence — but each step is more expensive and fewer steps can be taken per unit of compute, and the landscape-navigation benefits of noise are reduced. Smaller batches are cheaper per step and explore the landscape more aggressively, but the high variance means individual steps are less informative and the optimiser can struggle to make steady progress. There is also a hardware dimension: modern GPUs are designed for large parallel matrix operations and are underutilised by very small batches, so batch sizes are typically chosen as powers of two (32, 64, 128, 256) to align with GPU memory layouts. In practice, batch size is treated as a hyperparameter and tuned alongside the learning rate — the two are closely coupled, since a larger batch typically calls for a proportionally larger learning rate to make comparable progress per epoch.

5.3.3. Training Loop

A complete training run iterates over the dataset multiple times. One full pass through the dataset is called an **epoch**; one mini-batch update is called an **iteration**. Algorithm 2 states the full training loop.

Algorithm 2: Mini-Batch Gradient Descent

Input: Dataset $\mathcal{D}$, learning rate η , initial parameters θ_0 , number of epochs T

```

for  $epoch = 1, \dots, T$  do
    shuffle  $\mathcal{D}$  into random mini-batches  $\mathcal{B}_1, \mathcal{B}_2, \dots$ 
    for each mini-batch  $\mathcal{B}$  do
        forward: compute  $f_{\theta}(\mathbf{x}^{(i)})$  for  $i \in \mathcal{B}$ 
        loss:  $\hat{\mathcal{L}} = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \ell_i$ 
        backward: compute  $\nabla_{\theta} \hat{\mathcal{L}}$  via backpropagation
        update:  $\theta \leftarrow \theta - \eta \nabla_{\theta} \hat{\mathcal{L}}$ 
    end
end

```

Shuffling the dataset at the start of each epoch ensures that mini-batches are formed differently across epochs, preventing the network from overfitting to a fixed batch ordering. The loss on a held-out **validation set** is monitored throughout training to assess generalisation — a topic developed in the next session.

5.4. Setting Up for Optimisation

Before running any optimisation algorithm, two practical steps have a large effect on whether training succeeds: normalising the inputs and initialising the weights carefully. Both affect the geometry of the loss landscape and the behaviour of gradients from the very first step.

► Session 5: Why Feature Scale Matters, Input normalisation, Initialization: The Symmetry Problem, Initialization: Vanishing and Exploding Signals, Xavier and He Initialization

5.4.1. Input normalisation

The shape of the loss surface as a function of the parameters depends on the scale of the input features. To see why, consider a concrete example close to home. Suppose we want to predict the future income of MAI graduates based on two features: x_1 , their average final grade in the programme (ranging from 1 to 4), and x_2 , the total number of days they spent in the programme (ranging from 0 to roughly 1500). We fit a linear regression model with MSE loss:

$$\mathcal{L}(\theta_1, \theta_2; \mathcal{D}) = \frac{1}{N} \sum_{i=1}^N \left(\theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} - y^{(i)} \right)^2, \quad (5.6)$$

where θ_1 is the weight on the grade feature and θ_2 is the weight on the duration feature. The partial derivatives with respect to the two parameters are:

$$\frac{\partial \mathcal{L}}{\partial \theta_1} = \frac{2}{N} \sum_i r^{(i)} x_1^{(i)}, \quad \frac{\partial \mathcal{L}}{\partial \theta_2} = \frac{2}{N} \sum_i r^{(i)} x_2^{(i)}, \quad (5.7)$$

where $r^{(i)} = \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} - y^{(i)}$ is the residual. The gradient with respect to each parameter is scaled by the corresponding feature values. Since x_2 takes values up to 1500 while x_1 ranges only over $[1, 4]$, we have $x_2 \gg x_1$, and therefore $\partial \mathcal{L} / \partial \theta_2 \gg \partial \mathcal{L} / \partial \theta_1$ for the same residual. A tiny change in θ_2 causes a large change in the loss, while a large change in θ_1 barely moves it at all.

The consequence is that the loss contours are highly elongated ellipses, as shown in Figure 5.4 (top): the loss is very sensitive in the θ_2 direction and nearly flat in the θ_1 direction. Gradient descent oscillates violently across the steep direction while making negligible progress along the flat one — a classic symptom of an ill-conditioned optimisation problem.

To see what this means in practice, consider what happens during training without normalisation. The gradient with respect to θ_2 is large, so the update $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$ makes a large step in the θ_2 direction. If η is chosen to make reasonable progress along θ_2 , the step in the θ_1 direction is tiny — because the gradient there is small. The optimiser effectively ignores θ_1 for many iterations. Conversely, if η is increased to make progress on θ_1 , the step in θ_2 overshoots, causing the iterates to bounce back and forth across the valley without descending into it. There is no single learning rate that works well for both parameters simultaneously. In the worst case, training diverges: the overshoot in the θ_2 direction grows with each step, and the loss increases rather than decreasing. In the best case, training is simply very slow — the optimiser creeps along in the flat θ_1 direction while wasting most of its steps oscillating across θ_2 . Both pathologies become dramatically worse as the number of features grows and their scales become more varied.

The root cause is not the model or the data, but the mismatch in scales between the two features. A student's grade and the number of days they spent studying are both meaningful predictors of income, but they happen to be measured in very different units. The optimiser has no way to know this — it sees only numbers, and large numbers produce large gradients.

The standard fix is **z-score normalisation**, applied feature-wise before training:

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j}, \quad \mu_j = \frac{1}{N} \sum_{i=1}^N x_j^{(i)}, \quad \sigma_j = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_j^{(i)} - \mu_j)^2}. \quad (5.8)$$

After normalisation, the grade and duration features both have zero mean and unit variance. The optimiser now sees both on equal footing: a unit change in $\tilde{x}_1$ has the same scale as a unit change in $\tilde{x}_2$, and both gradients are of similar magnitude. The loss contours become approximately circular (Figure 5.4, bottom), and gradient descent converges much faster.

A critical implementation detail: μ_j and σ_j must be computed on the **training set only** and then applied identically to validation and test sets. Computing statistics on the full dataset would leak information about the test distribution into preprocessing — a subtle but consequential data leakage error.

5.4.2. Weight Initialisation

The choice of initial weights affects optimisation just as much as the loss landscape geometry. We already saw in Section 6.2.4 and Figure 5.3 that two trajectories starting

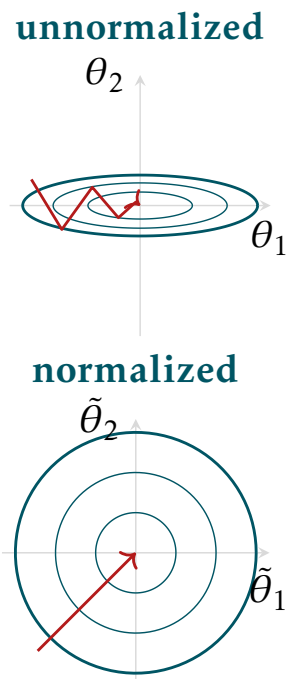


Figure 5.4: Unnormalised (top) vs normalised (bottom) loss contours.

from nearby but distinct points can converge to entirely different minima. The initialisation therefore determines not only how fast training proceeds, but which region of the loss landscape is explored and what solution is ultimately found. There are two distinct failure modes to avoid.

The symmetry problem. If all weights are initialised to the same value — for instance, all zeros, or any other fixed constant — then all neurons in a layer combine the same inputs with the same weights, and therefore compute identical pre-activations:

$$z_j^{(l)} = \sum_i \theta_{ji}^{(l)} h_i^{(l-1)} + \theta_{j0}^{(l)} = c \quad \forall j. \quad (5.9)$$

Every neuron in the layer produces the same output, and since all outputs are identical, every neuron in the *next* layer also receives the same inputs — propagating the symmetry forward through the entire network.

The problem compounds during the backward pass. Because every neuron in a layer has the same output, the loss is equally sensitive to each of them, and their gradients are identical. This means every neuron receives the same gradient signal and makes the same parameter update. After the update, the weights are still all equal — just shifted by the same amount. The symmetry is never broken, no matter how many gradient steps are taken.

The practical consequence is that the network has the width of a single neuron per layer, regardless of its architecture. All the neurons in a layer learn the same function, and the expressive capacity that width is supposed to provide is completely wasted. Weights must therefore be initialised **randomly** to break this symmetry — ensuring that different neurons start from different points and specialise in different directions during training.

Vanishing and exploding signals. Random initialisation is necessary but not sufficient — the scale of the random weights matters critically. To see why, consider an L -layer network with linear activations and weights drawn independently with $\mathbb{E}[\theta] = 0$ and $\text{Var}[\theta] = \sigma^2$, with n_{in} inputs per layer. The pre-activation at each layer is a weighted sum of n_{in} independent inputs:

$$z_j^{(l)} = \sum_{i=1}^{n_{\text{in}}} \theta_{ji}^{(l)} h_i^{(l-1)}. \quad (5.10)$$

Since the weights and inputs are independent and zero-mean, the variance of this sum is the sum of the variances of the individual terms. Each term $\theta_{ji} h_i$ has variance $\text{Var}[\theta] \cdot \text{Var}[h] = \sigma^2 \cdot \text{Var}[h]$, and there are n_{in} such terms, giving:

$$\text{Var}[z^{(l)}] = n_{\text{in}} \sigma^2 \cdot \text{Var}[h^{(l-1)}]. \quad (5.11)$$

With linear activations, $h^{(l)} = z^{(l)}$, so this relation applies recursively. After L layers:

$$\text{Var}[z^{(L)}] = (n_{\text{in}} \sigma^2)^L \cdot \text{Var}[x]. \quad (5.12)$$

The factor $n_{\text{in}} \sigma^2$ is multiplied once per layer, so the signal variance grows or shrinks geometrically with depth. This has severe practical consequences.

If $n_{\text{in}} \sigma^2 < 1$, the variance shrinks at every layer. By the time the signal reaches the output in a deep network, it is effectively zero — activations are all near zero, and the network cannot distinguish between different inputs. More critically, backpropagation propagates gradients *backward* through the same weights, so the gradient variance shrinks by the same factor at each layer. The gradients reaching the early layers are vanishingly small: those layers receive almost no learning signal and their weights barely move during training, regardless of how many gradient steps are taken. The network learns only from its final layers, and its early feature detectors remain random.

If $n_{\text{in}} \sigma^2 > 1$, the variance grows at every layer. Activations become very large, and gradients blow up exponentially as they propagate backward. A single gradient step moves the parameters by an enormous amount, completely undoing any progress made in previous steps. In practice this manifests as loss values that spike to infinity or NaN within the first few iterations of training.

Both failure modes become dramatically worse with depth — a factor of 0.9 per layer is barely noticeable in a three-layer network but reduces the gradient to $0.9^{50} \approx 0.005$ in a fifty-layer one. The only stable regime is $n_{\text{in}} \sigma^2 = 1$, which keeps the signal variance constant across all layers and gives the target initialisation variance $\sigma^2 = 1/n_{\text{in}}$.

Remark 4. The derivation above assumes linear activations and is exact under independence. With nonlinear activations the argument is approximate, but the qualitative conclusion — that $n_{\text{in}} \sigma^2$ controls stability — remains valid.

Xavier and He initialisation. The condition $\sigma^2 = 1/n_{\text{in}}$ ensures variance is preserved in the forward pass. A symmetric argument applied to the backward pass yields the condition $\sigma^2 = 1/n_{\text{out}}$. These two conditions cannot generally be satisfied simultaneously. Glorot and Bengio (2010) proposed taking their harmonic mean:

$$\sigma^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}}, \quad \theta_{ji}^{(l)} \sim \mathcal{U}\left[-\frac{\sqrt{6}}{\sqrt{n_{\text{in}} + n_{\text{out}}}}, \frac{\sqrt{6}}{\sqrt{n_{\text{in}} + n_{\text{out}}}}\right]. \quad (5.13)$$

This is **Xavier** (or Glorot) initialisation, designed for symmetric activations such as tanh or sigmoid.

For ReLU activations, approximately half the neurons are zeroed at any given forward pass, which halves the effective variance relative to the linear case. He et al. (2015) showed that this requires compensating with an extra factor of two:

$$\sigma^2 = \frac{2}{n_{\text{in}}}, \quad \theta_{ji}^{(l)} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right). \quad (5.14)$$

This is **He** (or Kaiming) initialisation.

5.5. Momentum and Adaptive Methods

Even with good initialisation and normalised inputs, vanilla mini-batch gradient descent can be slow or unstable. Three failure modes motivate progressively more sophisticated update rules.

► Session 5:
Limitations of
Vanilla GD,
Momentum,
RMSProp, Adam

5.5.1. Momentum

Mini-batch gradients are noisy: each batch produces a slightly different gradient estimate, causing the parameter updates to fluctuate. In a narrow valley — where the loss is steep across the valley and flat along it — this noise produces rapid oscillations across the valley while progress along the valley is slow. **Momentum** addresses this by accumulating a velocity vector $\mathbf{m}_t$, defined as an exponential moving average of past gradients:

$$\mathbf{m}_t = \beta \mathbf{m}_{t-1} + (1 - \beta) \nabla_{\theta} \hat{\mathcal{L}}_t, \quad (5.15)$$

$$\theta \leftarrow \theta - \eta \mathbf{m}_t. \quad (5.16)$$

The coefficient $\beta \in [0, 1)$ — typically 0.9 — controls how much history is retained. Gradients in directions that are consistent across many steps accumulate in $\mathbf{m}_t$, amplifying progress along those directions. Gradients in directions that alternate sign (oscillations across a narrow valley) cancel out, damping the oscillations. The net effect is faster, smoother progress along the loss landscape.

This raises an apparent tension with our earlier argument that mini-batch noise is beneficial for landscape exploration. The resolution is that momentum does not remove the stochasticity — it smooths the gradient *direction* across recent batch updates, but each update still uses a different random batch. The randomness that drives landscape exploration comes from sampling different data points each time; that remains intact. What momentum removes is the erratic variation in step direction caused by the high variance of individual batch gradients — the kind of oscillation that makes the optimiser zigzag across a valley without making progress. The two ideas are therefore complementary: stochastic batching provides the exploratory randomness, and momentum channels that into more directed, efficient steps.

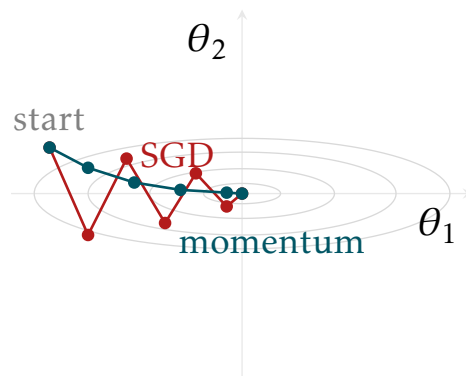


Figure 5.5: SGD oscillates across a narrow valley. Momentum follows a smoother path.

Figure 5.5 contrasts SGD and momentum on an elongated loss landscape. SGD zigzags across the narrow direction; momentum dampens the oscillations and converges more directly.

5.5.2. RMSProp

Momentum smooths the update direction but uses the same effective learning rate for every parameter. In practice, different parameters may have very different gradient magnitudes — some are sensitive (large gradients) and others are nearly flat (small gradients) — and a single η is simultaneously too large for the former and too small for the latter. This problem persists even after input normalisation: normalisation fixes the scale mismatch at the input, but does not control what happens inside the network. As weights evolve during training, different parameters in different layers develop very different gradient magnitudes, and this imbalance changes continuously.

RMSProp addresses this by tracking, for each parameter separately, how large its gradients have been recently. The natural measure is the squared gradient, which

removes the sign so that both positive and negative gradients contribute equally to the magnitude estimate. RMSProp maintains an exponential moving average of these squared gradients:

$$\mathbf{v}_t = \gamma \mathbf{v}_{t-1} + (1 - \gamma) (\nabla_{\theta} \hat{\mathcal{L}}_t)^2, \quad (5.17)$$

with $\gamma \approx 0.99$. A parameter with consistently large gradients accumulates a large $\mathbf{v}_t$; one with small gradients accumulates a small $\mathbf{v}_t$. The update divides by $\sqrt{\mathbf{v}_t}$ to normalise the gradient by its recent typical magnitude:

$$\theta \leftarrow \theta - \frac{\eta}{\sqrt{\mathbf{v}_t} + \epsilon} \nabla_{\theta} \hat{\mathcal{L}}_t, \quad (5.18)$$

with $\epsilon \approx 10^{-8}$ for numerical stability. Parameters with large recent gradients receive a smaller effective step; those with small gradients receive a larger one — automatically equalising the effective learning rate across all dimensions.

The key distinction from momentum is that RMSProp accumulates squared gradients and carries no directional information. Momentum is about smoothing the *direction* of updates; RMSProp is about adapting their *magnitude*. The two are complementary, which motivates combining them.

5.5.3. Adam

Adam (Kingma and J. Ba 2015) (Adaptive Moment Estimation) combines momentum and RMSProp into a single update rule, maintaining both a smoothed gradient direction and a per-parameter magnitude estimate simultaneously:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \nabla_{\theta} \hat{\mathcal{L}}_t, \quad (5.19)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) (\nabla_{\theta} \hat{\mathcal{L}}_t)^2. \quad (5.20)$$

Both moving averages are initialised at zero, which biases them toward zero early in training when few gradient estimates have been accumulated. Adam corrects for this by rescaling both estimates:

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}. \quad (5.21)$$

When t is small, β^t is close to 1 and the correction is large, inflating the early underestimates back to a reasonable scale. As t grows, $\beta^t \rightarrow 0$, the corrections approach 1, and the bias correction quietly switches off. The parameter update is then:

$$\theta \leftarrow \theta - \frac{\eta}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \hat{\mathbf{m}}_t. \quad (5.22)$$

The default hyperparameters $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ work well across a wide range of architectures and tasks, which is why Adam has become the standard default optimiser in deep learning — it requires little tuning and handles the varied gradient scales that arise naturally in deep networks.

5.6. Learning Rate Schedules

A fixed learning rate is a compromise: a large η enables fast initial progress but overshoots near a minimum; a small η allows fine-grained convergence but is impractically slow early on. **Learning rate schedules** vary η over the course of training to get the best of both regimes. Figure 5.6 illustrates the three schedules discussed below.

Step decay. The simplest schedule reduces η by a multiplicative factor $\gamma < 1$ every k epochs:

$$\eta_t = \eta_0 \cdot \gamma^{\lfloor t/k \rfloor}. \quad (5.23)$$

The staircase pattern visible in Figure 5.6 is characteristic: η is held constant for k steps then drops abruptly. This is simple and interpretable, but the sudden drops can cause instability and the flat regions between drops waste the opportunity for finer progress as the optimiser approaches a minimum.

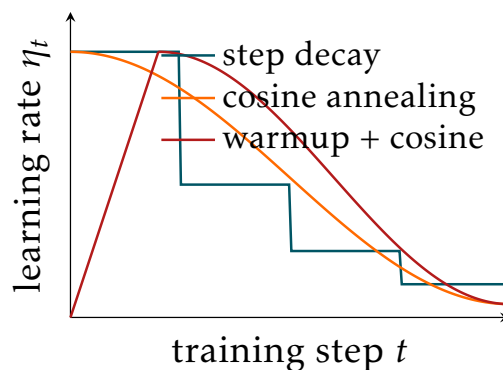


Figure 5.6: Three learning rate schedules.

Cosine annealing. A smooth alternative follows a cosine curve from η_0 down to a minimum $\eta_{\min}$:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min}) \left(1 + \cos \frac{\pi t}{T} \right), \quad (5.24)$$

where T is the total number of training steps. The decay is fast at first — when the optimiser is still far from a minimum and large steps are useful — and slows gradually as training progresses. This smooth profile avoids the instabilities of step decay and is widely used in modern training recipes.

Linear warmup with cosine decay. Large-scale training — particularly for Transformers — uses a warmup phase in which η is ramped up linearly from zero over the first t_w steps, followed by cosine decay:

$$\eta_t = \begin{cases} \eta_0 \cdot t/t_w & t \leq t_w \\ \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min}) \left(1 + \cos \frac{\pi(t-t_w)}{T-t_w} \right) & t > t_w. \end{cases} \quad (5.25)$$

The warmup is motivated by the state of the network at the very start of training: parameters are randomly initialised and far from any good solution, so gradients can be large and unreliable. Starting with a small η and ramping it up gradually prevents these early noisy gradients from causing large destabilising updates before the optimiser has settled into a reasonable region of the loss landscape.

6. Generalization & Regularization

These notes accompany the seventh lecture. Sessions 1–6 built the full machinery for training deep networks: forward pass, backpropagation, optimisation, and the PyTorch API. Throughout, we minimised a training loss and measured progress by how low it went. This session steps back and asks a more fundamental question: is minimising training loss the right objective? The answer is no - the true goal is performance on new, unseen data from the population. We develop the concept of generalisation, examine why deep networks generalise surprisingly well despite being massively overparameterised, and then study the main techniques for ensuring generalisation in practice.

► Session 7

6.1. Generalisation

We begin by formalising what a good model actually means and establishing the tools for measuring it.

6.1.1. True Goal and Generalisation Gap

Throughout this course we have trained models by minimising the empirical loss on a training dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta; \mathcal{D}) = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(\mathbf{x}^{(i)}), y^{(i)}). \quad (6.1)$$

But what we actually want is a model that performs well on **new, unseen examples drawn from the same population**. The true objective is the **population loss**:

$$\mathcal{L}_{\text{pop}}(\theta) = \mathbb{E}_{(\mathbf{x}, y) \sim p_{\text{data}}} [\ell(f_{\theta}(\mathbf{x}), y)], \quad (6.2)$$

where p_{data} is the true data-generating distribution. This is an **expectation** - the average loss over all possible examples from the population. On some examples the model will do better than average, on others worse; this reflects natural variation in the data and is unavoidable. Optimising the average is the right objective. The training set $\mathcal{D}$ is a finite sample from p_{data} , and minimising $\mathcal{L}(\theta; \mathcal{D})$ is only a proxy for minimising $\mathcal{L}_{\text{pop}}(\theta)$.

The **generalisation gap** quantifies how well the proxy serves the true objective:

$$\text{generalisation gap} = \mathcal{L}_{\text{pop}}(\theta) - \mathcal{L}_{\text{train}}(\theta) \geq 0. \quad (6.3)$$

A model with a small generalisation gap has learned something genuinely useful about the data-generating process. A model with a large gap has overfit - it has memorised properties of the training set that do not hold for the population.

6.1.2. Train, Validation, and Test Split

In practice, a dataset is divided into three non-overlapping subsets with distinct roles:

Training set - used to fit model parameters by minimising $\mathcal{L}(\theta; \mathcal{D}_{\text{train}})$. The model sees this data repeatedly during training.

Validation set - used to evaluate the model during development: selecting architecture, tuning hyperparameters, and deciding when to stop training. The model does not train on validation data, but validation performance influences decisions about the model. It is therefore touched many times during development.

Test set - used for final, honest evaluation of the deployed model. It is a held-out sample from p_{data} used to estimate $\mathcal{L}_{\text{pop}}$ - a **measurement tool, not the true objective**. Repeatedly evaluating on the test set and adjusting the model is a form of **overfitting the test set**: the test loss becomes an optimistic estimate of true population performance. This is why the test set must be touched only once, at the very end of development, and must not influence any modelling decision.

Remark 5. The key requirement for validation and test sets is that they are **representative of the population** - they should reflect the full variation in p_{data} . A more complex population requires larger sets to estimate performance reliably. At the same time, every example allocated to validation or test is unavailable for training,

so there is a tradeoff. Typical splits of 80/10/10 or 70/15/15 are reasonable starting points, but the right split depends on dataset size and population complexity.

6.1.3. Overfitting and Underfitting

Two failure modes arise from the training-population gap:

Underfitting occurs when the model is too simple to capture the structure in the data. Both training loss and test loss are high, and the generalisation gap is small - the model fails on training data and test data alike. Remedies include increasing model capacity, training longer, or tuning the learning rate and optimiser.

Overfitting occurs when the model is too complex and memorises training data, including its noise. Training loss is low but test loss is high, producing a large generalisation gap. The model has learned patterns that are specific to the training set and do not generalise. Remedies include regularisation (discussed in section 6.3), obtaining more data, or reducing model capacity.

The primary diagnostic tool is monitoring the generalisation gap during training - figure 6.1 shows an example plot of training loss and validation loss together as a function of epochs. When validation loss stops improving and begins to rise while training loss continues to fall, overfitting is occurring.

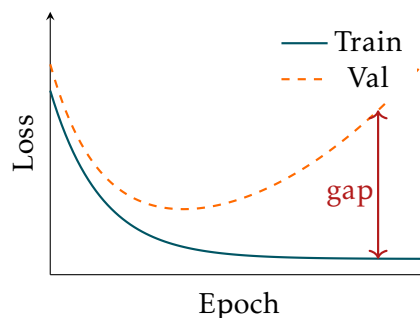


Figure 6.1: Training and validation loss over epochs. Train loss decreases monotonically; validation loss initially decreases then increases as the model begins to overfit. The gap between the two curves is the generalisation gap.

6.2. Generalisation Paradox

Classical statistical learning theory makes a clear prediction: models with more parameters than training examples should overfit catastrophically. Deep networks violate this prediction routinely - they are massively overparameterised yet generalise well. This section examines why, starting from the classical bias-variance framework and then developing the modern understanding of why deep networks are different.

► Session 7:
Bias-Variance
Tradeoff, Limits of
Classical View,
Many Good
Minima, Flat vs
Sharp Minima,
Double Descent

6.2.1. Bias-Variance Tradeoff

Classical statistical learning theory decomposes the expected test error into two competing terms. The **bias** measures how wrong the model is on average across all possible training sets - a high-bias model is too simple to capture the true data-generating process. The **variance** measures how sensitive the model is to the specific training set used - a high-variance model fits each training set very well but produces different predictions for different training sets, so it fails on new data.

The classical prescription is to choose model complexity to balance the two: simple models have high bias and low variance (underfitting), complex models have low bias and high variance (overfitting), and the optimal model sits somewhere in between. Adding more parameters reduces bias but increases variance - so classical theory predicts that overparameterised models should overfit badly.

6.2.2. Limits of the Classical View

Deep networks have huge numbers of parameters - far more than needed to memorise the training data. They routinely drive training loss close to zero, nearly memorising the training set. Classical bias-variance theory predicts this should lead to catastrophic overfitting. Yet in practice, deep networks generalise well. This is the **generalisation paradox** of deep learning, and it is not fully resolved by classical theory. Understanding why deep networks generalise requires looking beyond the classical framework.

6.2.3. Many Good Minima

One part of the explanation is that overparameterised networks have a vast number of local minima, and most of them have very similar loss values. Networks trained from different random initialisations converge to different minima yet achieve similar generalisation performance. This means the choice of which minimum gradient descent finds is less critical than classical theory suggests - there are simply many equivalent solutions, and the optimiser finds one of them.

A theoretical underpinning for this observation comes from the connection to statistical physics. Choromanska et al. (2015) showed, using a correspondence with spin-glass models, that in deep networks the local minima with high loss become exponentially rare as network size grows - almost all local minima cluster near the global minimum. The more practically relevant obstacle is not local minima at all but **saddle points** - critical points where the loss curves upward in some directions and downward in others. Dauphin et al. (2014) argued that in high-dimensional spaces, saddle points vastly outnumber local minima, and that escaping saddle points is the main challenge for gradient-based optimisation in deep networks. The noise introduced by mini-batch SGD helps here too: stochastic gradient estimates provide perturbations that can push iterates away from saddle regions where the exact gradient would stall.

6.2.4. Flat vs Sharp Minima

Not all minima generalise equally well. A **sharp minimum** is one where the loss changes rapidly as parameters move away from the minimum - the basin is narrow and deep. A **flat minimum** is one where the loss changes slowly - the basin is wide and shallow. Figure 6.2 illustrates the difference.

The connection between flatness and generalisation was first formalised by Hochreiter and Schmidhuber (1997a), who argued using minimum description length that flat minima correspond to simpler, more compressible models and therefore generalise better. The intuition is the following: when we move from the training distribution to the test distribution, the loss surface shifts slightly. A sharp minimum that is optimal for the training distribution may be far from optimal for the test distribution - a small shift in the surface is enough to push the parameters out of the narrow basin, causing a large increase in

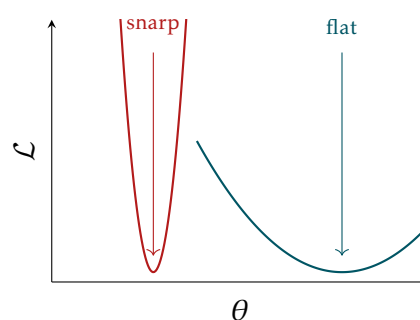


Figure 6.2: Sharp and flat minima. The sharp minimum (carnelian) has high curvature - a small shift in parameters causes a large increase in loss. The flat minimum (petrol) has low curvature - loss remains low over a wide region of parameter space.

loss. A flat minimum is more robust: even if the surface shifts, the parameters remain in a region of low loss.

The practical importance of this observation was demonstrated by Keskar et al. (2017), who showed empirically that large-batch SGD consistently converges to sharp minima that generalise poorly, while small-batch SGD finds flat minima that generalise well. This provides a direct explanation for the well-known generalisation gap between large-batch and small-batch training, and confirms that mini-batch SGD acts as an **implicit regulariser** - it biases the optimiser toward solutions that generalise better, without any explicit change to the loss function.

6.2.5. Double Descent

The classical bias-variance curve predicts a single U-shaped relationship between model complexity and test error - one sweet spot of optimal complexity. Belkin et al. (2019) showed that this picture is incomplete: the **double descent** phenomenon reveals that beyond the interpolation threshold, test error decreases again as complexity grows further.

As model complexity grows, test error first follows the classical U-shape. At the **interpolation threshold** - the point where the model has just enough capacity to fit the training data exactly - test error peaks. This is the worst operating point: the model is maximally rigid, forced to commit every degree of freedom to fitting training points exactly, with no slack for smooth interpolation. Beyond this threshold, as complexity continues to grow, test error decreases again. Nakkiran et al. (2020) demonstrated that this phenomenon occurs in deep neural networks specifically, appearing not only as a function of model size but also as a function of training epochs and dataset size.

The intuition for why overparameterisation helps is that excess capacity allows smoother, more natural interpolation between training points. When a model has just enough parameters to fit the training data, every degree of freedom is fully committed - the interpolant is maximally constrained and can produce sharp, jagged behaviour between training points. When the model is overparameterised, there are infinitely many solutions that fit the data exactly, and gradient descent from a small random initialisation selects among them. Bartlett, Montanari, and Rakhlin (2020) showed for linear models that gradient descent finds the **minimum-norm solution** - the interpolant that fits the training data while keeping the parameter vector as small as possible. Minimum-norm solutions are smooth by construction: large oscillations between training points require large parameter magnitudes, so minimising the norm suppresses them.

This connects back to the **implicit regularisation of mini-batch SGD** discussed in section 6.2.4. The noise in gradient estimates not only favours flat minima over sharp ones, but also biases the optimiser toward low-norm solutions. Both effects work in

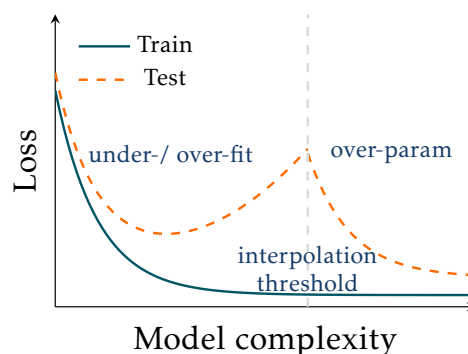


Figure 6.3: Double descent. The classical U-shaped bias-variance curve (left) predicts a single optimal complexity. Beyond the interpolation threshold, test loss decreases again as model complexity grows further.

the same direction: the optimisation procedure itself acts as a regulariser, steering training toward solutions that generalise well - without any explicit penalty term in the loss. Deep networks operate in this overparameterised regime, to the right of the interpolation threshold, and benefit from both effects.

6.3. Regularisation Techniques

SGD provides implicit regularisation for free. We now turn to explicit techniques that can be added on top to further control the generalisation gap.

6.3.1. Weight Decay

Large weights indicate that the model relies heavily on specific features - small changes in input produce large changes in output, increasing sensitivity to the training set and reducing generalisation. The classical fix is to penalise large weights directly by adding a regularisation term to the loss:

$$\tilde{\mathcal{L}}(\theta) = \mathcal{L}(\theta) + \frac{\lambda}{2} \|\theta\|^2. \quad (6.4)$$

This is **L2 regularisation**, also called **weight decay** - the gradient of the penalty term, $\lambda\theta$, shrinks all weights toward zero at each update step, distributing reliance more evenly across features. The hyperparameter $\lambda > 0$ controls the strength and is tuned on the validation set.

In PyTorch, weight decay is available via the `weight_decay` parameter within optimisers. Weight decay is a useful starting point but it is less central in deep learning than in classical ML. Mini-batch SGD already provides implicit regularisation by biasing toward low-norm solutions, reducing the marginal benefit of an explicit penalty. Moreover when using Adam, standard L2 regularisation interacts undesirably with the adaptive learning rates (Loshchilov and Hutter 2019).

6.3.2. Dropout

Dropout (Srivastava et al. 2014) is one of the most powerful and widely used regularisation techniques in deep learning. It randomly zeros each activation during training with probability p , called the **dropout rate**. Each forward pass effectively uses a different subnetwork - a random subset of the neurons, as illustrated in Figure 6.4.

Dropout prevents neurons from **co-adapting**: no neuron can rely on specific other neurons always being present, because those neurons may be absent in the next forward pass. Each neuron is therefore forced to be independently useful - it must learn a feature that is valuable on its own, not one that only makes sense in combination with specific other neurons. On the other hand, it also prevents **over-specialisation**: neurons are discouraged from learning highly specific, narrow features that are only useful in one context. The result is a network that learns more distributed,

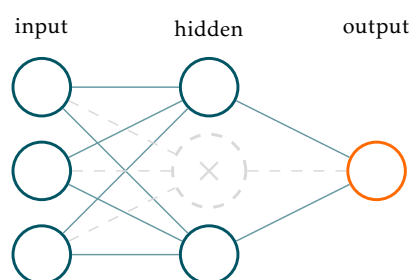


Figure 6.4: Dropout during training. One neuron (grey, crossed) is randomly zeroed. Each forward pass uses a different subnetwork.

robust representations - each neuron contributes meaningfully on its own, and the representations generalise better to unseen data.

Train and evaluation modes. During training, only a fraction $(1 - p)$ of neurons are active on average - the network learns with reduced capacity. During evaluation the behaviour changes in two ways. First, all neurons are active - no masking is applied. Second, the outputs must be rescaled to account for the difference in the number of active neurons between training and evaluation. Without correction, the full network at test time would produce larger activations than the network was trained to expect, changing the effective scale of every layer.

The standard correction is to multiply activations by $(1 - p)$ at test time. An equivalent and more computationally convenient formulation is **inverted dropout**: scale active activations *up* by $\frac{1}{1-p}$ during training instead, so that no correction is needed at evaluation. This is what PyTorch implements - `nn.Dropout(p)` scales active activations by $\frac{1}{1-p}$ during training and passes activations through unchanged during evaluation. The net effect is the same; the advantage is that the evaluation forward pass requires no extra operations.

Typical values of p range from 0.1 to 0.5, tuned on the validation set. Dropout works best when applied to **fully connected layers**. In other architectures - convolutional networks (CNNs), recurrent networks (RNNs), transformers - the standard formulation is less effective or requires adaptation. Dropout also interacts with batch normalisation in a non-trivial way; this is addressed in section 6.4.1.

Ensemble interpretation. A deeper understanding of why dropout works comes from the ensemble perspective. A network with n neurons and dropout rate p implicitly defines 2^n different subnetworks - one for each possible dropout mask. During training, each mini-batch trains a different subnetwork, but all subnetworks share the same weights. This is an extremely parameter-efficient form of ensemble training: instead of training 2^n separate models, a single set of weights is shared across all of them.

At test time, using the full network with rescaled activations approximates averaging the predictions of all 2^n subnetworks simultaneously. Ensemble averaging is a classical and well-understood technique for reducing variance: individual models may overfit in different directions, but their average tends to cancel out idiosyncratic errors and generalise better. Dropout achieves this ensemble effect at the cost of training and deploying a single model - a remarkable result given the exponential number of subnetworks implicitly represented.

6.3.3. `model.train()` and `model.eval()`

During training we want certain layers to behave stochastically - dropout should randomly zero activations, producing a different subnetwork for each forward pass. During evaluation and inference we want deterministic, stable behaviour - dropout should be disabled and all neurons should be active. These are two distinct operational regimes that require different behaviour from the same layers. Other layers also have dual behaviour - most notably batch normalisation, which we will encounter in section 6.4.1.

PyTorch implements this through a **model mode**: every `nn.Module` instance is either in *training mode* or *evaluation mode*. The mode is set by calling `model.train()` or `model.eval()` on your instantiated model, where `model` is whatever name you gave

it. Both calls propagate recursively to all submodules, so a single call on the top-level model is sufficient regardless of how many layers it contains. Layers that respond to the mode switch include `nn.Dropout` and, as we will see later, `nn.BatchNorm`; all other standard layers - `nn.Linear`, `nn.ReLU`, and so on - are unaffected.

The correct pattern in a training loop is:

```
model = MyNetwork(...)          # your instantiated model

# training loop
model.train()
for X_batch, y_batch in train_loader:
    optimizer.zero_grad()
    loss = loss_fn(model(X_batch), y_batch)
    loss.backward()
    optimizer.step()

# validation loop
model.eval()
with torch.no_grad():
    for X_batch, y_batch in val_loader:
        loss = loss_fn(model(X_batch), y_batch)
```

Remark 6. Forgetting to call `model.eval()` before validation or inference is a silent bug - no error is raised, but predictions are wrong because dropout is still active and batch normalisation uses incorrect statistics. Always call `model.eval()` before validation and `model.train()` before resuming training.

6.3.4. Data Augmentation

Data augmentation **creates new training examples** by applying transformations to existing ones from the training set. The fundamental requirement is **invariance**: the correct prediction must be invariant to the transformation, which means the transformation must be **label-preserving**. These are two ways of saying the same thing - if rotating an image of a cat still produces an image of a cat, the model should be invariant to rotation, and rotation is therefore a valid label-preserving transformation.

This constraint gives augmentation two complementary benefits. First, it **increases the effective size of the training set** - more diverse training examples reduce overfitting regardless of their content. Second, and more fundamentally, it **explicitly teaches the model the invariances that matter for the problem**: a model trained on cats in many orientations learns that orientation does not affect the label; a model trained on patient measurements with small added noise learns that minor measurement variation does not affect the diagnosis.

For image data, common augmentations include random horizontal flips, rotations, crops, and colour jitter; for tabular data, Gaussian noise on continuous features and random feature masking are typical choices. Though these are reasonable starting points, they are not universally valid.

The choice of transformations requires **domain knowledge** - only the practitioner knows which invariances are valid for their specific problem. Every augmentation is an

implicit claim about the data-generating process: it asserts that transformed examples are plausible draws from p_{data} with the same label. A horizontal flip is valid for natural image classification - a cat seen from the left is the same cat seen from the right. The same flip applied to digit recognition is wrong - a 6 rotated 180° becomes a 9. Aggressive colour changes are harmless for classifying animals but damaging for medical imaging, where colour carries diagnostic information. **Augmentation pipelines from papers and tutorials must never be transferred blindly to a new problem.** Every augmentation must be justified by asking: is this transformation label-preserving for *my* problem and *my* data?

In practice, augmentation is applied **on the fly during data loading**: at each training epoch, transformations are sampled randomly and applied to training examples as they are loaded into a mini-batch. The training set itself is never modified - the model sees a different random version of each example at every epoch, continuously expanding the effective variety of training data.

Note that invariance can also be built directly into the architecture rather than learned from augmented data - certain architectures are invariant to specific transformations by construction. This will become relevant when we study convolutional networks.

6.3.5. Early Stopping

Training longer always reduces training loss - the optimiser will keep finding ways to fit the training data more precisely. But beyond a certain point this comes at the cost of generalisation: the model begins to memorise training-specific patterns that do not hold for the population, and the generalisation gap grows. The validation loss, which tracks performance on held-out data, reveals this: it initially decreases alongside training loss, but eventually levels off and begins to rise as overfitting sets in, as illustrated in Figure 6.5.

The natural response is to stop training at the point where the model generalises best - the minimum of the validation loss curve - rather than training until convergence. This is **early stopping**. It is a form of regularisation because it constrains the effective training time: a model trained for fewer steps has had less opportunity to memorise the training data, limiting its effective capacity without changing the model architecture or the loss function.

In practice, the best model encountered during training is saved at each epoch and the final deployed model is this saved checkpoint, not the model at the end of training. Since validation loss is noisy and may fluctuate temporarily before resuming its descent, training is not stopped at the first sign of non-improvement. Instead, a **patience** parameter k specifies how many consecutive epochs without improvement to tolerate before stopping. All the ingredients for early stopping are already present in the training loop from session 6 - it simply adds a stopping criterion and model checkpointing.

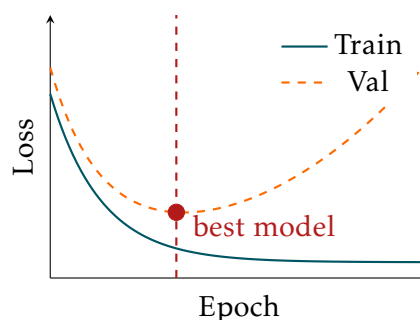


Figure 6.5: Early stopping. Training loss decreases monotonically; validation loss eventually increases. The best model is saved at the validation minimum and deployed rather than the final model.

6.4. Training Stability

Regularisation solves the problem of overfitting - it allows us to use large, overparameterised networks without the model memorising the training data. But deeper networks introduce a separate class of problems that have nothing to do with overfitting: **training instabilities**. These are problems that make training impossible or unreliable regardless of whether the model would overfit. More layers amplify small issues, and three instabilities are particularly important: internal covariate shift, vanishing and exploding gradients, and the dying ReLU problem.

6.4.1. Internal Covariate Shift and Batch Normalisation

In section 5.4.1 we saw that normalising the inputs to the network stabilises training: unnormalised features with very different scales produce an ill-conditioned loss surface and make gradient descent behave poorly. The same problem occurs at every layer of a deep network, not just the first one - and it is aggravated by the fact that training is a dynamic process.

Consider a layer l receiving activations from layer $l - 1$ as inputs. The distribution of outputs of layer l depends on two things: the data and the parameters of all preceding layers. After a gradient update, both change simultaneously. The parameters of layer l and all preceding layers update, changing the distribution of outputs each layer produces affecting all subsequent layers down the road. This effect compounds with depth: a layer deep in the network receives inputs that have been transformed through many layers, each of which updated its parameters, so the distribution it sees can shift dramatically between update steps. The non-linearity amplifies this further - a shift in the pre-activation distribution can produce a very different post-activation distribution, potentially pushing activations into saturating regions where gradients vanish, or dramatically increasing the variance of the activations. This phenomenon of changing distributions within layers is called **internal covariate shift** (Ioffe and Szegedy 2015).

The consequences are the same as for unnormalised inputs: slower convergence, need for lower learning rates, and high sensitivity to initialisation. The solution follows the same logic too - if normalising the inputs to the network stabilises the first layer, normalising the activations at every layer stabilises all of them. This is exactly what **batch normalisation** does: it explicitly normalises the activations at each layer to have zero mean and unit variance within each mini-batch, then applies learnable affine transformations:

$$\hat{h}_j^{(i)} = \frac{h_j^{(i)} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}, \quad \tilde{h}_j^{(i)} = \gamma_j \hat{h}_j^{(i)} + \beta_j, \quad \mu_j = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} h_j^{(i)}, \quad \sigma_j^2 = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} (h_j^{(i)} - \mu_j)^2 \quad (6.5)$$

where μ_j and σ_j^2 are the mean and variance of feature j computed across all examples $i \in \mathcal{B}$ in the current mini-batch, ϵ is a small constant for numerical stability, and γ_j and β_j are learnable per-feature scale and shift parameters that allow the normalised activation to be rescaled and shifted. By keeping the input distribution to each layer approximately stable, batch normalisation allows higher learning rates, reduces sensitivity to initialisation, and makes training of very deep networks practical.

During training, the normalisation uses statistics computed from the current mini-batch - mean and variance are different for each batch, injecting a small amount of

noise into the normalisation. This is analogous to the noise in mini-batch SGD and has a **mild implicit regularisation effect**, though this is not the primary motivation for using batch normalisation.

During evaluation, mini-batch statistics are replaced by **running statistics** - exponential moving averages of μ_B and σ_B^2 accumulated during training. This gives deterministic, stable behaviour at test time: the normalisation no longer depends on which other examples happen to be in the same batch. This is why batch normalisation is one of the layers affected by `model.train()` and `model.eval()` - forgetting to call `model.eval()` before inference means the network normalises using noisy mini-batch statistics rather than the stable running statistics, producing wrong and irreproducible predictions.

In PyTorch, `nn.BatchNorm1d` is used for fully connected networks and `nn.BatchNorm2d` for convolutional networks - covered in a later session. Batch norm is typically inserted after the linear transformation and before the activation function.

Batchnorm and Dropout. Using batch norm and dropout together requires care. The fundamental problem is a **variance shift** (Li et al. 2019): during training with dropout active, some activations are zeroed, reducing the variance of the distribution entering batch norm. The running statistics accumulated during training therefore reflect this reduced variance. At test time, dropout is disabled and all activations are present - the variance is higher, but the running statistics used for normalisation were computed under the lower-variance training distribution. This mismatch produces systematically wrong normalisation at test time and degrades performance. In practice, three approaches are used:

- **Batch norm only** - sufficient in many cases, particularly in convolutional networks where batch norm provides strong regularisation on its own
- **Dropout only** - effective in MLPs with no interaction problems
- **Both** - possible if dropout is placed *after* batch norm, so that running statistics are computed on the full activations; results may still be suboptimal and require careful tuning

An alternative that avoids these complications entirely is **layer normalisation** (J. L. Ba, Kiros, and G. E. Hinton 2016). Rather than normalising across the batch dimension per feature, layer norm normalises across the feature dimension per example:

$$\hat{h}_j^{(i)} = \frac{h_j^{(i)} - \mu^{(i)}}{\sqrt{\sigma^{2(i)} + \epsilon}}, \quad \tilde{h}_j^{(i)} = \gamma_j \hat{h}_j^{(i)} + \beta_j \quad \mu^{(i)} = \frac{1}{D} \sum_{j=1}^D h_j^{(i)}, \quad \sigma^{2(i)} = \frac{1}{D} \sum_{j=1}^D (h_j^{(i)} - \mu^{(i)})^2 \quad (6.6)$$

where $\mu^{(i)}$ and $\sigma^{2(i)}$ are the mean and variance computed across all D features of example i , ϵ is a small constant for numerical stability, and γ_j and β_j are learnable per-feature scale and shift parameters. Because statistics are computed per example independently, there are no running statistics to accumulate and no train/eval difference - layer norm behaves identically in both modes. This means layer norm and dropout combine cleanly. In MLPs, the typical order is: `nn.Linear` $\rightarrow$ `nn.LayerNorm` $\rightarrow$ activation $\rightarrow$ `nn.Dropout`. Transformers use a related but slightly different pattern involving residual connections - this will be covered when we study sequence models.

Batch norm and layer norm are two points in a larger design space of normalisation techniques. **Group norm** normalises across groups of channels per example and is useful when batch sizes are small. **RMS norm** is a simplified layer norm without mean

subtraction, used in modern large language models such as LLaMA. The field has not settled on a single winner - the right choice depends on the architecture, batch size, and task.

6.4.2. Vanishing and Exploding Gradients

The second major training instability arises during backpropagation. The gradient of the loss with respect to the parameters of the first layer involves a product of Jacobians across all layers:

$$\frac{\partial \mathcal{L}}{\partial \theta^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(L)}} \cdot \frac{\partial \mathbf{h}^{(L)}}{\partial \mathbf{h}^{(L-1)}} \cdots \frac{\partial \mathbf{h}^{(2)}}{\partial \mathbf{h}^{(1)}} \cdot \frac{\partial \mathbf{h}^{(1)}}{\partial \theta^{(1)}}. \quad (6.7)$$

This product of L Jacobian matrices compounds as it travels backwards from the loss to the earliest layers. If the singular values of the Jacobians are consistently less than 1, the product shrinks exponentially with depth - **vanishing gradients**. The earliest layers receive gradients close to zero, their parameters barely update, and the deep network effectively behaves as a shallow one. If the singular values are consistently greater than 1, the product grows exponentially - **exploding gradients**. The earliest layers receive enormous gradients, a single update step moves the parameters by a huge amount, and training diverges.

For standard MLPs and CNNs, **Xavier and He initialisation** discussion in section 5.4.2 and **batch normalisation** of section 6.4.1 largely solve the vanishing and exploding gradient problem in practice. Initialisation keeps the Jacobian singular values close to 1 at the start of training; batch normalisation stabilises the forward pass distributions throughout training, which indirectly stabilises the backward pass. Gradient instability is therefore rarely a practical concern for these architectures.

The situation is more severe in recurrent networks and transformers, where gradients must propagate through long sequences of dependencies. Here, initialisation and normalisation alone are not sufficient and dedicated solutions are needed. **Gradient clipping** rescales the gradient if its norm exceeds a threshold τ before the update step, preventing any single step from being catastrophically large. In PyTorch: `torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)`, called between `loss.backward()` and `optimizer.step()`. **Spectral normalisation** constrains the spectral norm of weight matrices, bounding the Jacobian singular values by construction. Both are standard tools in recurrent and attention-based architectures, which we will study in later sessions.

6.4.3. Dying ReLU and Activation Functions

The third instability is the **dying ReLU** problem. The ReLU activation function has a zero gradient for all negative pre-activations:

$$\frac{\partial \text{ReLU}(z)}{\partial z} = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0. \end{cases} \quad (6.8)$$

This sounds alarming: ReLU has zero gradient for half the input space. In a deep network, the vanishing gradient problem would seem to be inevitable.

In practice, the dying ReLU problem is less severe than it first appears. A neuron dies only if its pre-activation is negative for *all* training examples simultaneously - in a diverse dataset, pre-activations vary across examples and the average

gradient over a mini-batch is nonzero. He initialisation further reduces the risk by keeping pre-activations in a healthy range at the start of training. Batch normalisation helps too: by keeping the pre-activation distribution centred at zero throughout training, it prevents the drift toward predominantly negative values that would increase the risk of neurons dying. It also guarantees that approximately half the pre-activations remain positive at all times - and for sufficiently wide networks, this is always enough active neurons to represent the necessary functions. ReLU therefore remains the recommended baseline: simple, computationally cheap, and well-protected by the standard training recipe of He initialisation and batch normalisation.

When dying ReLU does become a problem, the fix is to use an activation function that has a nonzero gradient for negative inputs. **Leaky ReLU** introduces a small negative slope α :

$$\text{LeakyReLU}(z) = \max(\alpha z, z), \quad \alpha \ll 1. \quad (6.9)$$

ELU (Exponential Linear Unit) uses a smooth exponential curve for negative inputs, avoiding the hard zero entirely. More recent activations - **GELU**, **Swish**, and **Mish** - are smooth, differentiable everywhere, and are used in modern architectures such as transformers. There is no universal winner; ReLU and LeakyReLU cover most practical needs for the feedforward networks considered in this course.

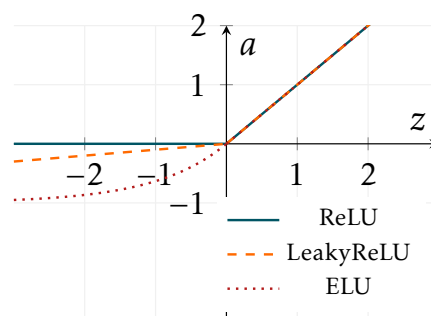


Figure 6.6: ReLU, LeakyReLU ($\alpha = 0.1$), and ELU. LeakyReLU introduces a small negative slope; ELU uses a smooth exponential curve. Both maintain a nonzero gradient for negative pre-activations.

6.5. Embracing Randomness

A recurring observation throughout this lecture is that randomness appears at every stage of deep learning training - not as a nuisance to be eliminated but as a deliberate feature that contributes to generalisation. Table 1 summarises the main sources of randomness and their effects.

► Session 7:
Embracing
Randomness,
Summary

Source	Where	Effect
Weight initialisation	start of training	which minimum is found
Data shuffling	each epoch	different batches, prevents order memorisation
Mini-batch SGD	every update step	implicit regularisation, flat minima
Dropout	forward pass	ensemble effect, prevents co-adaptation
Data augmentation	data loading	invariance, population coverage

Table 1: Sources of randomness in deep learning training and their effects on generalisation.

What is striking is that none of these are accidents. Each reflects a deliberate design choice that the field has converged on precisely because it improves generalisation. Weight initialisation is random to break symmetry and explore different regions of the loss landscape. Shuffling is random to prevent the model from exploiting the order of examples. Mini-batch SGD is stochastic by design - full-batch gradient descent is

available but rarely used, because the noise from mini-batching implicitly regularises. Dropout injects noise into the forward pass deliberately. Data augmentation applies random transformations by choice.

The deeper insight is that the right question is not how to make training more deterministic, but how to design the right kinds of randomness. This theme extends well beyond the techniques covered in this lecture. More advanced methods in representation learning, generative modelling, and self-supervised learning return to randomness again and again - as a source of supervision, as a form of regularisation, or as the fundamental mechanism by which a model learns to represent uncertainty. Embracing randomness, rather than treating it as an obstacle, is one of the defining characteristics of modern deep learning.

7. Classification & Maximum Likelihood

► Session 8

Sessions 1–7 developed the full machinery for training deep networks on regression problems - forward pass, backpropagation, optimisation, regularisation, and the PyTorch API, all using mean squared error as the loss. This session introduces the second major problem type: classification. We begin with the practical setup - what classification is, why accuracy cannot be directly optimised, and how sigmoid and softmax map network outputs to probability distributions. We then develop the theoretical foundation through maximum likelihood estimation, showing that MSE and cross-entropy are not arbitrary choices but principled consequences of distributional assumptions about the data. Finally, information theory gives a geometric interpretation of cross-entropy as a measure of distributional distance, and we close with practical considerations for implementing classification in PyTorch.

7.1. Classification Problem

7.1.1. From Regression to Classification

► Session 8:
Classification
Problem, From
Accuracy to Loss,
What We Need

In regression, the output space is continuous: $y \in \mathbb{R}$ (or $y \in \mathbb{R}^m$ for multi-output regression). The network predicts a real-valued quantity and the loss measures the distance between prediction and target. In classification, the output space is discrete: $y \in \{1, \dots, K\}$ for K classes. The network must assign each input to one of a finite set of categories - image class, spam or not, medical diagnosis. There is no natural notion of distance between classes: class 3 is not “between” class 2 and class 4 in any meaningful sense.

7.1.2. Accuracy and Why We Cannot Optimise It Directly

The natural performance measure for classification is **accuracy** - the fraction of examples correctly classified:

$$\text{acc} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}^{(i)} = y^{(i)}], \quad (7.1)$$

where $\hat{y}^{(i)}$ is the predicted class for example i and $\mathbf{1}[\cdot]$ is the indicator function. Despite being the quantity we care about, accuracy cannot be used as a training loss. The network outputs real numbers $f_{\theta}(\mathbf{x}) \in \mathbb{R}$ rather than class labels, so the indicator

function cannot be applied directly. Moreover, even if we knew how to predict classes from a neural network, the indicator function is piecewise constant - its gradient with respect to the network parameters is zero almost everywhere and undefined at the discontinuities, as illustrated in Figure 7.1. Accuracy would provide no useful signal for gradient descent.

The key idea is to move from predicting numbers to predicting a **probability distribution over classes**. Instead of predicting a hard class label - "this example is of class k " - the network predicts the probability that an example belongs to each class: $p(y = k | \mathbf{x}; \theta)$ for all k . A smooth, differentiable loss function can then be defined over these probabilities rather than over hard class labels, as illustrated in Figure 7.1.

How do we make the neural network predict probabilities? In short, we do not. The network continues to output real-valued **logits** $\mathbf{z} = f_{\theta}(\mathbf{x}) \in \mathbb{R}^K$ for every category $k \in 1, \dots, K$ which we then pass into a **suitable transformation** function so that the transformed values can be interpreted as probabilities. A valid probability distribution must satisfy these properties:

$$p_k \geq 0 \quad \forall k \quad \text{and} \quad \sum_{k=1}^K p_k = 1. \quad (7.2)$$

The next two sections introduce these transformations for binary and multiclass classification respectively.

7.2. Binary Classification

In binary classification there are two classes, $y \in \{0, 1\}$, and the output is fully determined by a single probability - the probability of class 1, since class 0 has probability $1 - \hat{p}$ by complementarity. The network therefore can output a single logit $z = f_{\theta}(\mathbf{x}) \in \mathbb{R}$ and we need a function that maps this real-valued logit to $(0, 1)$ - the space of valid probabilities. The **sigmoid function** is the natural choice:

$$\hat{p} = \sigma(z) = \frac{1}{1 + e^{-z}}. \quad (7.3)$$

It is monotone, smooth everywhere, and maps the entire real line to the open interval $(0, 1)$. Figure 7.2 shows its shape: symmetric around $\sigma(0) = 0.5$, saturating toward 1 for large positive z and toward 0 for large negative z . The output $\hat{p}$ is interpreted as $p(y = 1 | \mathbf{x}; \theta)$ - the model's estimated probability that the input belongs to class 1. The probability of class 0 is $1 - \hat{p}$.

The decision boundary between the two classes is at $z = 0$: if the logit (the output of the NN) is positive the model predicts class 1, if negative it predicts class 0. Importantly,

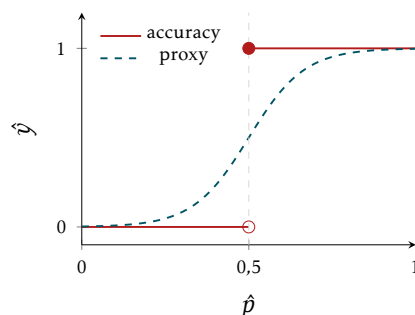


Figure 7.1: Step function (accuracy) vs a smooth proxy. The step function has zero gradient almost everywhere; the smooth proxy provides a useful gradient signal for optimisation.

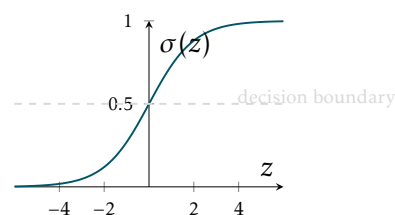


Figure 7.2: Sigmoid function. Maps any real-valued logit to $(0, 1)$, symmetric around $\sigma(0) = 0.5$.

at inference time there is no need to compute $\hat{p}$ explicitly - since sigmoid is monotone, the sign of z alone determines the predicted class.

7.2.1. Binary Cross-Entropy

With the sigmoid providing a predicted probability $\hat{p} \in (0, 1)$, we need a loss function defined over probabilities rather than raw numbers. MSE is inappropriate here - it does not respect the probabilistic structure of the output and penalises predictions symmetrically regardless of confidence. What we want is a loss that is zero when the model assigns all probability to the correct class and grows without bound as it becomes confidently wrong. The $-\log$ function has exactly this shape: $-\log \hat{p} \rightarrow 0$ as $\hat{p} \rightarrow 1$ and $-\log \hat{p} \rightarrow \infty$ as $\hat{p} \rightarrow 0$. This motivates the **binary cross-entropy** loss:

$$\ell_{BCE}(\hat{p}, y) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p}). \quad (7.4)$$

The two terms activate depending on the true label: when $y = 1$ only the first term contributes, penalising low $\hat{p}$; when $y = 0$ only the second term contributes, penalising high $\hat{p}$. Figure 7.3 shows both curves. This choice is not arbitrary - Section 7.4 will show that binary cross-entropy is the exact consequence of assuming a Bernoulli data-generating process and maximising the likelihood.

The training loss is the average binary cross-entropy over all examples:

$$\mathcal{L}_{BCE}(\theta) = -\frac{1}{N} \sum_{i=1}^N [y^{(i)} \log \hat{p}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)})], \quad (7.5)$$

where $\hat{p}^{(i)} = \sigma(f_{\theta}(\mathbf{x}^{(i)}))$ is the predicted probability for example i . Minimising this loss over the training set pushes the model to assign high probability to the correct class for each example.

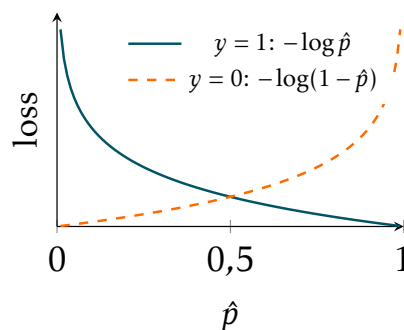


Figure 7.3: Binary cross-entropy loss as a function of predicted probability $\hat{p}$. Both terms go to infinity when the model is confidently wrong.

7.3. Multiclass Classification

Generalising from binary to multiclass classification, we now have K classes and the network produces K logits $\mathbf{z} = f_{\theta}(\mathbf{x}) \in \mathbb{R}^K$. We need a transformation that maps these to a valid probability distribution - all K values must be non-negative and sum to 1 simultaneously.

Simple hack would be to make the values positive, e.g. by absolute value or square, and divide by the sum across the categories. This way all would be positive and would sum to 1. The challenge is, however, in preserving the ordering of the logits: class with higher logit should receive higher probability. Simple squaring or absolute values destroy the sign information - a logit of -3 and a logit of $+3$ would produce the same value, making it impossible to distinguish a strong negative prediction from a strong positive one. On the other hand, the exponential e^{z_k} is a much better choice: it is strictly positive, differentiable everywhere, and strictly monotone - larger logits always produce larger values, preserving the preference ordering. Dividing by the sum ensures

normalisation so that accross categoriees we have got the correct total probability. This gives us the **softmax function**:

$$\hat{p}_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad k = 1, \dots, K. \quad (7.6)$$

which satisfies $\hat{p}_k > 0$ for all k and $\sum_k \hat{p}_k = 1$, and is differentiable everywhere.

Relationship to softmax. Binary classification could in principle be handled as a special case of multiclass classification with $K = 2$ - using two output nodes, softmax, and categorical cross-entropy. As shown in equation (7.7), the two are mathematically equivalent:

$$\hat{p}_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2}} = \frac{1}{1 + e^{-(z_1 - z_2)}} = \sigma(z_1 - z_2). \quad (7.7)$$

In practice the two-output formulation is not used. With two logits, only the difference $z_1 - z_2$ determines the output - the individual values of z_1 and z_2 are irrelevant. One degree of freedom is therefore redundant and unidentifiable from the loss. A single output node with sigmoid eliminates this redundancy: the single logit z directly encodes the relative evidence for class 1 over class 0 with no superfluous parameter. Therefore the binary formulation is the standard choice whenever there are exactly two classes.

7.3.1. Categorical Cross-Entropy

With softmax producing a valid probability distribution $\hat{\mathbf{p}}$ over K classes, we need a loss function that measures how well this distribution matches the true label. Labels are represented as **one-hot vectors** $\mathbf{y} \in \{0, 1\}^K$, where $y_k = 1$ for the correct class and $y_k = 0$ for all others. By the same argument as for binary cross-entropy - we want a loss that is zero when the model is confident and correct, and grows without bound when it is confident and wrong - the natural choice is the negative log probability of the correct class. The **categorical cross-entropy** loss is:

$$\ell_{CE}(\hat{\mathbf{p}}, \mathbf{y}) = - \sum_{k=1}^K y_k \log \hat{p}_k = -\log \hat{p}_y, \quad (7.8)$$

where $\hat{p}_y$ is the predicted probability of the correct class. Since only the term for the correct class survives the sum (all other $y_k = 0$), the loss simply measures how much probability the model assigns to the right answer.

The training loss averages over all examples:

$$\mathcal{L}_{CE}(\boldsymbol{\theta}) = -\frac{1}{N} \sum_{i=1}^N \log \hat{p}_{y^{(i)}}^{(i)}. \quad (7.9)$$

One important property to note: the loss is minimised as $\hat{p}_y \rightarrow 1$, meaning the model is always rewarded for increasing its confidence in the correct class - it is never penalised for being overconfident. This has practical consequences for calibration that we will discuss in Section 7.7.1. The principled derivation of this loss from maximum likelihood will be given in Section 7.4.

7.4. Maximum Likelihood

The losses introduced above were motivated intuitively - they penalise wrong predictions in sensible ways and work well in practice. But why MSE for regression and cross-entropy for classification? Could we have chosen differently? The answer is yes - many loss functions are conceivable - but not all choices are equally well-motivated. **Maximum likelihood (ML) estimation** provides a principled framework for making this choice: given an explicit assumption about how the data was generated, ML identifies the most suitable loss function as a direct consequence of that assumption. The choice is no longer a design decision made by intuition but a logical consequence of what we believe about the data-generating process. Different assumptions lead to different losses, and being explicit about the assumption makes the choice transparent and justifiable. We will see that MSE and cross-entropy are not two unrelated design choices but instances of the same principle under different distributional assumptions.

7.4.1. Maximum Likelihood Principle

Real data is noisy. Even for the same input $\mathbf{x}$, repeated measurements of the output y would differ - due to measurement error, natural variability, or factors not captured by $\mathbf{x}$. A deterministic model $f_{\theta}(\mathbf{x})$ produces a single prediction and ignores this variability entirely. A **probabilistic model** instead specifies a conditional distribution $p(y | \mathbf{x}; \theta)$ over outputs given inputs, explicitly modelling the uncertainty in the output. The network f_{θ} defines the parameters of this distribution - its mean for regression, or its class probabilities for classification.

Given that the model assigns a probability to every possible output, we can ask: how probable is the data we actually observed under this model? If the model is a good fit for the data, the observed outputs should be highly probable under it; if it is a poor fit, the observed outputs should be improbable. This suggests fitting the model by choosing parameters that make the observed data as probable as possible.

The **likelihood** function $L(\theta) = p(y | \mathbf{x}; \theta)$ formalises this idea: it is the same probability of the observed data under the model, but now viewed as a function of the parameters θ with the data $(\mathbf{x}, y)$ fixed. Figure 7.4 illustrates the intuition: the petrol distribution assigns high probability to the the observed data points - high likelihood that data were generated from this distribution, while the orange distribution assigns low probability - low likelihood of this parameterization. ML method chooses the parameters that maximise this probability - the distribution that most plausibly generated the observed data.

So far we have considered the likelihood of a single observation $(\mathbf{x}, y)$. In practice we have a full dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$ and we want to find parameters that make the *entire* dataset probable - not just one example. The likelihood we want to maximise

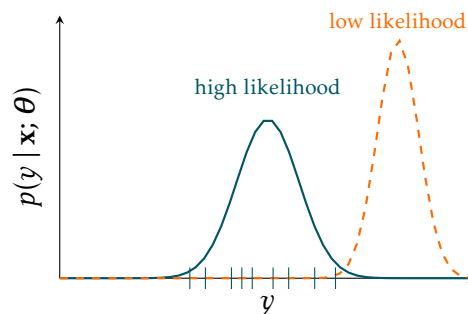


Figure 7.4: Likelihood intuition. Data points (ticks) were generated by one of two distributions. The petrol distribution is centred on the data - it is a plausible generator. The orange distribution is shifted and narrower - it could have generated the data, but with much lower probability - low likelihood.

is therefore the joint probability of all observed outputs $L(\theta) = p(\mathcal{D}; \theta)$. To write this down, we need an assumption about the relationship between examples. The standard assumption is that examples are **independently and identically distributed** (i.i.d.): **independent** means that knowing the output of one example tells us nothing about the outputs of others; **identically distributed** means all examples are drawn from the same distribution p_{data} . Together, these assumptions mean the joint probability of the dataset factorises into a product over individual examples:

$$L(\theta) = p(\mathcal{D}; \theta) = \prod_{i=1}^N p(y^{(i)} | \mathbf{x}^{(i)}; \theta). \quad (7.10)$$

Maximum likelihood estimation searches over all possible parameter settings and selects the one that makes the observed dataset most probable under the model:

$$\theta^* = \arg \max_{\theta} L(\theta). \quad (7.11)$$

Working directly with the likelihood product is unwieldy: multiplying many probabilities together quickly produces extremely small numbers that are both hard to interpret and prone to numerical underflow. Taking the logarithm resolves this. Since log is strictly monotone, maximising $\log L(\theta)$ gives exactly the same solution as maximising $L(\theta)$ - the log does not change which parameter setting wins, only the scale on which we measure it. The **log-likelihood** turns the product into a sum, which is far more tractable both numerically and analytically:

$$\log L(\theta) = \sum_{i=1}^N \log p(y^{(i)} | \mathbf{x}^{(i)}; \theta). \quad (7.12)$$

Finally, in machine learning we prefer to minimize our objectives rather than maximize. This is easy to resolve as **maximising the log-likelihood** is equivalent to **minimising the negative log-likelihood** (NLL) - just flip the sign and we can minimize:

$$\theta^* = \arg \max_{\theta} \log L(\theta) = \arg \min_{\theta} \underbrace{-\frac{1}{N} \sum_{i=1}^N \log p(y^{(i)} | \mathbf{x}^{(i)}; \theta)}_{\mathcal{L}(\theta)}. \quad (7.13)$$

This NLL is our loss function.

So far the derivation has been abstract - we have worked with a generic distribution $p(y | \mathbf{x}; \theta)$ without specifying what it is. In practice, this is where we make a concrete assumption about the data-generating process and plug it in. The result is a specific, usable loss function. The choice of distributional assumption $p(y | \mathbf{x}; \theta)$ determines which loss emerges - different assumptions lead to different losses. The next three subsections work through the three cases we care about: Gaussian for regression, Bernoulli for binary classification, and Categorical for multiclass classification, but in more advanced methods such as deep generative modeling this is where the magic often happens.

7.4.2. Gaussian Assumption → MSE

For regression, the natural probabilistic assumption is that the true output is the network prediction plus independent Gaussian noise $y = f_{\theta}(\mathbf{x}) + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma^2)$: the network $f_{\theta}(\mathbf{x})$ predicts the mean of a Gaussian distribution centred on the true relationship, and each observation deviates from this mean by a random noise term. Figure 7.5 illustrates this: at each input x , the conditional distribution $p(y | x; \theta)$ is a Gaussian centred on the regression line, and the data points scatter around it. Formally:

$$p(y | \mathbf{x}; \theta) = \mathcal{N}(y; f_{\theta}(\mathbf{x}), \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - f_{\theta}(\mathbf{x}))^2}{2\sigma^2}\right). \quad (7.14)$$

Taking the log-likelihood for one example yields:

$$\log p(y | \mathbf{x}; \theta) = -\frac{(y - f_{\theta}(\mathbf{x}))^2}{2\sigma^2} - \frac{1}{2} \log(2\pi\sigma^2). \quad (7.15)$$

The NLL averaged over the training set is:

$$-\frac{1}{N} \sum_{i=1}^N \log p(y^{(i)} | \mathbf{x}^{(i)}; \theta) = \frac{1}{2\sigma^2} \underbrace{\frac{1}{N} \sum_{i=1}^N (y^{(i)} - f_{\theta}(\mathbf{x}^{(i)}))^2}_{\text{MSE}} + \text{const.} \quad (7.16)$$

Here we see how the MSE loss emerges from minimising the NLL under a Gaussian assumption. The $\frac{1}{2\sigma^2}$ and the additive constant do not affect which θ minimises the expression and hence we can freely ignore them when searching for the network parameters θ . We see that MSE is not an arbitrary choice but the principled consequence of assuming Gaussian observation noise. This connects back to Session ?? where MSE was introduced: we now understand why it is the right loss for regression under this assumption.

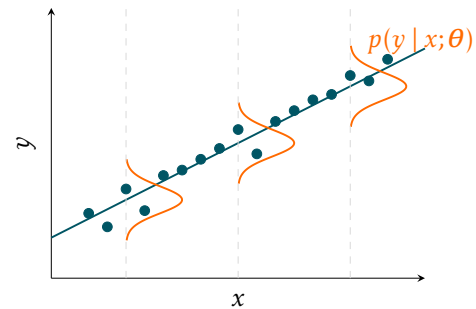


Figure 7.5: Regression under Gaussian noise. The network predicts the mean of a Gaussian distribution at each input x . Data points scatter around the regression line, with the conditional distribution $p(y | x; \theta)$ shown at three locations.

7.4.3. Bernoulli

Assumption → Binary Cross-Entropy

For binary classification, the label $y \in \{0, 1\}$ takes exactly the values of a Bernoulli random variable - it is either 0 or 1, with no intermediate values. The natural probabilistic assumption is therefore that the label follows a Bernoulli distribution with success probability $\hat{p} = \sigma(f_{\theta}(\mathbf{x}))$, where the network predicts the logits (also known as log-odds) and sigmoid converts it to a probability. The Bernoulli distribution has a particularly clean form: it assigns probability $\hat{p}$ to outcome 1 and probability $1 - \hat{p}$ to outcome 0, which can be written compactly as:

$$p(y | \mathbf{x}; \theta) = \text{Bernoulli}(y; \hat{p}) = \hat{p}^y (1 - \hat{p})^{1-y}. \quad (7.17)$$

It is worth pausing to check this formula: when $y = 1$ the expression gives $\hat{p}^1(1 - \hat{p})^0 = \hat{p}$, and when $y = 0$ it gives $\hat{p}^0(1 - \hat{p})^1 = 1 - \hat{p}$. Taking the log:

$$\log p(y | \mathbf{x}; \boldsymbol{\theta}) = y \log \hat{p} + (1 - y) \log(1 - \hat{p}). \quad (7.18)$$

The NLL averaged over the training set is:

$$-\frac{1}{N} \sum_{i=1}^N \log p(y^{(i)} | \mathbf{x}^{(i)}; \boldsymbol{\theta}) = -\frac{1}{N} \sum_{i=1}^N \underbrace{\left[y^{(i)} \log \hat{p}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right]}_{\mathcal{L}_{BCE}(\boldsymbol{\theta})}. \quad (7.19)$$

This is exactly the binary cross-entropy loss from Section 7.2. Minimising the NLL under a Bernoulli assumption is equivalent to minimising binary cross-entropy - the loss we motivated intuitively now has a principled derivation.

7.4.4. Categorical Assumption $\rightarrow$ Categorical Cross-Entropy

For multiclass classification, the label $y \in \{1, \dots, K\}$ takes one of K discrete values - exactly the support of a Categorical distribution. The natural probabilistic assumption is that the label follows a Categorical distribution with class probabilities $\hat{\mathbf{p}} = \text{softmax}(f_{\boldsymbol{\theta}}(\mathbf{x}))$, where the network predicts the logits and softmax converts them to a valid probability distribution. Using one-hot encoding $\mathbf{y} \in \{0, 1\}^K$, the Categorical distribution can be written compactly as:

$$p(y | \mathbf{x}; \boldsymbol{\theta}) = \text{Categorical}(y; \hat{\mathbf{p}}) = \prod_{k=1}^K \hat{p}_k^{y_k} = \hat{p}_y. \quad (7.20)$$

Again it is worth checking: the product $\prod_k \hat{p}_k^{y_k}$ has all terms equal to $\hat{p}_k^0 = 1$ except for the correct class $k = y$, where the term is $\hat{p}_y^1 = \hat{p}_y$. The whole expression therefore collapses to just the predicted probability of the correct class. Taking the log:

$$\log p(y | \mathbf{x}; \boldsymbol{\theta}) = \sum_{k=1}^K y_k \log \hat{p}_k = \log \hat{p}_y. \quad (7.21)$$

The NLL averaged over the training set is:

$$-\frac{1}{N} \sum_{i=1}^N \log p(y^{(i)} | \mathbf{x}^{(i)}; \boldsymbol{\theta}) = -\frac{1}{N} \sum_{i=1}^N \underbrace{\log \hat{p}_{y^{(i)}}^{(i)}}_{\mathcal{L}_{CE}(\boldsymbol{\theta})}. \quad (7.22)$$

This is exactly the categorical cross-entropy loss from Section 7.3. Minimising the NLL under a Categorical assumption is equivalent to minimising categorical cross-entropy - completing the picture: all three losses we have been using are maximum likelihood estimators under their respective distributional assumptions.

7.5. Information Theory

Maximum likelihood gave us a statistical justification for cross-entropy: it is the NLL under a Categorical assumption. But this derivation treats cross-entropy as a computational object - a formula that emerges from an algebraic manipulation. It does not tell us what cross-entropy *means* geometrically, or why it is a natural measure of the distance between two probability distributions.

Information theory, developed by Claude Shannon in his landmark 1948 paper (Shannon 1948), was originally motivated by a practical engineering problem: how much can a message be compressed without losing information, and at what rate can information be transmitted reliably over a noisy channel? To answer these questions, Shannon developed a mathematical theory of information that quantifies uncertainty, surprise, and the cost of miscommunication. The central concepts - entropy, cross-entropy, and KL divergence - turn out to be exactly the right tools for understanding what happens when we train a classifier and in extension any machine learning model.

The connection is this: training minimises the cross-entropy loss, which is equivalent to minimising the KL divergence between the true label distribution and the model's predicted distribution. KL divergence measures how far apart two distributions are in the information-theoretic sense - minimising it means bringing the model as close as possible to the truth. This reframes training not as curve fitting but as distribution matching, which is a more fundamental and illuminating perspective. This section builds up the necessary concepts from first principles.

7.5.1. Information Content

We begin with the most basic question: how much information does a single event carry? Intuitively, learning that a rare event occurred is more surprising and more informative than learning that a common one did. If someone tells you it rained in London today, you learn little - it happens often. If they tell you it snowed in June, you learn a lot. We want to quantify this notion of surprise formally, as a function of the event's probability.

The **information content** (or self-information) of event k occurring with probability p_k is:

$$I(k) = -\log_2 p_k. \quad (7.23)$$

The unit is **bits** when using $\log_2$. A certain event ($p_k = 1$) carries zero information - you already knew it would happen. A coin flip ($p_k = 0.5$) carries 1 bit - you needed one binary question to predict it. A rare event ($p_k = 0.01$) carries approximately 6.6 bits - it takes many binary questions to narrow it down.

Why $-\log$? The logarithm is not an arbitrary choice. Three natural requirements uniquely determine the form $I(k) = -\log p_k$ (Shannon 1948):

1. **Monotonicity:** more probable events carry less information - I should be a decreasing function of p_k
2. **Additivity:** for two independent events A and B , the information of both occurring should equal the sum of their individual informations: $I(A \cap B) = I(A) + I(B)$. Since $p(A \cap B) = p(A)p(B)$ for independent events, this requires a function satisfying $f(p \cdot q) = f(p) + f(q)$, which forces $f(p) = -\log p$

3. **Continuity:** small changes in probability should produce small changes in information - no discontinuous jumps

Beyond the axiomatic justification, Shannon's source coding theorem (Shannon 1948) gives $-\log_2 p_k$ a concrete operational meaning: it is the minimum number of bits needed to encode event k in an optimal lossless compression scheme. Information content is not just a mathematical abstraction - it is the fundamental currency of compression.

Morse code illustrates this concretely. Morse code encodes letters of English alphabet into a series of dashes and dots (equivalents of zeros 0 and ones 1) based on their frequency as they appear in standard English language. The idea is to make communication and transmission of messages efficient - you use few bits (dots or dashes) for encoding very common letters and you can afford to use long codes for rare letters as they anyway do not appear very often. Letters actually have very different frequencies - for example E and T are common, Q and Z are rare. Morse code assigns short codes to such frequent letters (E is a single dot, T is a single dash) and longer codes to rare ones (Q is - - . -, Z is - - . .).

This matches the information content principle: frequent letters carry little information and deserve short codes; rare letters carry a lot of information and require longer ones. The optimal code length for letter k is $-\log_2 p_k$ bits, where p_k is the probability of that letter occurring in English language - exactly the information content.

Note that in machine learning we typically do not work with $\log_2$ but with natural logarithm $\log$. This changes the *currency* from bits into something we call **nats** but does not change the fundamental principles.

7.5.2. Entropy

Information content tells us how many bits (zeros and ones, or dots and dashes) to assign to a single symbol. A message is a sequence of many symbols, each encoded independently. The total length of a message is the sum of the code lengths of its symbols. If we use optimal codes - assigning $-\log_2 p_k$ bits to symbol k - the total length of a message of n symbols is $\sum_i (-\log_2 p_{k_i})$. How long is the encoding of a typical symbol? The answer is the expected code length, which is the **entropy**:

$$H(p) = \mathbb{E}_{k \sim p}[I(k)] = - \sum_k p_k \log p_k. \quad (7.24)$$

Entropy tells us the average number of bits per symbol under optimal encoding - how long our messages will be on average.

Consider two extremes - see figure 7.6. If the distribution is highly concentrated - a few symbols are very frequent and the rest almost never appear - we can assign very short codes to the frequent ones. Since they dominate the messages, the average code length is short. This is **low entropy**: the distribution is uneven, the frequent symbols are easy to predict, and messages compress well.

English is like this: E constitutes roughly 13% of standard English text, T about 9%, while Q and Z appear in less than 0.1% - try it, take some English text and calculate the

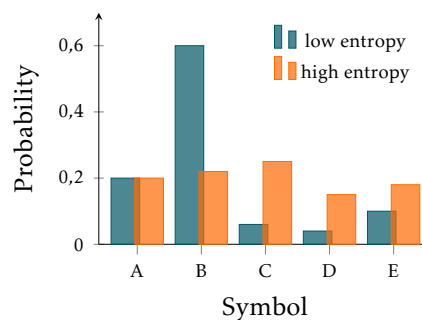


Figure 7.6: Low entropy (petrol): one dominant symbol, distribution concentrated, messages compress well. High entropy (orange): near-uniform distribution, no dominant symbol, messages are longer on average.

letter frequencies! An English text compresses efficiently because most characters are predictable. If the distribution is more even - many symbols with similar frequencies - we cannot benefit from assigning short codes to a few dominant ones. Every symbol needs a reasonably long code, and messages are longer on average. This is **high entropy**: the distribution is spread out and messages compress poorly.

This connects directly to uncertainty. When we randomly pick a letter from an English text, it is often E - we can make a reasonable guess. Low entropy means we can bet on the outcome: the distribution is concentrated enough that one or a few outcomes are much more likely than others, and the bet is not too risky. High entropy means the distribution is close to uniform - any symbol is about as likely as any other, guessing is hard, and betting is risky. Entropy is therefore simultaneously a measure of compression efficiency, predictability, and uncertainty. No lossless encoding can achieve an average code length below $H(p)$ bits per symbol - it is a fundamental lower bound on compression (Shannon 1948).

7.5.3. Binary Entropy and Coding

For a binary distribution $p = (p, 1 - p)$ entropy is:

$$H(p) = -p \log_2 p - (1 - p) \log_2 (1 - p). \quad (7.25)$$

This is shown in figure 7.7 as a function of the probability p . A fair coin ($p = 0.5$) has entropy 1 bit - the optimal code assigns 1 bit per outcome, matching practice. A biased coin ($p = 0.9$) has entropy approximately 0.47 bits - theory says we can encode the information about the outcome of the coin flip with shorter messages than 1 bit per symbol on average. However, this bound is only achievable in practice by encoding long sequences together rather than one symbol at a time. A single binary outcome always requires at least 1 bit - the minimum addressable unit. The theoretical savings come from treating a whole sequence as a single message: a long sequence of biased coin flips can be compressed well below 1 bit per flip on average, because the sequence as a whole has far fewer probable patterns than the 2^n possible ones. The entropy bound applies to the per-symbol rate in the limit of long sequences, not to individual symbols.

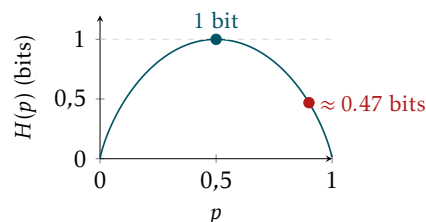


Figure 7.7: Binary entropy $H(p) = -p \log_2 p - (1 - p) \log_2 (1 - p)$ as a function of p . Maximised at $p = 0.5$ (1 bit), zero at $p \in \{0, 1\}$.

7.5.4. Cross-Entropy

Entropy tells us the minimum average code length when we know the true distribution and design the code accordingly. But what happens when we use a wrong distribution when designing our code?

Imagine trying to encode a Finnish text using Morse code designed for English. English Morse assigns short codes to frequent English letters like E and T, which are much less dominant in Finnish. Finnish has its own letter frequencies - for example Finnish love to use 'k', it forms about 4% of Finnish texts while in English that would be less than 1%. In English you can assign the latter 'k' a long-binary code - it is not used so often - but in Finnish it is used all the time, it should have a short code. Overall, the English-designed code will assign long codes to common Finnish letters and short

codes to rare ones, producing messages that are longer than necessary. A Morse code designed specifically for Finnish would be shorter. The **cross-entropy** measures this mess-up.

More precisely, cross-entropy measures the average code length when we use a model distribution $\hat{p}$ to design the code but events actually follow the true distribution p :

$$H(p, \hat{p}) = - \sum_k p_k \log \hat{p}_k. \quad (7.26)$$

Using the wrong distribution always costs extra bits: $H(p, \hat{p}) \geq H(p)$, with equality if and only if $p = \hat{p}$ - only a perfectly matched code achieves the minimum. The extra cost is exactly the KL divergence, which we define in the next subsection.

The connection to our classification loss follows naturally from this framing. In classification, the “symbols” are the class labels - each example belongs to one of K classes. The true distribution p over classes is the one-hot label vector: the correct class has probability 1 and all others have probability 0. If we knew this true distribution, we would assign a code of length $-\log_2 p_k$ to each class - the correct class would get a code of length 0 (it is certain) and all others would get infinite length codes (they never occur). In practice we do not know the true distribution for a new example; instead our neural network learns an estimate $\hat{p} = \text{softmax}(f_\theta(\mathbf{x}))$ of how probable each class is.

Using this estimated distribution to design our code, the cross-entropy measures the average code length we achieve:

$$H(p, \hat{p}) = - \sum_k y_k \log \hat{p}_k = -\log \hat{p}_y = \ell_{CE}(\hat{p}, \mathbf{y}). \quad (7.27)$$

When the model assigns high probability to the correct class, $\hat{p}_y \approx 1$ and the code length $-\log \hat{p}_y \approx 0$ - the class is easy to describe, the message is short. When the model assigns low probability to the correct class, $\hat{p}_y \approx 0$ and the code length $-\log \hat{p}_y \rightarrow \infty$ - the model is surprised by the correct answer, the message is long. Training minimises this cross-entropy: we are learning a probability distribution over classes that allows us to describe the true class labels as efficiently as possible. The better the model’s estimated distribution matches the true one, the shorter our messages - and the lower our loss.

7.5.5. KL Divergence

Cross-entropy measures the total average code length when using the wrong distribution. But part of that length is unavoidable - even with the perfect code, a message of n symbols drawn from p requires at least $H(p)$ bits per symbol. The truly interesting quantity is the **extra** cost we pay for using the wrong distribution: how many unnecessary bits does our Finnish text accumulate because we used English Morse code instead of Finnish Morse code? This extra cost is the **Kullback-Leibler divergence** (Kullback and Leibler 1951):

$$D_{KL}(p \parallel \hat{p}) = H(p, \hat{p}) - H(p) = \sum_k p_k \log \frac{p_k}{\hat{p}_k}. \quad (7.28)$$

The KL divergence is the gap between the code length we achieve with $\hat{p}$ and the minimum code length achievable with the true distribution p . It is zero if and only if $p = \hat{p}$ - our estimated distribution is perfect and we pay no unnecessary bits. It is always

non-negative - we can never do better than the true distribution. It is not symmetric: the extra cost of encoding p with $\hat{p}$ is generally different from encoding $\hat{p}$ with p , so it is not a distance in the mathematical sense.

Returning to classification: the true label distribution p is fixed - it is determined by the data, not by our model. The entropy $H(p)$ is therefore a constant that does not depend on the parameters θ . The cross-entropy $H(p, \hat{p})$ differs from $D_{KL}(p||\hat{p})$ only by this constant:

$$\arg \min_{\theta} H(p, \hat{p}) \equiv \arg \min_{\theta} D_{KL}(p||\hat{p}). \quad (7.29)$$

Minimising the cross-entropy loss is therefore equivalent to minimising the KL divergence between the true label distribution and the model's predicted distribution. Training is not just curve fitting - it is **distribution matching**: we are finding the model parameters that bring the predicted distribution as close as possible to the true one, as measured by the extra bits we would waste using the model's code instead of the ideal one.

This equivalence holds for any distributional assumption. For Gaussian regression, minimising MSE is equivalent to minimising the KL divergence between two Gaussians. For binary classification, minimising binary cross-entropy is equivalent to minimising the KL divergence between two Bernoullis. The loss function is always the NLL, and minimising it is always distribution matching - the specific distributions involved depend on what we assume about the data-generating process.

Beyond the coding interpretation, KL divergence is used throughout machine learning and statistics as a general measure of the difference between two probability distributions - how much information is lost when $\hat{p}$ is used to approximate p . It appears in variational inference, generative models, reinforcement learning, and many other settings where we need to quantify how far apart two distributions are. The asymmetry is practically meaningful: $D_{KL}(p||\hat{p})$ penalises placing low probability where p places high probability - it is large when the model misses probability mass that the true distribution assigns. $D_{KL}(\hat{p}||p)$ has the opposite character - it penalises placing high probability where p places low probability. Generally, if you wish to go deep in deep learning, it is good to know what KL divergence is.

7.5.6. Three Views, One Objective

Table 2 summarises the equivalence across all three cases. The three perspectives - loss minimisation, maximum likelihood, and KL divergence minimisation - are the same optimisation problem viewed through different lenses. MLE gives the statistical motivation, the loss function gives the practical objective, and KL divergence gives the geometric interpretation. This triple perspective is not only useful but also quite beautiful - when things work well in mathematics and science, the same truth tends to reveal itself from multiple directions at once, and each new angle deepens the understanding. Here, a statistician fitting a model, an engineer minimising a loss, and an information theorist compressing a message are all doing the same thing without knowing it.

7.6. Practical Implementation

The theory developed in the previous sections now needs to be translated into working code. PyTorch provides all the necessary building blocks, but several practical pitfalls

Perspective	Gaussian	Bernoulli	Categorical
Loss	MSE	binary CE	categorical CE
MLE	maximise $\log \mathcal{N}$	maximise $\log \text{Ber}$	maximise $\log \text{Cat}$
Info theory	$\min D_{KL}$	$\min D_{KL}$	$\min D_{KL}$

Table 2: Three perspectives on the same optimisation problem for three distributional assumptions.

are worth knowing about before writing the first classification training loop.

7.6.1. PyTorch Loss Functions

PyTorch provides two versions of each classification loss: one that takes probabilities as input and one that takes logits. Always prefer the logits version - these are numerically stable implementation of the sigmoid $\rightarrow$ binary-cross-entropy and softmax $\rightarrow$ cross-entropy operations:

- **Binary classification:** `nn.BCEWithLogitsLoss` (logits) or `nn.BCELoss` (probabilities)
- **Multiclass classification:** `nn.CrossEntropyLoss` (logits) or `nn.NLLLoss` (log-probabilities)

`nn.CrossEntropyLoss` expects either integer class indices of shape $(N,)$ for standard usage, or float probability distributions of shape (N, K) for soft targets (e.g. label smoothing). One-hot integer vectors are not supported.

Numerical Stability Intermezzo Computing the cross-entropy loss requires $\log \hat{p}_k = \log \text{softmax}(z)_k$. Naively:

$$\log \hat{p}_k = \log \frac{e^{z_k}}{\sum_j e^{z_j}} = z_k - \log \sum_j e^{z_j}. \quad (7.30)$$

This is numerically unstable: large logits cause e^{z_k} to overflow to `inf`, and very negative logits cause $\log \sum_j e^{z_j}$ to underflow to `-inf`.

The **log-sum-exp trick** resolves this by subtracting the maximum logit before exponentiating:

$$\log \sum_j e^{z_j} = c + \log \sum_j e^{z_j - c}, \quad c = \max_j z_j. \quad (7.31)$$

After subtracting c , the largest term is $e^0 = 1$ (no overflow) and all other terms are ≤ 1 (the sum is always ≥ 1 , so no underflow in the log). The subtraction is mathematically exact: c appears in both numerator and denominator and cancels. `nn.CrossEntropyLoss` and `nn.BCEWithLogitsLoss` apply this trick internally.

7.6.2. Predictions and Accuracy

At inference time, converting logits to a predicted class does not require computing probabilities explicitly. Since softmax is monotone, the class with the highest logit always has the highest probability:

$$\hat{y} = \arg \max_k z_k. \quad (7.32)$$

For binary classification, the decision boundary is at $z = 0$: predict class 1 if $z > 0$, class 0 otherwise. No sigmoid is needed.

In **multi-label classification**, each example can belong to multiple classes simultaneously (e.g. an image may contain both a cat and a dog). This is distinct from multiclass classification. Each output node is treated independently with its own sigmoid and binary cross-entropy: `nn.BCEWithLogitsLoss` with multi-dimensional targets of shape (N, K) . Standard accuracy is replaced by per-class metrics.

7.7. Advanced Considerations

The previous section covered the mechanics of implementation. This section steps back and examines deeper issues that arise when deploying classification models in practice. These are not edge cases - they are systematic problems that affect almost every real classification task. Understanding them is the difference between a model that works on a clean benchmark and one that is trustworthy in deployment.

7.7.1. Overconfidence and Calibration

Cross-entropy with hard one-hot labels always rewards higher confidence: the loss $-\log \hat{p}_y$ is minimised as $\hat{p}_y \rightarrow 1$, so the network is perpetually rewarded for being more confident and never penalised for overconfidence. The result is that trained classifiers tend to assign high probability to their predictions even when uncertain. Shown an out-of-distribution input, a network may assign 99% confidence to a wrong class. From an information theory perspective, the predicted distribution has too low entropy - the model is too certain.

A **well-calibrated** model is one whose predicted probabilities reflect true empirical frequencies: if it says 80% confidence, it should be correct 80% of the time. Most neural networks are not well-calibrated out of the box - they are overconfident. The **reliability diagram** visualises calibration by grouping predictions into bins by confidence level and plotting the actual accuracy within each bin against the predicted confidence (Figure 7.8). A perfectly calibrated model lies on the diagonal; an overconfident model sits below it.

Temperature scaling (Guo et al. 2017) is a simple post-hoc calibration method: divide the logits by a scalar temperature $\tau > 1$ before softmax:

$$\hat{\mathbf{p}} = \text{softmax}\left(\frac{\mathbf{z}}{\tau}\right). \quad (7.33)$$

$\tau > 1$ softens the distribution, reducing overconfidence. τ is a single scalar fitted on the validation set after training - the model weights are not modified. Despite its simplicity, temperature scaling works surprisingly well in practice.

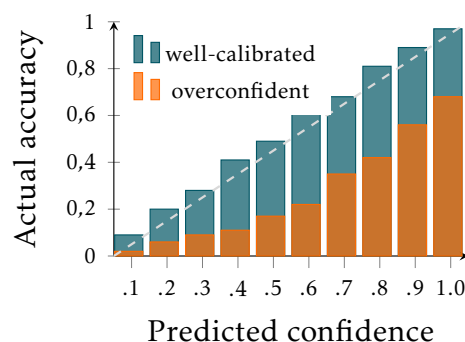


Figure 7.8: Reliability diagram. A well-calibrated model (petrol) lies on the diagonal. A typical overconfident network (orange) has actual accuracy below predicted confidence.

► Session 8:
Overconfidence,
Label Smoothing,
Calibration, Class
Imbalance, Weak
Labels

► Session 8:
Overconfidence,
Label Smoothing,
Calibration, Class
Imbalance, Weak
Labels

7.7.2. Label Smoothing

Label smoothing (Szegedy et al. 2016) directly addresses overconfidence by replacing hard one-hot targets with soft targets:

$$y_k^{\text{smooth}} = \begin{cases} 1 - \varepsilon & k = y \\ \varepsilon / (K - 1) & k \neq y \end{cases} \quad (7.34)$$

where ε is a fixed hyperparameter (typically 0.1). With soft targets the model can never assign probability 1 to the correct class - the loss would be infinite from the small positive targets on wrong classes - so the softmax cannot saturate. This produces predictions with higher entropy and more calibrated uncertainty.

Label smoothing is now standard in modern architectures: Inception v3 (Szegedy et al. 2016), Vision Transformers, and modern ResNets all use it, consistently improving both accuracy and calibration with minimal cost. A secondary benefit is reducing the impact of individual noisy labels - a wrong label in the training set causes less damage when the targets are soft rather than hard one-hot vectors.

In PyTorch: `nn.CrossEntropyLoss(label_smoothing=0.1)`.

7.7.3. Class Imbalance

When the training set has far more examples of some classes than others, standard cross-entropy is dominated by frequent classes. The model learns that predicting the majority class for every input is cheap - it achieves low loss without learning anything useful about rare classes. This is particularly common in medical diagnosis, fraud detection, and any domain where the events of interest are rare.

Weighted loss assigns higher weight to rare classes:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N w_{y^{(i)}} \log \hat{p}_{y^{(i)}}^{(i)}, \quad (7.35)$$

where w_k is inversely proportional to the frequency of class k .

In PyTorch: `nn.CrossEntropyLoss(weight=class_weights)`.

Alternatively, **oversampling** duplicates rare class examples (with or without augmentation) and **undersampling** removes frequent class examples to balance the training set. When class imbalance is present, accuracy is a misleading evaluation metric - precision, recall, and F1 score are more informative, as they separately measure performance on positive and negative classes.

7.7.4. Weak Labels

Labels in real datasets are often imperfect. **Noisy labels** arise when annotators make mistakes - the loss is then computed against wrong targets, pushing the model in the wrong direction. **Partial labels** occur when only some examples are labelled; missing labels are often treated as negatives, which can be misleading if the missing label simply means the class was not checked rather than absent. **Ambiguous labels** arise when multiple valid answers exist and hard labels force an arbitrary choice - a medical image that two radiologists interpret differently, or a sentence that can be classified as either neutral or positive.

In all these cases, **the model optimises toward the label rather than the truth**. Mitigations include label cleaning (detecting and correcting noisy labels), label smoothing (which reduces the impact of individual wrong labels), and collecting multiple annotations per example and aggregating them to reduce noise. The fundamental issue is that the quality of the learned model is bounded by the quality of the labels - **label quality matters as much as model quality**.

8. Convolutional Neural Networks

Convolutional neural networks (CNNs) are a class of neural networks designed to process data that has a natural spatial or sequential structure. Two properties make them well suited to images: **local connectivity**, which restricts each neuron to a small spatial neighbourhood rather than the entire input, and **translation invariance**, which ensures that the same pattern is detected regardless of where it appears in the image. Both properties are achieved through the **convolution operation**, which slides a learned filter across the input and applies it identically at every position. A useful side effect of this design is **parameter sharing**: because the same filter weights are reused at every spatial location, the number of learnable parameters is independent of the image size – a decisive advantage over fully connected layers when inputs are large.

8.1. Why Not MLPs for Images?

Before motivating the convolutional layer, it is worth understanding precisely why a standard fully connected network is poorly suited to image data. The problem is not expressiveness — given enough neurons, an MLP can in principle approximate any function — but **computational cost** and **inductive bias**.

► Images in
Computers;
Images as Data;
How to Look at
Images

8.1.1. Images as Structured Tensors

A digital image is represented as a three-dimensional tensor of shape $C \times H \times W$, where C is the number of channels, H is the height in pixels, and W is the width in pixels. Each entry is a pixel intensity, either an integer in the range $[0, 255]$ or a normalised float in $[0, 1]$, stored in 8 bits per channel. The two formats relevant for most deep learning work are **grayscale** ($C = 1$), where pixel values run from black (0) to white (255), and **RGB** ($C = 3$), the standard format for camera images and screens, with separate red, green, and blue channels. Other formats exist – RGBA with an additional transparency channel, CMYK used in printing, and high-dimensional medical or hyperspectral images that may carry many channels at 16-bit depth – but these are typically converted to RGB or grayscale before being fed to a network.

Image sizes vary enormously across tasks. Benchmark datasets designed for classification at modest resolution include MNIST (28×28 , grayscale, 784 pixels per image) and ImageNet (224×224 , RGB, 150 528 pixels). Real-world sources produce far larger inputs: a medical CT slice is typically 512×512 (262 144 pixels), a full-HD screen captures 1920×1080 pixels (over 6 million), and a modern phone camera can produce 4000×3000 images with 36 million pixels per frame.

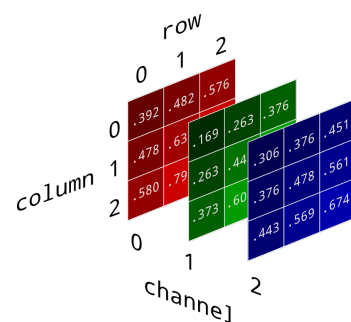


Figure 8.1: An RGB image decomposed into its three colour channels.

8.1.2. The Parameter Explosion

The standard MLP approach to images is to **flatten** the input tensor to a vector $\mathbf{x} \in \mathbb{R}^{H \cdot W \cdot C}$ and pass it through a sequence of fully connected layers. The weight matrix of the first hidden layer alone has dimensions $(H \cdot W \cdot C) \times d_1$, where d_1 is the number of hidden units. For a modest hidden layer of $d_1 = 1024$ units applied to an ImageNet image, this gives $150\,528 \times 1024 \approx 154$ million parameters – for a *single* layer. Training such a model would be prohibitively expensive in memory and computation, and the model would require an enormous amount of data just to avoid overfitting.

8.1.3. The Missing Inductive Bias

Parameter count alone does not fully explain the problem. A fully connected layer treats every input pixel as equally related to every other pixel. This is wrong for images: pixels that are spatially close carry far more information about each other than pixels at opposite ends of the image. More importantly, a fully connected layer must **relearn** the same feature detector from scratch at every spatial location. A filter that detects a horizontal edge in the top-left corner of an image is identical in function to one that detects a horizontal edge in the bottom-right, but an MLP represents them as entirely separate parameters.

Human vision operates differently. We look for **local patterns** – edges, corners, textures – and we recognise the same pattern regardless of where it appears in our visual field. Simple detected features combine into increasingly complex ones as we move from early to later visual processing stages. The convolutional layer translates these three properties directly into a neural network primitive: local connectivity restricts each neuron to a small spatial patch; parameter sharing uses the **same filter weights at every location**; and as a consequence, any feature a filter detects at one position is detected identically everywhere else, giving translation invariance as an emergent property.

8.2. The Convolution Operation

The convolution operation is the mathematical engine of a CNN. This section develops it in full generality, starting from the simple one-dimensional case and extending to the multi-channel two-dimensional setting used in practice.

8.2.1. 1D Convolution

Let $\mathbf{x} \in \mathbb{R}^n$ be an input signal and $\boldsymbol{\kappa} \in \mathbb{R}^k$ be a **filter** (also called a **kernel**) of length k . The 1D convolution of $\mathbf{x}$ with $\boldsymbol{\kappa}$ produces an output signal whose value at position i is the dot product of the filter with the local patch of $\mathbf{x}$ starting at i :

$$(\mathbf{x} * \boldsymbol{\kappa})[i] = \sum_{j=0}^{k-1} \mathbf{x}[i+j] \boldsymbol{\kappa}[j]. \quad (8.1)$$

Here i indexes the output position and j indexes the position within the filter. The filter **slides** across the signal from left to right: at each position i , it computes a single

► Convolution —
The Operation; 1D
Convolution; 2D
Convolution;
Padding; Stride;
Multiple Input
Channels;
Multiple Output
Channels;
Convolution as
Matrix
Multiplication

dot product, producing one value of the output. For a filter of length $k = 3$ with weights $(1, -1, 2)$ and the input patch $(1, 3, 4)$ at position $i = 1$, the output is $1 \cdot 1 + 3 \cdot (-1) + 4 \cdot 2 = 6$.

This operation embodies both desirable properties simultaneously. Local connectivity is enforced because each output value depends on only k contiguous input values, not the entire signal. Translation invariance follows from applying the same filter weights κ at every position i : if a pattern shifts by one position in the input, the same filter detects it one position later in the output. Parameter sharing is the mechanism that makes this possible – and as a consequence, the number of learnable parameters is k , independent of the signal length n .

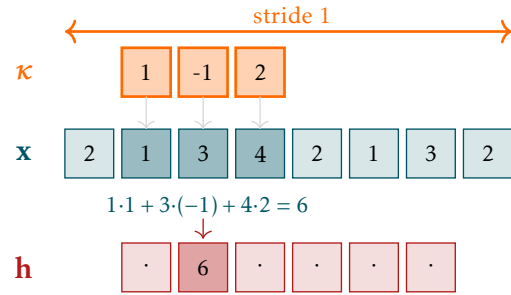


Figure 8.2: 1D convolution: a filter of length 3 slides over an input signal, computing a dot product at each position.

8.2.2. 2D Convolution

For images, the filter must slide in two spatial dimensions. Given a single-channel input $\mathbf{X} \in \mathbb{R}^{H \times W}$ and a filter $\kappa \in \mathbb{R}^{k \times k}$, the 2D convolution at output position (i, j) is:

$$(\mathbf{X} * \kappa)[i, j] = \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} \mathbf{X}[i + p, j + q] \kappa[p, q]. \quad (8.2)$$

The double sum ranges over all rows p and columns q within the filter. Output position (i, j) thus sees the $k \times k$ patch of $\mathbf{X}$ anchored at (i, j) . The filter is applied once per valid anchor position; the complete output is assembled by sliding the filter across the entire input.

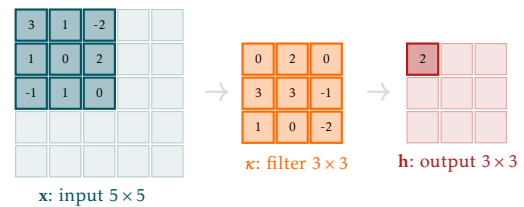


Figure 8.3: A single application of a 3×3 filter: the dot product with one local patch produces one output value.

Without any padding, the filter can be placed at $(H - k + 1) \times (W - k + 1)$ valid positions, so the output has shape $(H - k + 1) \times (W - k + 1)$. For a 5×5 input with a 3×3 filter, the output is 3×3 .

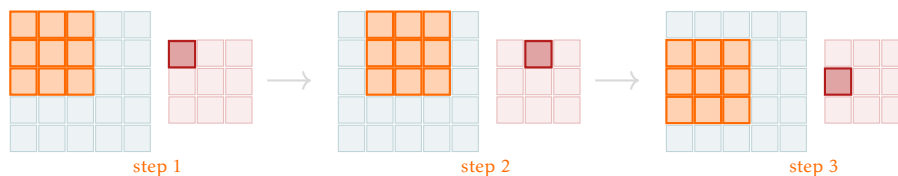


Figure 8.4: The full 2D convolution: the filter slides across all valid positions, producing one output value per position. Without padding, a $k \times k$ filter reduces an $H \times W$ input to $(H - k + 1) \times (W - k + 1)$.

8.2.3. Padding

A practical difficulty with the basic formula above is that the output is smaller than the input. With L convolutional layers and no padding, the spatial dimensions shrink by $k - 1$ at each layer, quickly consuming the available resolution. For a 10-layer network with 3×3 filters, a 224×224 input would shrink to 204×204 , which may be acceptable, but a 28×28 input would be exhausted long before the network is deep enough to learn useful features.

The standard solution is to add a border of zeros around the input before convolving. Two conventions are in common use. **Valid padding** ($p = 0$) applies no padding at all, so the output strictly shrinks; this is appropriate when spatial reduction at each layer is intentional. **Same padding** sets $p = (k - 1)/2$, which adds enough zeros to keep the output the same size as the input for odd filter sizes. Same padding is by far the most common choice in modern architectures: it decouples spatial resolution from filter size and makes it straightforward to design networks of arbitrary depth. Border pixels receive less coverage than central pixels, but in practice this edge effect is negligible.

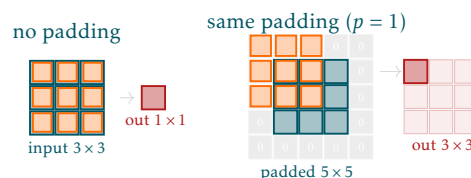


Figure 8.5: Same padding adds zeros around the input border so the output has the same spatial dimensions as the input.

8.2.4. Stride

Stride controls **how far the filter moves** between consecutive applications. With stride $s = 1$ (the default), the filter advances one pixel at a time. With stride $s = 2$, it skips every other position, halving the output dimensions. Strided convolution is an alternative to pooling for spatial downsampling: it reduces resolution while simultaneously applying a learned filter, rather than discarding information with a fixed max or average operation.

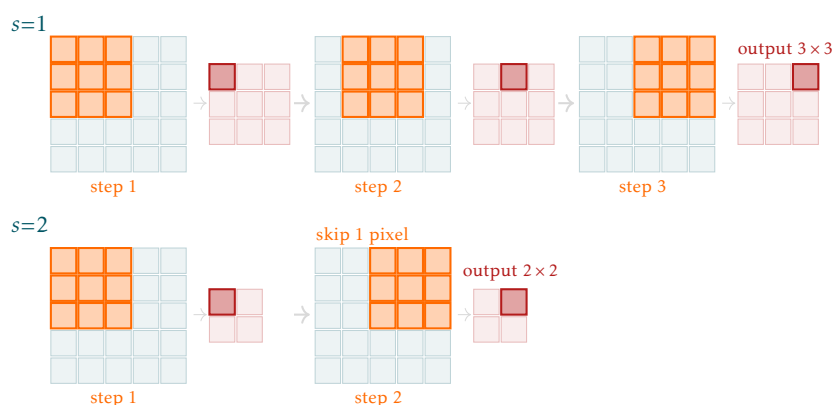


Figure 8.6: Stride 1 (top) and stride 2 (bottom) applied to the same input. Stride 2 halves the output spatial dimensions.

Pooling and strided convolution serve a similar downsampling purpose. In earlier architectures such as VGGNet, explicit max pooling layers handle all spatial reduction. More recent designs, including ResNet, replace some or all intermediate pooling with

strided convolution, on the grounds that a learned downsampling filter is more flexible than a fixed one. Global average pooling at the end of the network, discussed in Section 8.5, remains standard across essentially all modern architectures.

The general output size formula for an input of height H , filter size k , padding p , and stride s is:

$$H_{\text{out}} = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1, \quad (8.3)$$

and analogously for the width.

8.2.5. Multiple Input Channels

Real images have $C_{\text{in}} \geq 1$ channels. A single filter must respond to the full depth of its input, so it too must have C_{in} slices – one per input channel. The filter $\kappa \in \mathbb{R}^{C_{\text{in}} \times k \times k}$ is applied by convolving each slice independently with its corresponding input channel and summing the results:

$$(\mathbf{X} * \kappa)[i, j] = \sum_{c=0}^{C_{\text{in}}-1} \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} \mathbf{X}[c, i+p, j+q] \kappa[c, p, q]. \quad (8.4)$$

The inner double sum convolves filter slice c with input channel c ; the outer sum over c accumulates the results across all channels. Regardless of C_{in} , a single filter produces a single **feature map** – a $1 \times H' \times W'$ output. The feature map captures a single learned pattern detected across all input channels simultaneously.

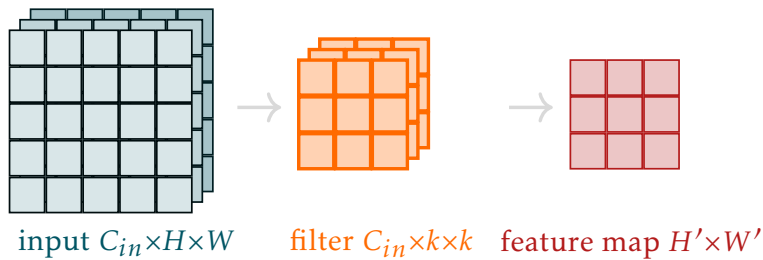


Figure 8.7: A filter with C_{in} slices applied to a multi-channel input. Each filter slice convolves with one channel; the results are summed to produce one feature map.

8.2.6. Multiple Output Channels

A single filter detects a single pattern. To detect C_{out} different patterns, we use C_{out} independent filters, each producing its own feature map. Filter f produces:

$$\mathbf{h}_f[i, j] = \sum_{c=0}^{C_{\text{in}}-1} \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} \mathbf{X}[c, i+p, j+q] \kappa_f[c, p, q], \quad f = 0, \dots, C_{\text{out}} - 1. \quad (8.5)$$

The C_{out} feature maps are stacked into an output tensor of shape $C_{\text{out}} \times H' \times W'$. The total number of learnable parameters in the layer, including one bias per filter, is $C_{\text{out}} \times C_{\text{in}} \times k \times k + C_{\text{out}}$. This is **independent of the spatial dimensions** H and W of the input – a critical contrast with a fully connected layer, where parameter count grows linearly with image size.

In typical CNN designs, the number of channels **increases** with depth while spatial dimensions decrease. Early layers use few filters to detect simple low-level patterns in the original image resolution; later layers use many filters to detect complex semantic patterns in reduced-resolution feature maps.

8.2.7. Convolution as Sparse, Tied Matrix Multiplication

It is instructive to observe that convolution is **a special case of matrix multiplication**. If we flatten both the input and the output to vectors, the convolution operation can be written as $\mathbf{h} = \mathbf{W}\mathbf{x}$ where $\mathbf{W}$ is a specific matrix structure. Two structural constraints distinguish it from a general weight matrix. First, **sparsity**: each row of $\mathbf{W}$ has only k^2 non-zero entries (the filter size), because each output neuron depends on only a small local patch of the input. Second, **weight tying**: the non-zero entries in every row are identical copies of the same filter weights, shifted by one position from row to row.

This perspective is theoretically important for two reasons. Since convolution is a linear transformation, the composition $\text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Conv} \rightarrow \text{ReLU} \rightarrow \dots$ has exactly the same representational structure as an MLP, but with far fewer parameters and an explicit inductive bias toward spatial locality. Furthermore, the sparse tied structure means that the gradient of the loss with respect to the filter weights is simply the **sum of gradients across all spatial positions** – which is exactly the operation applied in MLP backpropagation.

8.3. CNN Architecture

Individual convolutional layers are stacked into a complete network according to a design pattern that has remained largely stable since the early 2010s. This section describes how the layers fit together and what the resulting network learns.

8.3.1. Receptive Field

The **receptive field** of a neuron is the region of the input image that can influence that neuron's output. For a single convolutional layer with a 3×3 filter, each output neuron sees a 3×3 patch of the input. After a second 3×3 layer, each output neuron is influenced by a 5×5 patch of the *first* layer's input, because its 3×3 neighbourhood in

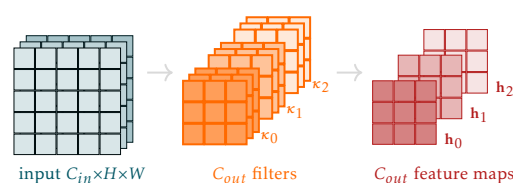


Figure 8.8: C_{out} filters each applied independently to the full input volume, producing C_{out} feature maps stacked into the output tensor.

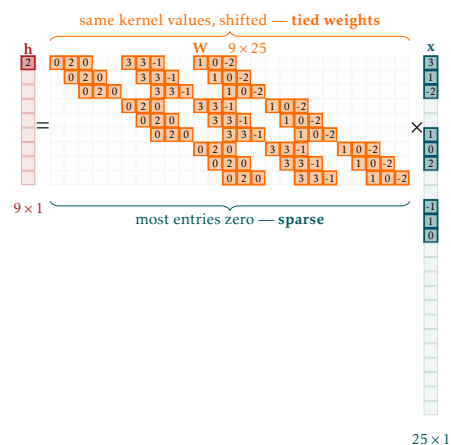


Figure 8.9: The weight matrix corresponding to a convolution. Non-zero entries (coloured) repeat the same filter values; most entries are zero (local connectivity).

► Receptive Field;
What Do CNNs
Learn?; Pooling;
CNN Building
Block; Full CNN
Architecture

the first feature map each covers a 3×3 patch of the original input. In general, L layers of 3×3 convolutions yield a receptive field of $(2L + 1) \times (2L + 1)$.

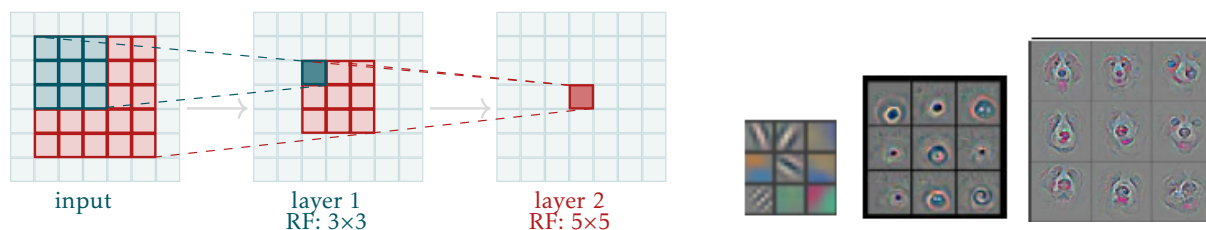


Figure 8.10: Left: receptive field growth with depth – each additional 3×3 layer extends the receptive field by 2 pixels in each dimension. Right: features visualised at successive layers of a CNN trained on ImageNet (Zeiler and Fergus 2014); early layers detect edges and colour gradients, later layers respond to textures, object parts, and faces.

This is the key to the **hierarchical feature** story. Early layers have small receptive fields and detect simple local patterns: oriented edges, colour gradients, and small textures. Later layers, with large receptive fields, can combine these local detections into representations of complex structures such as object parts or entire objects. Zeiler and Fergus (2014) made this hierarchy directly visible by developing a technique for projecting feature map activations back to pixel space. Their results confirmed that early layers respond to edges and colour blobs, intermediate layers to textures and repeated patterns, and deep layers to object parts and faces.

8.3.2. Pooling

Convolution produces feature maps that are spatially highly redundant: neighbouring output neurons have strongly overlapping receptive fields and therefore highly correlated values. **Pooling** is a fixed, non-trainable operation that reduces spatial redundancy by summarising each local region into a single value.

The most common variant is **max pooling**, which takes the maximum value within a rectangular region $\mathcal{R}_{ij}$:

$$y[i, j] = \max_{p, q \in \mathcal{R}_{ij}} x[p, q]. \quad (8.6)$$

Max pooling retains the strongest activation in each region, which corresponds intuitively to detecting whether a feature is present at all within that neighbourhood, regardless of its precise location. **Average pooling** instead computes the mean, which carries information about the average activation level. The pooling operation applies independently to each channel, leaving the channel dimension unchanged.

A 2×2 pool with stride 2 (the standard choice) halves both spatial dimensions: a $C \times H \times W$ feature map becomes $C \times H/2 \times W/2$. Each pooling layer therefore doubles the effective receptive field of all subsequent neurons. Pooling and strided convolution serve a similar purpose – downsampling – but strided convolution is generally preferred in modern architectures because the downsampling filter is learned rather than fixed.

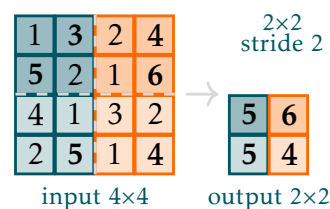


Figure 8.11: 2×2 max pooling reduces each 2×2 region to its maximum, halving both spatial dimensions.

8.3.3. The Standard CNN Building Block

Modern CNNs organise their convolutional layers into a repeating **building block**. The standard pattern is:

$$\text{Conv} \rightarrow \text{BatchNorm} \rightarrow \text{ReLU} \rightarrow (\text{Pool}),$$

where the pooling step is applied selectively rather than after every layer. Each component has a distinct role: the convolution extracts features via learned filters; batch normalisation stabilises the distribution of activations and is essential for training networks deeper than roughly ten layers; ReLU introduces the non-linearity without which a stack of convolutional layers collapses to a single linear map; and pooling reduces spatial resolution.

8.3.4. Full CNN Architecture: Backbone and Head

A complete CNN for image classification consists of two functionally distinct stages. The first stage – the **backbone** – is a sequence of convolutional blocks that transform the raw image into a rich feature representation. Across these blocks, the spatial dimensions decrease (via pooling or strided convolution) while the channel count increases, following a characteristic pattern such as $3 \rightarrow 32 \rightarrow 64 \rightarrow 128$ channels. The backbone acts as a **general-purpose feature extractor**: its filters learn to detect progressively more abstract and semantically meaningful patterns in the image.

The second stage – the **head** – maps the extracted feature representation to the desired output. For image classification, the head typically flattens (or globally average-pools) the final feature map and passes it through one or more fully connected layers, ending with a softmax over the class scores typically not implemented directly but implicitly subsumed to the loss function. The head is inherently **task-specific**: the same backbone can be paired with different heads for classification, detection, or segmentation. This backbone/head distinction becomes central in Section 9, when we discuss how to reuse pretrained backbones for new tasks.

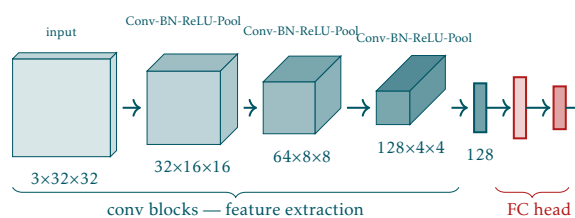


Figure 8.12: Full CNN architecture for image classification. The backbone (convolutional blocks) extracts features; the head (fully connected layers) maps to class scores.

8.4. Classic Architectures

The history of CNN architectures is not merely a catalogue of engineering milestones – it is a record of the key ideas that unlocked the potential of deep learning for vision. Each architecture discussed below contributed one or more advancements that are now standard practice.

8.4.1. LeNet-5

LeNet-5 (LeCun, Bottou, et al. 1998b) was the first CNN to demonstrate practical success on a real-world task. Trained on handwritten digits, it introduced the now-

► LeNet-5;
AlexNet; VGG;
ResNet; Residual
Block

canonical pattern of alternating convolutional layers and pooling layers followed by fully connected classification layers:

Conv $\rightarrow$ Pool $\rightarrow$ Conv $\rightarrow$ Pool $\rightarrow$ FC $\rightarrow$ FC,

with approximately 60 000 parameters in total. The network was small by modern standards but sufficiently powerful to be deployed commercially for cheque reading. Its fundamental limitation was a shortage of compute: deeper networks capable of handling more complex tasks could not be trained efficiently on the hardware of the 1990s.

8.4.2. AlexNet

The publication of **AlexNet** by Krizhevsky, Sutskever, and G. E. Hinton (2012b) in 2012 is generally identified as the beginning of the modern deep learning era for computer vision. AlexNet won the **ImageNet Large Scale Visual Recognition Challenge (ILSVRC 2012)** by a margin that stunned the computer vision community: it achieved a top-5 error rate of 15.3%, compared to 26.2% for the second-placed entry. The architecture – five convolutional layers followed by three fully connected layers, with approximately 60 million parameters – was not radically different in structure from LeNet, but it incorporated four innovations that collectively made deep CNNs tractable. **ReLU activations** replaced sigmoid and tanh nonlinearities, dramatically accelerating training by avoiding the vanishing gradient problem that afflicts saturating activations. **Dropout** was applied in the fully connected layers as a regulariser. **Data augmentation** – random crops and horizontal flips during training – reduced overfitting. And critically, the network was trained on two **NVIDIA GTX 580 GPUs**, demonstrating that GPU parallelism could make large-scale CNN training feasible.

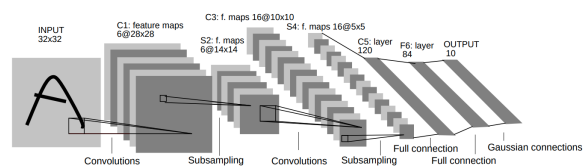


Figure 8.13: LeNet-5 architecture: two convolutional blocks followed by fully connected layers for digit classification.

8.4.3. VGGNet

VGGNet (Simonyan and Zisserman 2015) pursued a different question: what is the effect of network depth when all other design choices are held constant? The answer was a principled simplification: use only 3×3 filters throughout the entire network, and increase depth systematically. The flagship VGG-16 model uses 13 convolutional layers and 3 fully connected layers with approximately 138 million parameters.

The key insight was that two stacked 3×3 convolutions cover the same receptive field as one 5×5 convolution, but with strictly fewer parameters ($2 \times 9 = 18$ vs 25) and an additional non-linearity between them. This makes the network more expressive while remaining parameter-efficient. The main weakness of VGGNet is its three large

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv-3-64	conv-3-64	conv-3-64	conv-3-64	conv-3-64	conv-3-64
	LRN	conv-3-64	conv-3-64	conv-3-64	conv-3-64
conv-3-128	conv-3-128	conv-3-128	conv-3-128	conv-3-128	conv-3-128
		conv-3-128	conv-3-128	conv-3-128	conv-3-128
conv-3-256	conv-3-256	conv-3-256	conv-3-256	conv-3-256	conv-3-256
conv-3-256	conv-3-256	conv-3-256	conv-3-256	conv-3-256	conv-3-256
		conv-1-256	conv-1-256	conv-3-256	conv-3-256
conv-3-512	conv-3-512	conv-3-512	conv-3-512	conv-3-512	conv-3-512
conv-3-512	conv-3-512	conv-3-512	conv-3-512	conv-3-512	conv-3-512
		conv-1-512	conv-1-512	conv-3-512	conv-3-512
conv-3-512	conv-3-512	conv-3-512	conv-3-512	conv-3-512	conv-3-512
conv-3-512	conv-3-512	conv-3-512	conv-3-512	conv-3-512	conv-3-512
		conv-1-512	conv-1-512	conv-3-512	conv-3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Figure 8.14: VGG architecture: uniform 3×3 filters stacked to increasing depth.

fully connected layers, which together account for roughly 100 million of the 138 million parameters and impose a heavy memory burden. Despite this, VGGNet’s clean and uniform design made it an excellent baseline and its features were widely reused for transfer learning.

8.4.4. ResNet

By 2015, it was well established that deeper networks generally achieve better accuracy, yet practitioners had observed a frustrating phenomenon: beyond a certain depth, adding more layers actually **degraded** training accuracy. This was not overfitting but an optimization failure - the training loss itself was higher for deeper models. **ResNet** (He et al. 2016) diagnosed the problem and proposed a structural fix.

The key observation is that a deeper network should, in principle, be at least as good as its shallower counterpart: the additional layers could simply learn the identity mapping and pass their input through unchanged. Yet learning the identity is apparently difficult for a standard convolutional layer. The **residual connection** (also called a skip connection) reformulates the problem by adding a direct shortcut from the input of a block to its output:

$$\mathbf{y} = \text{ReLU}(\mathcal{F}(\mathbf{x}) + \mathbf{x}). \quad (8.7)$$

Here $\mathcal{F}(\mathbf{x})$ represents two (or three) convolutional layers. Instead of learning the full output mapping $\mathbf{y} = \mathcal{F}(\mathbf{x})$, the block learns the **residual** $\mathcal{F}(\mathbf{x}) = \mathbf{y} - \mathbf{x}$. When the optimal transformation is close to the identity, the block only needs to learn values close to zero rather than memorising a copy of its input – a much easier optimisation task. The shortcut also provides a **direct gradient path** from the loss back to early layers, substantially reducing vanishing gradient problems in very deep networks.

The basic residual block consists of two Conv-BN-ReLU sequences. For deeper variants (ResNet-50 and above), a **bottleneck block** replaces these two layers with three: a 1×1 convolution that reduces the channel count, a 3×3 convolution that processes the reduced representation, and a 1×1 convolution that restores the channel count. This achieves a similar receptive field to the basic block with significantly fewer parameters. When the shortcut must bridge layers with different channel counts or spatial dimensions, a **projection shortcut** is used: a 1×1 strided convolution applied to $\mathbf{x}$ to match the shape of $\mathcal{F}(\mathbf{x})$.

With residual connections, He et al. (2016) successfully trained networks with 50, 101, and 152 layers – depths that had been completely inaccessible before. ResNet remains one of the most widely used backbone architectures in computer vision.

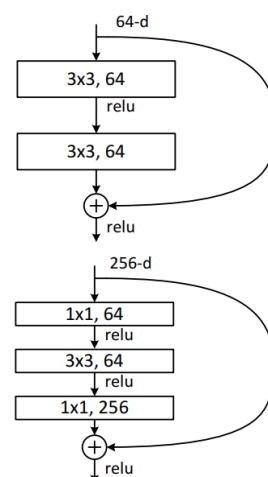


Figure 8.15: Residual block.

8.5. Modern Details

Beyond the high-level architecture choices discussed above, several lower-level design elements have become standard ingredients in modern CNNs. This section covers three of the most consequential: 1×1 convolutions, batch normalisation adapted to the convolutional setting, and global average pooling.

► 1×1
Convolutions;
Batch Norm in
CNNs; Global
Average Pooling

8.5.1. 1×1 Convolutions

A 1×1 convolution applies a learned linear combination across channels at each spatial location independently:

$$y_c[i, j] = \sum_{c'=0}^{C_{\text{in}}-1} w_{c,c'} x_{c'}[i, j] + b_c. \quad (8.8)$$

This is equivalent to applying a fully connected layer at every spatial position with shared weights. Despite its name, a 1×1 convolution has no spatial extent: it performs no spatial mixing, but it mixes information **across channels**. Its uses are widespread: it changes the channel count cheaply (expanding or compressing the feature volume), injects a non-linearity between channel combinations without spatial processing, and serves as the projection shortcut in residual connections when dimensions must change.

8.5.2. Batch Normalisation in CNNs

Batch normalisation was introduced in section 6 for fully connected networks. In the convolutional setting, the natural generalisation is **BatchNorm2d**, which normalises over the batch *and* spatial dimensions for each channel separately. For channel c , the mean and variance are computed as:

$$\mu_c = \frac{1}{N \cdot H \cdot W} \sum_{n,i,j} x_c^{(n)}[i, j], \quad (8.9)$$

$$\sigma_c^2 = \frac{1}{N \cdot H \cdot W} \sum_{n,i,j} \left(x_c^{(n)}[i, j] - \mu_c \right)^2, \quad (8.10)$$

where N is the batch size. There is one mean and variance per channel – not per spatial location – reflecting the parameter-sharing philosophy of convolution: the statistics of a feature map should be consistent across spatial positions. Each channel also has learnable scale and shift parameters γ_c and β_c .

The standard placement in a CNN block is Conv $\rightarrow$ BatchNorm2d $\rightarrow$ ReLU. BatchNorm2d is practically essential for training deep CNNs: without it, networks deeper than roughly ten layers become extremely difficult to optimise. It allows higher learning rates, reduces sensitivity to weight initialisation, and acts as a mild regulariser.

8.5.3. Global Average Pooling

Flattening the final convolutional feature map before passing it to fully connected layers can be expensive. If the last feature map has shape $C \times H' \times W'$, the flattened vector has $C \cdot H' \cdot W'$ entries, and a fully connected layer from that vector to d hidden units requires $C \cdot H' \cdot W' \cdot d$ parameters.

Global average pooling (GAP) eliminates this cost by reducing each feature map to a single scalar:

$$y_c = \frac{1}{H \cdot W} \sum_{i=1}^H \sum_{j=1}^W x_c[i, j], \quad c = 1, \dots, C. \quad (8.11)$$

The result is a vector of length C – one number per channel – which is then passed directly to the classification layer without any intermediate fully connected layers. The intuition is that each feature map represents one learned concept; its average activation across the image measures how strongly that concept is present. This representation is compact, spatial-dimension-agnostic (the same GAP layer works for any input resolution), and dramatically reduces parameter count compared to flattening. In PyTorch, GAP is implemented as `nn.AdaptiveAvgPool2d((1, 1))` followed by `flatten`.

8.6. Summary

Convolutional neural networks impose two structural biases on the processing of spatial data: local connectivity, which restricts each neuron to a small spatial neighbourhood, and translation invariance, which ensures that a learned feature detector responds identically regardless of where a pattern appears. Both are achieved through the convolution operation, which applies the same filter weights at every spatial location – a design that also keeps parameter count independent of image size.

Together, these allow CNNs to extract meaningful features from images using orders of magnitude fewer parameters than a comparably capable fully connected network. The building blocks – convolution, batch normalisation, ReLU, and pooling – are assembled into a backbone that extracts a hierarchy of increasingly abstract features, followed by a task-specific head.

The development from LeNet (1998) to AlexNet (2012), VGGNet (2014), and ResNet (2016) charts the empirical discovery of the principles that make deep CNNs work in practice: GPU training, ReLU activations, systematic use of depth, and – most consequentially – residual connections that allow gradient information to flow through networks of arbitrary depth. Session 9 builds directly on the backbone/head distinction introduced here, showing how pretrained backbones can be reused for new tasks without training from scratch.

9. CNNs in Practice

Session 9 introduced the theory and architecture of convolutional neural networks. This session focuses on how CNNs are actually used in practice. The central message is straightforward: you will almost never train a CNN from scratch. Instead, you will download a pretrained model and adapt it to your task. We examine when and how to do this, cover the practical PyTorch tools involved, and then extend beyond image classification to the three major visual tasks that CNNs also power: object detection, semantic segmentation, and instance segmentation.

► Session 10

9.1. Transfer Learning

Training a CNN from scratch on every new problem would be enormously wasteful — the low-level visual features learned by early layers are essentially the same regardless of the task. Transfer learning exploits this by reusing weights learned on one task as the starting point for another. This section covers when and how to do this effectively.

► Session 10: Three Regimes, Why Pretrained Models Work, Feature Extraction vs Fine-tuning, When to Use Which, Practical Tuning

9.1.1. Three Regimes

There are three distinct regimes for working with CNNs, each appropriate in different circumstances.

Training from scratch means initialising all weights randomly and optimising the full network on your own dataset. This requires a large labelled dataset (tens of thousands to millions of examples), significant compute (days to weeks on multiple GPUs), and is only justified when you have a genuinely novel architecture, a domain so different that pretrained weights would be useless, or when you are a large organisation with the necessary resources. For most practitioners, training from scratch is not a realistic option.

Pretraining followed by fine-tuning means deliberately designing a two-phase training pipeline. In the first phase, you train on a large **surrogate dataset** - one that is not your final target but is large enough to learn rich visual representations. In the second phase, you fine-tune on your small target dataset. This approach is valuable when no suitable pretrained model exists for your domain, but a related large dataset does. For example, a medical imaging researcher might pretrain on a large general radiology dataset and then fine-tune on a small collection of annotated cases for a specific rare condition. The surrogate dataset does not need to be perfectly aligned with the target task - it just needs to be large enough to learn useful features that transfer.

Downloading and adapting is the dominant modern workflow. Pretrained models are freely available from torchvision, timm, and HuggingFace, trained by large organisations on massive datasets. You download a model, replace its classification head with one appropriate for your task, and either use it as a fixed feature extractor or fine-tune some layers. This approach works surprisingly well even for tasks quite different from the original pretraining, requires relatively little data and compute, and is how the vast majority of real CNN applications are built today.

9.1.2. Why Pretrained Models Work

The reason transfer learning works so well is the hierarchical nature of what CNNs learn. As demonstrated by Zeiler and Fergus 2014, different layers of a CNN learn features at different levels of specificity:

- **Early layers** learn general low-level features: edges, colour gradients, oriented bars. These features are essentially universal — they appear in medical images, satellite imagery, microscopy, and natural photographs alike. A filter that detects a horizontal edge in an ImageNet-trained network is equally useful for detecting edges in a chest X-ray.
- **Later layers** learn increasingly task-specific features: object parts, textures associated with specific categories, combinations of lower-level features that are meaningful for the source task. These features are less transferable — a network trained to classify ImageNet categories will have late-layer features tuned for dogs, cars, and furniture, which may not be useful for classifying skin lesions.

This hierarchical structure means that pretrained weights are almost always a better starting point than random initialisation, even when the target domain differs substantially from the source. The question is not whether to use pretrained weights, but how much of the network to adapt.

Modern pretrained models are trained on increasingly large datasets: ImageNet (1.2M images, 1000 classes), LAION-5B (5.8 billion image-text pairs), and various proprietary

datasets used by large companies. Models trained on larger, more diverse datasets tend to transfer better across a wider range of target tasks.

9.1.3. Feature Extraction vs Fine-tuning

Recall that a CNN trained for classification consists of two functional parts. The **convolutional backbone** - the stack of convolutional and pooling layers - acts as a feature extractor: it maps an input image to a compact feature vector encoding what is present in the image. The **classification head** is typically a single fully connected layer (or a small MLP) that maps this feature vector to class scores. When we adapt a pretrained model, we discard the original head (which outputs scores for the source task's classes) and replace it with a new one sized for our target task.

Once you have a pretrained model, there are two main strategies for adapting it to a new task.

Feature extraction treats the pretrained network as a fixed feature extractor. All pretrained layers are frozen — their weights are not updated during training — and only a new classification head is trained. The pretrained network maps each input image to a feature vector, and the head learns to classify these vectors for the new task. Feature extraction is appropriate when the target dataset is small (hundreds to a few thousand examples), when compute is limited, or when the target domain is similar enough to the source that the pretrained features are directly useful.

Fine-tuning unfreezes some or all of the pretrained layers and trains them alongside the new head, typically with a much smaller learning rate. This allows the network to adapt its pretrained features to the specific characteristics of the target domain, at the cost of needing more data and compute. Fine-tuning is appropriate when the target dataset is large enough to avoid overfitting the deeper layers, or when the target domain is sufficiently different from the source that direct feature transfer is insufficient.

Catastrophic forgetting is a key risk in fine-tuning: if the learning rate is too large, the large noisy gradients from the randomly initialised head propagate back into the backbone and overwrite the pretrained weights, destroying the useful representations built up during pretraining. The standard mitigation is to use a learning rate 10×–100× smaller than the original pretraining learning rate when fine-tuning pretrained layers.

9.1.4. When to Use Which

The choice between feature extraction and fine-tuning depends on two factors: the size of the target dataset and the similarity between the source and target domains.

	Similar domain	Different domain
Small dataset	Feature extraction (freeze all, train head)	Feature extraction (try different layer depths)
Large dataset	Fine-tune all layers (small learning rate)	Fine-tune all layers (more epochs, careful tuning)

Feature extraction is the right choice when the target dataset is small. With few examples, fine-tuning the backbone risks overfitting - the network will memorise the training set rather than learning to generalise. If the target domain is similar to the source, the pretrained features transfer directly and a new head trained on top is often

sufficient. If the domain is different, it is worth experimenting with which layers to use as features: earlier layers capture more general low-level structure and may transfer better than later layers which are tuned to source-task semantics.

Fine-tuning becomes appropriate when enough labelled data is available to adapt the backbone without overfitting. With a similar domain, fine-tuning all layers with a small learning rate typically yields the best results. With a different domain, more epochs, a lower learning rate, and more aggressive data augmentation may be needed to fully adapt the representations.

9.1.5. Practical Tuning

The starting point for any transfer learning workflow is always to freeze the entire backbone and train only the new head. Once the head has stabilised - typically after a few epochs - you can begin unfreezing layers progressively from the top, since later layers are the most task-specific and most in need of adaptation. Earlier layers, which encode general low-level features, should remain frozen longer or not be touched at all.

When unfreezing layers, use different learning rates for different parts of the network. The new head, which starts from random initialisation, can tolerate a standard learning rate such as $1e-3$. Fine-tuned backbone layers should use a much smaller rate - $1e-5$ or lower - to avoid overwriting the pretrained representations. This technique, known as **discriminative learning rates**, is straightforward to implement in PyTorch by passing parameter groups with per-group learning rates to the optimiser.

It also helps to warm up the learning rate for the backbone layers at the start of fine-tuning: begin with a very small value and increase it gradually over the first few epochs. This gives the randomly initialised head time to stabilise and produce reasonable gradients before the backbone weights are significantly updated.

Early stopping is essential when fine-tuning - the combination of a small dataset and a powerful pretrained model means overfitting can happen within just a few epochs. Monitor validation loss carefully and save the checkpoint at the best validation performance rather than the final one.

Finally, data augmentation deserves particular attention when the target dataset is small. Standard augmentations - random crops, horizontal flips, colour jitter - are always worth applying. For domains far from natural images, domain-specific augmentations can be added on top and often provide a significant boost.

9.2. Practical CNN Training

This section covers the practical PyTorch tools for putting transfer learning into practice: where to find pretrained models, how to adapt them in code, and how to preprocess images correctly.

► Session 10:
torchvision and
timm, Adapting
Pretrained Model,
Image Transforms
Pipeline

9.2.1. Sources of Pretrained Models

Before writing any code, two decisions need to be made: which model architecture to use, and which pretrained weights to load. These choices interact and depend on your task, your dataset, and your available compute.

Choosing an architecture. The right architecture depends on several factors. Accuracy is the obvious one, but it should not be the only consideration. Larger models -

more layers, more parameters - generally achieve higher accuracy on standard benchmarks, but they are slower to train and run, consume more memory, and have a larger carbon footprint. For transfer learning on a small dataset, a smaller model is often the better choice: it is faster to fine-tune, less likely to overfit, and easier to deploy. A good starting point is to identify the accuracy-efficiency tradeoff that fits your constraints, then pick an architecture in that range. Resources such as the [Papers with Code](#) maintain up-to-date leaderboards comparing architectures on standard benchmarks, with links to the original papers and open-source implementations. This makes it straightforward to survey the accuracy-efficiency landscape, identify architectures worth considering, and find the corresponding code and pretrained weights.

Choosing weights. Once an architecture is chosen, the weights determine what features the backbone has learned. The common assumption - that weights performing well on the original task will transfer well to a new one - does not always hold. Transfer performance depends on how similar the source and target domains are, how the model was trained (the training recipe matters as much as the dataset), and sometimes on factors that are hard to predict in advance. Weights trained with modern recipes using mixup, label smoothing, or self-supervised objectives sometimes transfer better than older ImageNet checkpoints even for the same architecture. The safest approach is to treat weight selection as a hyperparameter: if compute allows, benchmark a few candidate weights on a small validation set rather than assuming the highest ImageNet accuracy corresponds to the best transfer performance.

Sources of pretrained models. Three main sources are available for PyTorch users.

`torchvision.models` is the official PyTorch model zoo, maintained by the PyTorch team. It covers a curated set of standard architectures - ResNet, VGG, EfficientNet, Vision Transformers - pretrained on ImageNet with well-documented training configurations. Because it is officially maintained, it is the most reliable source in terms of correctness and long-term stability. The API is simple and consistent, and each model entry documents exactly which weights are available and what normalisation statistics they require. The main limitation is breadth: torchvision covers the established classics but does not track the research frontier closely.

```
from torchvision.models import resnet50, ResNet50_Weights
model = resnet50(weights=ResNet50_Weights.DEFAULT)
```

`timm` (PyTorch Image Models) is a community-maintained library with over 700 pretrained models, including many architectures and weight variants not available in torchvision. It is the standard resource in vision research when you need something beyond the classics. A key advantage is that timm standardises the interface across architectures, so switching between models requires minimal code changes. The trade-off is that being community-maintained means quality and documentation can vary across models. In practice, the most popular models in timm are well-tested and widely used. Weights in timm often come from multiple sources and training recipes, which makes the choice richer but also more demanding - the [documentation](#) and model cards should be consulted carefully.

```
import timm
model = timm.create_model('resnet50', pretrained=True)
```

HuggingFace is a large model hub hosting contributions from both the research community and industry. For pure CNN-based vision tasks it largely overlaps with `timm`, but it becomes essential when working with more recent model families that combine vision with language - such as CLIP or DINO - based on architectures called Vision Transformers which go beyond what our introductory course covers. For now it is worth knowing that HuggingFace is where you would find them, and that the [transformers library](#) provides a consistent interface across a very wide range of model types. The quality and reliability of models on HuggingFace varies significantly - always check the model card and prefer models from established organisations or with a strong download and community track record.

For straightforward CNN transfer learning tasks, `torchvision` is the natural first stop for its reliability and simplicity. `timm` is the next step when you need more architectural choices or more recent weights. HuggingFace becomes relevant when your task requires vision-language models or when you are looking for community-contributed specialised weights.

9.2.2. Adapting a Pretrained Model

Once a pretrained model is loaded, the standard adaptation workflow has two steps: replace the classification head with one sized for your target task, and decide which parameters to train. In PyTorch this is done by manipulating `requires_grad` flags on parameter groups and constructing the optimiser accordingly. The following example shows feature extraction with a ResNet-50 backbone.

```
model = resnet50(weights=ResNet50_Weights.DEFAULT)

# freeze all pretrained layers
for param in model.parameters():
    param.requires_grad = False

# replace FC head with new classifier
num_features = model.fc.in_features
model.fc = nn.Linear(num_features, num_classes)

# train only the new head
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
```

In ResNet, the classification head is a single fully connected layer accessible as `model.fc` - it maps the 2048-dimensional output of the backbone to class scores. Replacing it with a new `nn.Linear` sized for your number of classes, while freezing all other parameters, gives a feature extraction setup where only the new head is trained.

For fine-tuning, selected backbone layers can be unfrozen and trained alongside the head, typically with a much smaller learning rate. The following example unfreezes the last ResNet block (`layer4`) and uses discriminative learning rates - a higher rate for the new head and a lower one for the unfrozen backbone layers.

```
# unfreeze last ResNet block
for param in model.layer4.parameters():
    param.requires_grad = True
```

```
# different learning rates per layer group
optimizer = torch.optim.Adam([
    {'params': model.layer4.parameters(), 'lr': 1e-5},
    {'params': model.fc.parameters(), 'lr': 1e-3},
])
```

All the basics from earlier sessions still apply: `model.train()` and `model.eval()` for mode switching, `optimizer.zero_grad()` before each backward pass, and the standard training loop.

9.2.3. Image Transforms Pipeline

Before images can be fed to a CNN, they must be converted into the format the network expects. This involves a pipeline of transformations applied to each image, constructed using `torchvision.transforms.Compose`.

Three transformations are always required. `ToTensor` converts a PIL image or NumPy array to a PyTorch tensor and scales pixel values from $[0, 255]$ to $[0, 1]$. `Resize` brings the image to the spatial dimensions the network expects - most ImageNet-pretrained models require 224×224 input, and feeding a different size will either cause an error or silently produce wrong results. `Normalize` then standardises each channel by subtracting the mean and dividing by the standard deviation.

Normalisation is important because the pretrained model was trained on data pre-processed with specific statistics - using different values shifts the input distribution away from what the network expects, degrading performance. When using a pretrained model, the normalisation statistics must therefore match those used during pretraining, not be computed from your own dataset. Both `torchvision` and `timm` document the correct statistics for each pretrained model; for ImageNet-pretrained models the standard values are mean $[0.485, 0.456, 0.406]$ and standard deviation $[0.229, 0.224, 0.225]$ per channel. Models pretrained on specialised data - medical images, satellite imagery, microscopy - can have statistics that differ substantially from these values, and using ImageNet statistics with such models will hurt performance.

Data augmentation applies random perturbations to training images to artificially increase the effective size of the dataset and improve generalisation. `RandomResizedCrop` crops a random region of the image and resizes it to the target size, encouraging the network to be robust to scale and position. `RandomHorizontalFlip` mirrors the image with 50% probability, useful for tasks where left-right symmetry holds. `ColorJitter` randomly varies brightness, contrast, saturation, and hue, making the model robust to lighting conditions. These are among the most commonly used augmentations, but many more are available - including `RandomRotation`, `RandomGrayscale`, `GaussianBlur`, and `RandomErasing` - and the full list is documented in the [torchvision transforms documentation](#).

The right choice of augmentations is task-dependent and is one of the places in the deep learning pipeline where domain expertise and careful thinking about the data genuinely matter. Horizontal flipping is appropriate for natural images but not for tasks where orientation carries meaning - reading text, analysing ECGs, or classifying asymmetric skin lesions. Aggressive colour jitter helps when lighting conditions vary but can destroy diagnostically relevant colour information in histology or dermatology images. Geometric distortions such as random rotation are useful for aerial or satellite

imagery where objects can appear at any angle, but inappropriate for tasks where vertical orientation is meaningful. There is no universal recipe: the best augmentation strategy comes from understanding what variability is present in the real deployment environment and what properties of the image are irrelevant to the label. A useful heuristic is to ask for each augmentation - could this transform turn a correctly labelled image into a wrongly labelled one? If yes, it should not be used.

Regardless of which augmentations are chosen, they must only be applied during training. At validation and test time the transform pipeline should contain only deterministic operations - resizing, cropping, and normalisation. Random augmentations at validation time would make the loss noisy across epochs, undermining early stopping, and would make test results non-reproducible.

```
from torchvision import transforms

train_transform = transforms.Compose([
    transforms.RandomResizedCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

val_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(...)
])
```

The transform pipeline is applied on the fly during data loading - each image is transformed as it is fetched by the `DataLoader`, not pre-processed and stored on disk. This has an important consequence: every time an image is loaded, a fresh set of random transformations is applied. The same image will be cropped to a different region, flipped or not, with different brightness, each time it appears in training. Over many epochs the network effectively sees many slightly different versions of each image, which is precisely what gives augmentation its regularising effect - it makes it harder for the network to memorise individual training examples and forces it to learn features that are robust to the kinds of variation the augmentations introduce.

The order of transformations in the pipeline matters. Geometric transformations such as `RandomResizedCrop` and `RandomHorizontalFlip` should come first, while the image is still in PIL format. `ToTensor` should follow, converting the image to a tensor, after which pixel-level operations such as `Normalize` and `ColorJitter` are applied. Applying `Normalize` before `ToTensor` would fail outright since `Normalize` operates on tensors. More subtly, applying colour augmentations before resizing is slightly wasteful - you are jittering pixels that may be discarded by the crop. Following the order shown in the code example above is a safe and standard convention.

9.3. Beyond Classification

Image classification assigns one or more labels to an entire image. Many real-world applications require richer outputs: locating objects, labelling every pixel, or distinguishing individual instances of the same class. Figure 9.1 illustrates the four main visual tasks. This section provides a brief introduction to each - enough to understand what these tasks involve and how CNNs are applied to them. A full treatment, including the details of modern architectures and training procedures, is beyond the scope of these notes and is typically covered by a dedicated course on computer vision.

► Session 10:
Vision Tasks,
Object Detection,
Evaluation – IoU
and mAP,
Semantic
Segmentation,
Instance
Segmentation,
Annotation Cost

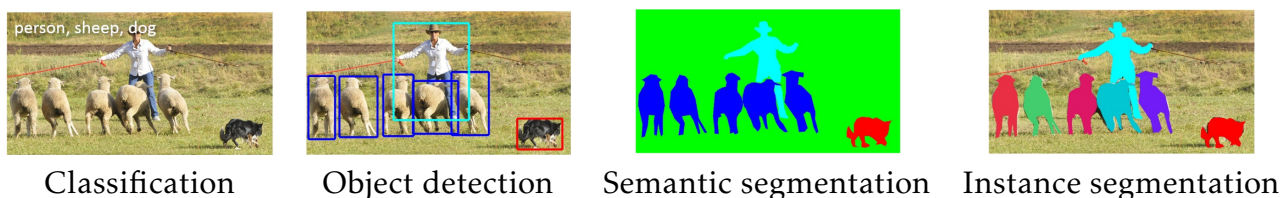


Figure 9.1: Four main visual tasks. Classification assigns one or more labels to the whole image. Object detection localises and classifies multiple objects with bounding boxes. Semantic segmentation assigns a class label to every pixel. Instance segmentation additionally distinguishes individual objects of the same class.

All four tasks share a common pattern: a pretrained CNN backbone extracts features, and a task-specific head produces the required output. This is why the transfer learning principles from Section 1 apply directly — the backbone can be pretrained on ImageNet and fine-tuned for any of these tasks.

9.3.1. Object Detection

Object detection requires the network to output a variable number of predictions, one per object present in the image. Each prediction consists of a bounding box - defined by four coordinates, typically the centre position, width, and height - a class label identifying what type of object it is, and a confidence score reflecting how certain the network is about that prediction. Unlike classification, the number of objects is not known in advance and can vary from image to image. A street scene might contain dozens of pedestrians, cars, and traffic signs simultaneously, each needing to be localised and identified independently.

Two-stage detectors. The R-CNN family of detectors (Ren et al. 2015) uses a two-stage approach. In the first stage, a **Region Proposal Network (RPN)** scans the feature map and proposes candidate regions that might contain objects, using anchor boxes of different scales and aspect ratios at each spatial location. In the second stage, each proposed region is extracted, pooled to a fixed size (RoI pooling), and passed through a classifier and bounding box regressor. Faster R-CNN introduced the RPN, making the whole pipeline end-to-end trainable and reducing inference time to around 0.2 seconds per image. Two-stage detectors generally achieve higher accuracy but are slower than single-stage approaches.

Single-stage detectors. The YOLO (Redmon et al. 2016) (You Only Look Once) family of models predict bounding boxes and class probabilities directly from the feature map

in a single forward pass. YOLO divides the image into an $S \times S$ grid; each cell predicts B bounding boxes with confidence scores and C class probabilities, producing an $S \times S \times (B \times 5 + C)$ output tensor. This single-pass design makes YOLO extremely fast and suitable for real-time applications. Modern YOLO versions (now at v10/v11) have largely closed the accuracy gap with two-stage detectors while maintaining high speed.

Evaluation. Object detection is evaluated using **Intersection over Union (IoU)** and **mean Average Precision (mAP)**.

IoU measures the overlap between a predicted bounding box and the ground truth box (Figure ??):

$$\text{IoU} = \frac{|\text{intersection}|}{|\text{union}|} \in [0, 1].$$

A prediction is counted as correct if its IoU with the ground truth exceeds a threshold, typically 0.5.

Average Precision (AP) for a single class summarises the precision-recall tradeoff across all possible confidence thresholds. As the threshold is lowered, more predictions are accepted: recall increases because more true objects are found, but precision falls because more false positives are included. Sweeping the threshold from high to low traces out the precision-recall curve. In practice, AP is not computed as the area under the smooth curve but as a stepped approximation: at each discrete recall level corresponding to an actual detection, the precision value is recorded, producing a staircase. AP is then the area under this staircase, equivalent to a weighted sum of precision values at each step. Mean AP (mAP) averages AP across all classes, giving a single number summarising detector performance across the full recognition vocabulary.

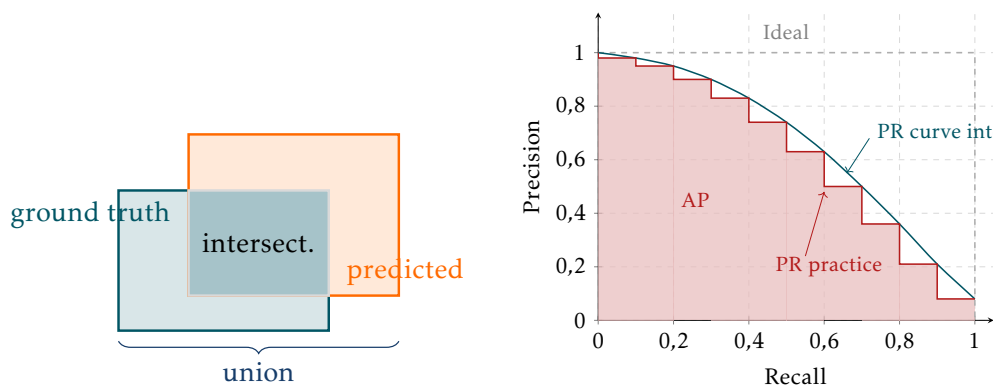


Figure 9.2: Left: IoU measures the overlap between predicted and ground truth bounding boxes. A prediction is counted as correct if IoU exceeds a threshold, typically 0.5. Right: Precision-recall curve for a single class. The ideal detector (dashed) maintains perfect precision at all recall levels. A real detector traces a smooth curve that falls as the confidence threshold is lowered. Average Precision (AP) is approximated as the area under the stepped interpolation; mean AP (mAP) averages this across all classes.

9.3.2. Semantic Segmentation

Semantic segmentation assigns a class label to every pixel in the image, producing an output label map of the same spatial size as the input. All pixels belonging to the

same class receive the same label, regardless of whether they belong to different object instances.

The dominant architecture for semantic segmentation is the **encoder-decoder**. The encoder is a CNN backbone that progressively reduces spatial resolution while extracting rich features. The decoder upsamples the low-resolution feature maps back to the input resolution, recovering spatial detail. The challenge is that the aggressive downsampling in the encoder discards fine spatial information that is needed to produce accurate pixel-level predictions.

U-Net (Ronneberger, Fischer, and Brox 2015), illustrated in Figure 9.3, addressed this by introducing **skip connections** between corresponding encoder and decoder layers. These connections concatenate high-resolution encoder features directly with the upsampled decoder features at each resolution level, allowing the decoder to recover spatial detail that would otherwise be lost. Originally developed for medical image segmentation, U-Net remains one of the most widely used segmentation architectures.

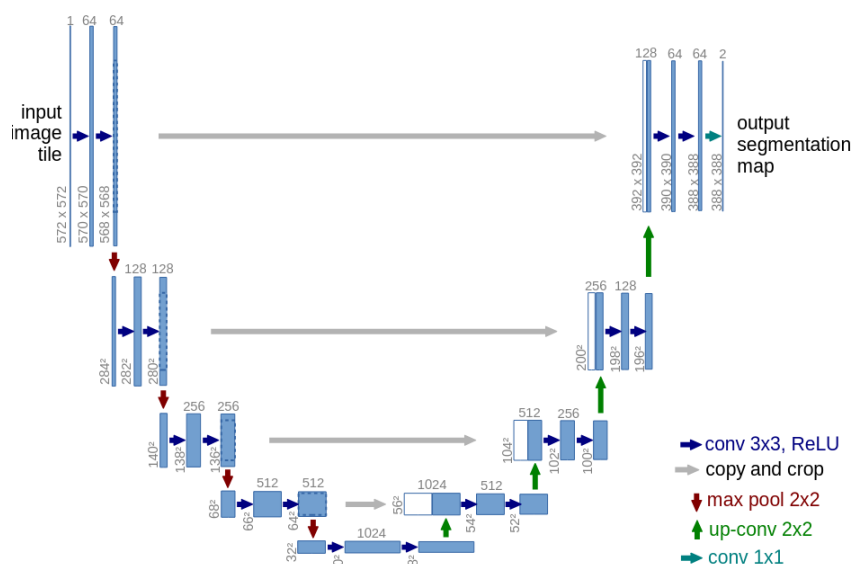


Figure 9.3: The U-Net architecture. The encoder (left) progressively halves spatial resolution while doubling feature channels. The decoder (right) upsamples back to the input resolution. Skip connections (horizontal arrows) concatenate encoder and decoder features at each resolution level, recovering fine spatial detail lost during encoding.

Semantic segmentation is evaluated using **mean Intersection over Union (mIoU)**: IoU is computed per class (comparing predicted and ground truth pixel masks for each class) and averaged across all classes.

9.3.3. Instance Segmentation

Instance segmentation goes beyond semantic segmentation by distinguishing individual object instances. Two people standing next to each other receive different instance masks even though they belong to the same class. The output is a set of per-instance binary masks, each paired with a class label and confidence score.

Panoptic segmentation unifies semantic and instance segmentation into a single output: background regions such as sky, road, and grass receive semantic labels, while foreground objects receive individual instance masks. It has become the modern standard for comprehensive scene understanding.

For practitioners, several powerful tools are freely available. **SAM** (Segment Anything Model) (Kirillov et al. 2023) is a foundation model trained on one billion masks that can segment any object in any image given a simple prompt - a point, a bounding box, or a text description - without task-specific fine-tuning. SAM has become widely used both for inference and as an annotation assistance tool, dramatically reducing the cost of creating segmentation datasets. It is available via the [Meta AI GitHub repository](#). **Detectron2** (Wu2019), also from Meta AI, is a full object detection and segmentation library implementing Mask R-CNN and many other architectures, available at [GitHub](#).

9.3.4. Annotation Cost

A practical consideration often underestimated by practitioners is annotation cost. The annotation requirements differ dramatically across tasks, as shown in Table 3.

Task	Annotation type	Time per image	Example dataset
Classification	Image-level label	Seconds	ImageNet
Detection	Bounding boxes	Minutes	COCO
Semantic segmentation	Pixel-wise labels	30–90 min	Cityscapes
Instance segmentation	Per-instance masks	Hours	COCO

Table 3: Annotation cost comparison across visual tasks.

In practice, data is almost always the bottleneck - not the model. Given a well-annotated dataset, training a competitive model is straightforward: the architectures are well understood, pretrained weights are freely available, and the training procedure is largely standardised. The hard part is getting the data right. A poorly annotated dataset will produce a poorly performing model regardless of how sophisticated the architecture is, and no amount of hyperparameter tuning will fix labels that are inconsistent, incomplete, or simply wrong.

This means that investing time and effort in data collection, cleaning, and annotation pays far greater dividends than optimising the model. Before reaching for a more complex architecture, it is worth asking whether the dataset is the real limiting factor. More data, better labels, and more careful quality control will often outperform a fancier model trained on the same flawed dataset.

Several practical strategies help manage annotation cost. **SAM** can generate high-quality segmentation masks interactively from simple point or bounding box prompts, reducing mask annotation time by an order of magnitude. **Semi-supervised and weakly supervised methods** exploit a small amount of expensive annotations combined with a large amount of cheap ones - for example using image-level labels to supervise a detection model. **Synthetic data** generated from 3D simulations or procedural pipelines provides automatically annotated training examples, particularly useful in domains where real annotations are prohibitively expensive such as autonomous driving or robotics. The common thread is to be clever about data: think carefully about what annotations are truly needed, use the cheapest label type that is sufficient for the task, and leverage tools that reduce the human effort required.

Acknowledgements

These lecture notes were prepared with the assistance of Claude, a large language model developed by Anthropic. All content has been reviewed, edited, and approved by the author.

References

- Ba, Jimmy Lei, Jamie Ryan Kiros, and Geoffrey E. Hinton (2016). “Layer Normalization”. In: *arXiv preprint arXiv:1607.06450*.
- Bartlett, Peter L., Andrea Montanari, and Alexander Rakhlin (2020). “Benign Overfitting in Linear Regression”. In: *Proceedings of the National Academy of Sciences*. Vol. 117. 48, pp. 30063–30070.
- Belkin, Mikhail et al. (2019). “Reconciling Modern Machine-Learning Practice and the Classical Bias-Variance Trade-off”. In: *Proceedings of the National Academy of Sciences* 116.32, pp. 15849–15854.
- Bengio, Yoshua, Olivier Delalleau, and Nicolas Le Roux (2004). “The Curse of Highly Variable Functions for Local Kernel Machines”. In: *Advances in Neural Information Processing Systems*.
- Choromanska, Anna et al. (2015). “The Loss Surfaces of Multilayer Networks”. In: *Proceedings of the 18th International Conference on Artificial Intelligence and Statistics*.
- Cybenko, George (1989). “Approximation by Superpositions of a Sigmoidal Function”. In: *Mathematics of Control, Signals and Systems* 2.4, pp. 303–314. doi: [10.1007/BF02551274](https://doi.org/10.1007/BF02551274).
- Dauphin, Yann et al. (2014). “Identifying and Attacking the Saddle Point Problem in High-Dimensional Non-Convex Optimization”. In: *Advances in Neural Information Processing Systems*.
- Deng, Jia et al. (2009). “ImageNet: A Large-Scale Hierarchical Image Database”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.
- Fukushima, Kunihiro (1980). “Neocognitron: A Self-Organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position”. In: *Biological Cybernetics* 36, pp. 193–202.
- Glorot, Xavier and Yoshua Bengio (2010). “Understanding the Difficulty of Training Deep Feedforward Neural Networks”. In: *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*. Ed. by Yee Whye Teh and Mike Titterton. Vol. 9. Proceedings of Machine Learning Research. PMLR, pp. 249–256.
- Glorot, Xavier, Antoine Bordes, and Yoshua Bengio (2011). “Deep Sparse Rectifier Neural Networks”. In: *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 315–323.
- Goodfellow, Ian, Yoshua Bengio, and Aaron Courville (2016). *Deep Learning*. Available at <https://www.deeplearningbook.org>. MIT Press.
- Guo, Chuan et al. (2017). “On Calibration of Modern Neural Networks”. In: *Proceedings of the 34th International Conference on Machine Learning*, pp. 1321–1330.
- He, Kaiming et al. (2015). “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification”. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 1026–1034.
- (2016). “Deep Residual Learning for Image Recognition”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778. URL: <https://arxiv.org/abs/1512.03385>.
- Hinton, Geoffrey E., Simon Osindero, and Yee-Whye Teh (2006). “A Fast Learning Algorithm for Deep Belief Nets”. In: *Neural Computation* 18.7, pp. 1527–1554.
- Hochreiter, Sepp and Jürgen Schmidhuber (1997a). “Flat Minima”. In: *Neural Computation* 9.1, pp. 1–42.

- Hochreiter, Sepp and Jürgen Schmidhuber (1997b). “Long Short-Term Memory”. In: *Neural Computation* 9.8, pp. 1735–1780.
- Hopfield, John J. (1982). “Neural Networks and Physical Systems with Emergent Collective Computational Abilities”. In: *Proceedings of the National Academy of Sciences* 79.8, pp. 2554–2558.
- Hornik, Kurt (1991). “Approximation Capabilities of Multilayer Feedforward Networks”. In: *Neural Networks* 4.2, pp. 251–257. doi: [10.1016/0893-6080\(91\)90009-T](https://doi.org/10.1016/0893-6080(91)90009-T).
- Ioffe, Sergey and Christian Szegedy (2015). “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift”. In: *Proceedings of the 32nd International Conference on Machine Learning*, pp. 448–456.
- Jumper, John, Richard Evans, Alexander Pritzel, et al. (2021). “Highly Accurate Protein Structure Prediction with AlphaFold”. In: *Nature* 596, pp. 583–589.
- Keskar, Nitish Shirish et al. (2017). “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima”. In: *International Conference on Learning Representations*.
- Kingma, Diederik P. and Jimmy Ba (2015). “Adam: A Method for Stochastic Optimization”. In: *International Conference on Learning Representations (ICLR)*. arXiv:1412.6980.
- Kirillov, Alexander et al. (2023). “Segment Anything”. In: *Proceedings of the IEEE International Conference on Computer Vision*, pp. 4015–4026.
- Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton (2012a). “ImageNet Classification with Deep Convolutional Neural Networks”. In: *Advances in Neural Information Processing Systems*. Vol. 25.
- (2012b). “ImageNet Classification with Deep Convolutional Neural Networks”. In: *Advances in Neural Information Processing Systems*. Vol. 25. URL: <https://papers.nips.cc/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html>.
- Kullback, Solomon and Richard A. Leibler (1951). “On Information and Sufficiency”. In: *The Annals of Mathematical Statistics* 22.1, pp. 79–86.
- LeCun, Yann, Yoshua Bengio, and Geoffrey Hinton (2015). “Deep Learning”. In: *Nature* 521, pp. 436–444.
- LeCun, Yann, Léon Bottou, et al. (1998a). “Gradient-Based Learning Applied to Document Recognition”. In: *Proceedings of the IEEE* 86.11, pp. 2278–2324.
- (1998b). “Gradient-Based Learning Applied to Document Recognition”. In: *Proceedings of the IEEE* 86.11, pp. 2278–2324. doi: [10.1109/5.726791](https://doi.org/10.1109/5.726791).
- Li, Xiang et al. (2019). “Understanding the Disharmony between Dropout and Batch Normalization by Variance Shift”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 2682–2690.
- Lighthill, James (1973). *Artificial Intelligence: A General Survey*. Tech. rep. Available at <https://www.aiai.ed.ac.uk/events/lighthill1973/lighthill.pdf>. Science Research Council.
- Loshchilov, Ilya and Frank Hutter (2019). “Decoupled Weight Decay Regularization”. In: *International Conference on Learning Representations*.
- Ministry of International Trade and Industry (1982). *Outline of Research and Development Plans for Fifth Generation Computer Systems*. Tech. rep. Institute for New Generation Computer Technology.
- Minsky, Marvin and Seymour Papert (1969). *Perceptrons: An Introduction to Computational Geometry*. MIT Press.

- Nair, Vinod and Geoffrey E. Hinton (2010). “Rectified Linear Units Improve Restricted Boltzmann Machines”. In: *Proceedings of the International Conference on Machine Learning*.
- Nakkiran, Preetum et al. (2020). “Deep Double Descent: Where Bigger Models and More Data Hurt”. In: *International Conference on Learning Representations*.
- Raina, Rajat, Anand Madhavan, and Andrew Y. Ng (2009). “Large-Scale Deep Unsupervised Learning Using Graphics Processors”. In: *Proceedings of the International Conference on Machine Learning*.
- Redmon, Joseph et al. (2016). “You Only Look Once: Unified, Real-Time Object Detection”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 779–788.
- Ren, Shaoqing et al. (2015). “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks”. In: *Advances in Neural Information Processing Systems*. Vol. 28.
- Ronneberger, Olaf, Philipp Fischer, and Thomas Brox (2015). “U-Net: Convolutional Networks for Biomedical Image Segmentation”. In: *Medical Image Computing and Computer-Assisted Intervention*, pp. 234–241.
- Rosenblatt, Frank (1958). “The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain”. In: *Psychological Review* 65.6, pp. 386–408.
- Rumelhart, David E., Geoffrey E. Hinton, and Ronald J. Williams (1986). “Learning Representations by Back-propagating Errors”. In: *Nature* 323, pp. 533–536.
- Shannon, Claude E. (1948). “A Mathematical Theory of Communication”. In: *The Bell System Technical Journal* 27.3, pp. 379–423.
- Silver, David, Aja Huang, Chris J. Maddison, et al. (2016). “Mastering the Game of Go with Deep Neural Networks and Tree Search”. In: *Nature* 529, pp. 484–489.
- Simonyan, Karen and Andrew Zisserman (2015). “Very Deep Convolutional Networks for Large-Scale Image Recognition”. In: *International Conference on Learning Representations*. URL: <https://arxiv.org/abs/1409.1556>.
- Srivastava, Nitish et al. (2014). “Dropout: A Simple Way to Prevent Neural Networks from Overfitting”. In: *Journal of Machine Learning Research* 15.1, pp. 1929–1958.
- Szegedy, Christian et al. (2016). “Rethinking the Inception Architecture for Computer Vision”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2818–2826.
- Vapnik, Vladimir (1995). *The Nature of Statistical Learning Theory*. Springer.
- Vaswani, Ashish et al. (2017). “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30.
- Werbos, Paul (1974). “Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences”. PhD thesis. Harvard University.
- Widrow, Bernard and Marcian E. Hoff (1960). “Adaptive Switching Circuits”. In: *IRE WESCON Convention Record*. Vol. 4, pp. 96–104.
- Zeiler, Matthew D. and Rob Fergus (2014). “Visualizing and Understanding Convolutional Networks”. In: *European Conference on Computer Vision*, pp. 818–833. URL: <https://arxiv.org/abs/1311.2901>.